A Fast Algorithm for Elliptic-Curve Discrete Logarithms

Kaarel Kaarelson
Independent researcher
kaarel.kaarelson@gmail.com

November 1, 2026 (version 2)

Abstract

On October 31, 2026, five days after OpenAI refuted ETH, we found the first algorithm for general elliptic-curve discrete logarithms over prime fields that is asymptotically faster than the square root. We show there exists a Las Vegas algorithm that beats Pollard's rho method for $m \geq 5$, which has been the best-known bound for 40 years in elliptic-curve cryptography. In 2004, Semaev showed that if decompositions of points could be found quickly, then index calculus would beat $\sqrt{p}$, but nobody has found a fast method in 22 years. Today, we finally have the answer. Using the counting engine behind Anthropic's $2^{o(n)}$-time algorithm for 3-SAT, we can deterministically count and find in $p^{o(1)}$ time decompositions $R = P_1 + \cdots + P_m$ of a point on an elliptic curve over $\mathbb{F}_p$ into points whose x-coordinates are below $p^{1/m}$. Combining this with Semaev's index calculus, our algorithm solves the discrete logarithm problem on every elliptic curve over $\mathbb{F}_p$, of any group order, in expected time $p^{2/m+o(1)}$. Letting m grow with p gives time $p^{o(1)}$, conditional on the published statements of the counting engine. Machine-checked in Lean 4: the decomposition search and its counting form for x-strings with sign bits, the counting engine they run on, the group law as integer constraints, the combinatorial core of the index calculus, and the arithmetic of the crossover schedules.

2020 Mathematics Subject Classification. 11Y16, 11G20, 94A60, 68Q25.

Contents

1 Introduction **4**

2 Preliminaries **8**
2.1 Notation and model of computation 9
2.2 Networks 10
2.3 Evaluation 11

3 The decomposition network **13**
3.1 Construction 13
3.2 Size bounds 14
3.3 The edge network and Proposition N 16

4 Fast decomposition **21**

5 Index calculus and coverage **24**
5.1 From decompositions to discrete logarithms 24
5.2 Summation polynomials 25
5.3 Isomorphic models and the second moment 25
5.4 Discrete logarithms for prime or squarefree order 28

6 Arbitrary group orders **30**
6.1 What goes wrong, and the fix 30
6.2 Algorithm 30
6.3 Lemmas 31
6.4 Every curve: exponent $3/m$ 32
6.5 Squarefree exponent or $m \leq 8$: exponent $2/m$ 32
6.6 Every curve, every m, by a planted diagonal 34

7 Growing m: time $p^{o(1)}$ **37**
7.1 The node cost 37
7.2 The uniform engine bound 38
7.3 Theorem 7 39

8 Formal verification **41**
8.1 The main Lean theorem 41
8.2 The counting theorem 42
8.3 What is proved 43
8.4 What is not in Lean 44
8.5 Reproducibility 45

9 Related work **45**

10 Limitations **50**

11 Conclusion and open problems **56**

Appendix A: Smaller networks (X and V8x) **58**

Appendix B: Genus 2, on average over curves **60**

Acknowledgments and declaration of AI use **61**

References **62**

1 Introduction

The problem. Let E be an elliptic curve over a prime field $\mathbb{F}_p$, let $P \in E(\mathbb{F}_p)$ and let $Q \in \langle P \rangle$. The elliptic-curve discrete logarithm problem (ECDLP) asks for an integer d with $dP = Q$. Elliptic-curve cryptography, proposed by Miller [1] and Koblitz [2], rests on the hardness of this problem for curves of prime order.

For general curves of prime order the best algorithm known is Pollard's rho method [3, 4, 5], which uses heuristically $O(\sqrt{r})$ group operations for P of prime order r, that is $p^{1/2+o(1)}$ for a curve of prime order; baby-step giant-step [6] achieves this bound rigorously. In the generic group model this is optimal: Nechaev [7] and Shoup [8] proved that an algorithm that uses only the group operations needs $\Omega(\sqrt{r})$ of them. An algorithm that beats rho must therefore read the coordinates of the points.

Some special families are easy. Anomalous curves ($|E(\mathbb{F}_p)| = p$) fall in polynomial time to the attacks of Semaev [9], Satoh and Araki [10] and Smart [11], and curves of small embedding degree to the pairing reductions of Menezes, Okamoto and Vanstone [12] and Frey and Rück [13]. Algorithms with precomputation, such as those of Bernstein and Lange [14, 15], work in a different model: after a group-dependent table is computed, each instance costs about $p^{1/3}$.

For the multiplicative group $\mathbb{F}_p^*$, index calculus [16] beats the square root: small integers factor over a base of small primes often enough, and the resulting relations are solved by linear algebra in subexponential time. For elliptic curves over $\mathbb{F}_p$ there is no analogue of factoring into small primes. Outside the special families above [4, 5], no uniform algorithm without precomputation has been known that solves ECDLP over prime fields in time $p^{1/2-\varepsilon}$ for any fixed $\varepsilon > 0$, on any infinite family of curves of prime order. To our knowledge no such algorithm had even been claimed heuristically (Section 9).

Semaev's question. Semaev [17] proposed an index calculus for $E(\mathbb{F}_p)$. Take as factor base the points whose x-coordinate, read as an integer in $[0, p)$, is below about $p^{1/n}$. A relation is a decomposition $R = P_1 + \cdots + P_n$ of a target point into factor-base points; Semaev's summation polynomial f_{n+1} vanishes at $(x(P_1), \ldots, x(P_n), x(R))$ exactly when such a decomposition exists over the algebraic closure. Semaev observed that if a bounded solution can be found in $t_{p,n}$ operations, the discrete logarithm costs essentially $t_{p,n}\, p^{1/n+\delta} + p^{2/n+2\delta}$, and that for $n \geq 5$ this "may be less than $O(p^{1/2})$" even for some exponential $t_{p,n}$ [17, pp. 5–6]. Two steps have been open since 2004: finding bounded solutions of $f_{n+1} \equiv 0 \pmod{p}$ quickly, and proving that enough points have one.

Over extension fields $\mathbb{F}_{q^n}$ the corresponding decomposition problem has algebraic structure (a Weil descent to a polynomial system over $\mathbb{F}_q$). Gaudry [18] used it to obtain, heuristically, an index-calculus algorithm that beats generic methods for fixed n as $q \to \infty$; Diem [19] proved a bound of the same kind rigorously for fixed n, and Diem [20, 21] gave rigorous subexponential algorithms when n grows suitably with q. The many later algorithms based on Gröbner bases rest on heuristics about the cost of solving the systems (see the survey of Galbraith and Gaudry [4] and Section 9). None of these methods applies to prime fields, where the factor base is an interval with no algebraic structure.

The October 2026 engine. On October 26, 2026, Anthropic [22] and OpenAI [23] independently posted preprints claiming deterministic algorithms that count the satisfying assignments of k-CNF formulas in time $2^{o(n)}$, which would refute the Exponential Time Hypothesis. Neither has been refereed. Rudich and Rousseau [24] used the Anthropic machinery to factor integers

deterministically in time $N^{o(1)}$: they count the divisors of N with a prescribed binary prefix with a network of multiplication checks, and recover the least prime factor bit by bit from the counts.

The core of the Anthropic paper is a counting engine (Section 2). It evaluates the value of a "network" (a sum over settings of nodes of products of local weights and pairwise interactions that factor through few bits) in time $2^{o(U)}$, where U is the total number of bits the nodes share, provided the network can be split into many clusters of small size. Its cost is governed by a sequence of exponents $e_\ell \to 0$, one for each level ℓ of a recursion; how fast e_ℓ decays is measured by a rate exponent $\alpha' > 0$ (Section 2; $\alpha' = \log_2(12/11)$ from the main text of [22], $\alpha' = 1/2$ from its Appendix B, which has a longer proof).

This paper. Throughout, $L = \lceil \log_2 p \rceil$, $m \geq 2$ is the number of summands, and a decomposition $R = P_1 + \cdots + P_m$ is *generic* if every partial addition $(P_1 + \cdots + P_{k-1}) + P_k$ adds two affine points with distinct x-coordinates (Definition 3.1). The *x-string* of a decomposition is the concatenation of the binary expansions of $x(P_1), \ldots, x(P_m)$, each followed by the sign bit of $y(P_i)$.

Theorem 1 (main theorem)
For every fixed $m \geq 2$ there is a Las Vegas algorithm that, given a prime $p > 3$ with $\lfloor p^{1/m} \rfloor \geq \max(12m, 300)$, a nonsingular curve $E : y^2 = x^3 + Ax + B$ over $\mathbb{F}_p$ of any group order and structure, $P \in E(\mathbb{F}_p)$ and $Q \in \langle P \rangle$, finds d with $dP = Q$, always correctly, in expected time $p^{2/m + o(1)}$. In particular it is asymptotically faster than Pollard's rho method for $m \geq 5$.

Theorem 1 is proved in Section 6.

Theorem 2 (fast decomposition)
For every fixed $m \geq 2$ there is a deterministic algorithm that, given a prime $p > 3$, a nonsingular curve $y^2 = x^3 + ax + b$ over $\mathbb{F}_p$ and an affine point R, outputs the lexicographically least x-string with sign bits of a generic decomposition $R = P_1 + \cdots + P_m$ with $x(P_i) < \lceil p^{1/m} \rceil$, or reports that there is none, in time $C_k 2^{L/k}$ for every $k \geq 1$, that is $p^{o(1)}$. This search statement is the Lean 4 theorem `subexponentialRate_all`. The counting form, which outputs the number of such decompositions with a given prefix in the same time, is the Lean 4 theorem `countingRate_all`.

Theorem 2 is proved in Section 4.

Theorem 3 (enough points decompose)
Let $E : y^2 = x^3 + Ax + B$ be any nonsingular curve over $\mathbb{F}_p$, let $m \geq 2$, $p \geq 100$, $\kappa \geq 1$ and $X = \lfloor p^{1/m} \rfloor \geq 12m$. For a uniformly random isomorphic model $E_u : y^2 = x^3 + u^4 Ax + u^6 B$, $u \in \mathbb{F}_p^*$, with probability at least $1 - 1/\kappa - 56/X$ a fraction at least $c_m(\kappa) = 2^{-m \log_2 m - O(m)}/\kappa$ of all points of $E(\mathbb{F}_p)$ have a generic decomposition on E_u into points with x-coordinate in $[1, X]$.

Theorem 3 is proved in Section 5.

The algorithm of Theorem 1 is an index calculus. Theorem 3 supplies enough relations, Theorem 2 finds each one, and the points follow from their x-strings by a Las Vegas square root. To make the relation matrix invertible for every group structure, the algorithm plants a random multiple of each factor-base point on its diagonal (Section 6).

Earlier versions. Theorem 1 was reached through three weaker results with independent proofs, all with the same oracle: expected time $p^{2/m+o(1)}$ when $E(\mathbb{F}_p)$ is cyclic of prime or squarefree order (Theorem 4, Section 5), $p^{3/m+o(1)}$ for every curve (Theorem 5), and $p^{2/m+o(1)}$ when the exponent of $E(\mathbb{F}_p)$ is squarefree or $m \le 8$ (Theorem 6, Section 6).

Letting m grow slowly with p gives expected time $p^{o(1)}$, conditional on (E) and on Proposition N (Section 7).

Theorem 7 (growing m)
Assume (E) and (N). There is a Las Vegas algorithm that, for all sufficiently large p, given a nonsingular curve $E : y^2 = x^3 + Ax + B$ over $\mathbb{F}_p$ of any group order and structure, $P \in E(\mathbb{F}_p)$ and $Q \in \langle P \rangle$, finds d with $dP = Q$, always correctly, in expected time $2^{O(L(\log L)^{-\alpha'/2})} = p^{o(1)}$, where $\alpha' > 0$ is the rate exponent of the engine. The implied constant is meaningful only in the word-operation model of (E); in the formal model it holds up to a machine constant (Section 7).

Assumptions and verification. The search in Theorem 2 (part (ii), x-string with sign bits) is a Lean theorem with no hypotheses: the engine it uses is formalized in Lean, not assumed. Theorems 1 and 3 are paper proofs, independently reviewed and checked numerically. Theorem 1 uses Theorem 2 as its decomposition oracle and Theorem 3 uses no engine, so neither depends on (E), the statements of the Anthropic paper that we use, which Section 2 quotes in full. (E) is used only for the deterministic search over the y-digits, which recovers the points without square roots, and for Theorem 7. The counting form of Theorem 2 (part (i)) is now also a Lean theorem. Theorem 3 uses no heuristic; it replaces the usual assumption that a random point decomposes with probability about $1/m!$, and it covers $A = 0$ through the repair of Lemma N$'$. Theorem 1 likewise uses no heuristic about the rank of the relation matrix. Sections 8 and 10 give the verification status of each part.

For $m \ge 5$ the exponent $2/m$ is below $1/2$, so for every $m \ge 5$ the algorithm is asymptotically faster than Pollard's rho method on every curve; at $m = 8$ the exponent is $1/4$. The constants are enormous. With the recursion of the Anthropic engine, the guarantee cannot beat rho unless $\log_2 \log_2 p \ge 2.014 \cdot 10^9$ for the smallest network we can use (V8x, whose value is checked numerically and whose size bounds are extrapolated beyond $L = 2^{400}$, not proved), unless $\log_2 \log_2 p \ge 5.154 \cdot 10^{11}$ for the compact network (value proved on paper), and unless $\log_2 \log_2 p \ge 6.756 \cdot 10^{16}$ for the edge network (value and sizes proved in Lean). A hand-audited block clustering of V8x lowers the first figure to $1.693 \cdot 10^8$ under one further audited assumption, and to $2.52 \cdot 10^8$ with the schedule of the Anthropic paper itself (Section 10). So p must have far more than 2^{10^8} bits. The results have no consequence for any curve in use.

Table 1 places these results among the known algorithms.

Ideas. Two observations make this work.

Averaging over isomorphic models replaces the decomposition heuristic. The standard analysis of index calculus [4] assumes that a random point decomposes with probability about $1/m!$. We prove a lower bound on that probability instead. Moment methods give it on average over curves, but a fixed curve is a single point of that probability space. The way around this is to average over $u \in \mathbb{F}_p^*$ in the models $E_u : y^2 = x^3 + u^4Ax + u^6B$ of one fixed curve, all isomorphic to E by $(x, y) \mapsto (u^2x, u^3y)$. The factor base $\{x < p^{1/m}\}$ on E_u is a dilated box on E. We bound the m-fold additive energy of the factor base on average over u: a relation among $2m$ points with prescribed x-coordinates t holds on E_u only if u^{-2} is a root of the polynomial

Method	Curves	Time	Proof
Generic algorithms, any field			
Pollard rho [3]	all	$O(\sqrt{r})$	heuristic
Baby-step giant-step [6]	all	$O(\sqrt{r})$	rigorous
Prime fields $\mathbb{F}_p$			
Semaev; Satoh–Araki; Smart [9, 10, 11]	anomalous only	polynomial	rigorous
MOV; Frey–Rück [12, 13]	small embedding degree	subexponential	rigorous
Summation polynomials [17, 25, 5]	all	none below $\sqrt{p}$	–
This work (Theorem 1)	**all**	$\boldsymbol{p^{2/m+o(1)}}$	**rigorous**
Extension fields $\mathbb{F}_{q^n}$			
Gaudry [18]	all, fixed n	$\tilde{O}(q^{2-2/n})$	heuristic
Diem [19]	all, fixed n	$\tilde{O}(q^{2-2/n})$	rigorous
Diem [20, 21]	all, n growing	subexponential	rigorous

Table 1: Algorithms for the ECDLP, without precomputation. r is the order of P; for a curve of prime order over $\mathbb{F}_p$, $\sqrt{r} = p^{1/2+o(1)}$. Our bound beats rho for $m \geq 5$; it is asymptotic and takes effect only when $\log_2 \log_2 p \geq 2.014 \cdot 10^9$ for V8x, $5.154 \cdot 10^{11}$ for the compact network and $6.756 \cdot 10^{16}$ for the edge network, the one proved in Lean (Section 10 gives the status of each figure). "Rigorous" for this work means: the decomposition step is a Lean theorem and the rest are paper proofs. Section 9 gives details.

$s \mapsto f_{2m}(st; A, B)$, which we prove is nonzero by a Kummer-theory argument over the function field $\overline{\mathbb{F}}_p(s)$ (Lemma N′). Cauchy–Schwarz then gives, for most u, a constant fraction of points with a decomposition (Theorem 3). Since the algorithm can pick u at random and the decomposition oracle accepts any model, this bound applies to every curve. The relation matrix needs no rank heuristic either: the sampling tags are uniform given the sampled points, which makes the final linear combination uniform.

The decomposition check holds $O(m \log p)$ bits. The engine is fast only when the total shared information U and the cluster size P satisfy $U = O(mL)$ and $gP = O(mL)$. A factor $\log L$ in U would destroy the $2^{o(L)}$ bound, since U is multiplied by $e_\ell \asymp (\log L)^{-\alpha'}$ with $\alpha' < 1$; the term gP enters only divided by the clique size, which tolerates such factors (Section 4). A decomposition of R into m points is witnessed by $O(m)$ field elements: the coordinates of the points, the slopes and partial sums of the $m - 1$ additions, and one quotient for each reduction modulo p. Each of the $O(m)$ resulting integer identities is checked by one Schönhage–Strassen [26] gadget. A gadget evaluates the identity in a transform domain of length $T \approx \sqrt{L}$, so that every node holds $O(L^{3/4})$ bits and every edge carries one ring element. Summed over the $O(m)$ gadgets this gives $U = O(mL)$, and the cluster bound holds for $g = O(L^{1/4})$ clusters (Section 3). The free bits of the problem are the m x-coordinates of about L/m bits each, so prefix search for the x-string costs $L + O(m)$ engine calls, and mL more give the y-coordinates.

The network is an adaptation of the template of Rudich and Rousseau, whose network checks the single product $de = N$. What is new is the m-fold composition of the group law, the reduction modulo p by explicit quotients, the range checks, and the size analysis that keeps U and gP at $O(mL)$ through m chained group-law checks.

Organization of the paper. Section 2 fixes notation and the model of computation and states, in full, what we use from the engine papers. Section 3 builds the decomposition networks: the edge network, whose value and size bounds are proved in Lean, and a hand-optimized network with explicit small constants. Section 4 proves Theorem 2. Section 5 proves Theorems 3 and 4. Section 6 extends Theorem 4 to every group order (Theorems 5 and 6, and the main theorem, Theorem 1). Section 8 reports what is formally verified in Lean 4. Section 9 discusses related work, Section 10 lists limitations, and Section 11 summarizes the results and open problems. Appendix A contains two smaller networks and Appendix B an extension to genus 2. The section *Acknowledgments and declaration of AI use*, at the end of the paper, describes how the work was done, including the AI models used and the dates.

Changes from version 1. This version adds Theorem 7 (Section 7), which was not in version 1, with its proof revised after an independent review. The counting form of Theorem 2 is now a Lean theorem, and so is more of the index calculus behind Theorem 1, including Lemma gen for every finite abelian group, so that the Lean reduction no longer assumes that $E(\mathbb{F}_p)$ is generated by two points. The new Lean modules are in the folder `v2/` of the repository (Section 8.5). The base case of the Engine Theorem is replaced by an elimination along the line graph of the clique (Lemma 10.1), which lowers the best crossover to $\log_2 \log_2 p \geq 6.292 \cdot 10^7$, or $4.372 \cdot 10^7$ with a free schedule (Figure 1); the arithmetic of these schedules is checked in Lean. Section 11 adds what a practical algorithm would need and the routes we tested towards one. The abstract has a new sentence on Theorem 7 and one on what is machine-checked. The network of Section 3.1 is repaired in three places and its value is proved on paper. The network V7 of version 1 counted a slightly different set; Appendix A replaces it with two repaired networks, X and V8x, which need no hypothesis on primes in progressions, and Hypothesis H of version 1 is removed. The constant c of the pieces of [22, Appendix B] is made explicit ($c = 12$), which justifies $x_0 = 18$, and the constants of the Anthropic Lean proof are traced ($C_1 = 44$, $C_0 = 300c + 3615$). The engine condition $2^{x_0} \geq c + 5$ is corrected (version 1 printed $2x_0$). The cluster bound of the compact network and of X and V8x now holds for every g. The main crossover estimate of Section 10 improves from $\log_2 \log_2 p \gtrsim 5.15 \cdot 10^{11}$ to $2.014 \cdot 10^9$ for V8x, but that figure rests on size bounds for V8x that are checked up to $L = 2^{400}$ and extrapolated beyond, and on a network value that is checked numerically and not proved; the edge network, whose value is a Lean theorem, has $6.756 \cdot 10^{16}$. A block clustering of V8x for case (ii) of Lemma 8 lowers the V8x figure to $1.693 \cdot 10^8$ (under the hand-audited assumption (R1)) and $2.52 \cdot 10^8$ (with the schedule of the Anthropic paper). The conclusion lists routes that did not give a further improvement.

2 Preliminaries

This section fixes notation and the model of computation (Section 2.1) and states the results of the Anthropic paper [22] that we use. We call them (E). The paper is an unrefereed preprint; its authors report a Lean 4 formalization. We use only the network-evaluation engine: Lemma 8 and parts (a) and (b) of Theorem A of [22], which we call the Engine Lemma and the Engine Theorem; to avoid a clash with our own numbering, we never write "Theorem A" for the latter. We use neither the reduction from 3-SAT nor the OpenAI paper, and from Rudich and Rousseau [24] we use the design pattern (Schönhage–Strassen multiplication checks and search from prefix counts) and, in the Lean pipeline of Section 8 on which the oracle of Theorems 1 and 4–6 rests, their Lean proof of the engine's correctness and running time. We do not use their factoring theorem,

except in Section 6 where any $N^{o(1)}$ factoring algorithm will do.

2.1 Notation and model of computation

Notation. Throughout, $p > 3$ is prime, curves are in short Weierstrass form with nonzero discriminant, and elements of $\mathbb{F}_p$ are identified with integers in $[0, p)$. Table 2 lists the symbols used in more than one section. The letter P denotes a point in the discrete-logarithm sections and a node size in the network sections; the context makes the meaning clear.

Curves and decompositions	
L	$\lceil \log_2 p \rceil$, the bit length of p
m	number of summands, fixed, $m \geq 2$
E, E_u	the curve $y^2 = x^3 + Ax + B$; its isomorphic model $y^2 = x^3 + u^4Ax + u^6B$
B_x	decomposition bound: $x(P_i) < B_x = \lceil p^{1/m} \rceil$
X	coverage bound: $x(P_i) \in [1, X]$, $X = \lfloor p^{1/m} \rfloor$
generic	every partial addition adds two affine points with distinct x (Def. 3.1)
x-string	the bits of $x(P_1), \ldots, x(P_m)$, each followed by the sign bit of $y(P_i)$
Networks and the engine	
U, P, g	total shared bits; bits per node or cluster; number of clusters (Def. 2.2)
τ	cost of one local evaluation
c_1, c_2	constants with $U \leq c_1 mL$ and $gP \leq c_2 mL$
(E), (N)	the engine statements we use; the network size bounds
Discrete logarithms	
P, Q, d	base point, target, $Q = dP$
G, N	the group $E(\mathbb{F}_p)$ and its order (n in Section 5)
$c_m(\kappa)$	the coverage constant of Theorem 3

Table 2: Notation used across sections. Symbols used in one section only are defined there.

Model of computation. Three cost measures appear in the paper.

- The Lean theorem behind Theorem 2(ii) (Section 8) counts steps of a Turing machine in Mathlib's TM2 model (finitely many stacks, finite alphabets) on the binary encoding of the input (p, a, b, x_R, y_R, π). Its bound is worst case over all inputs of a given length ℓ. On reduced inputs $(a, b, x_R, y_R < p)$ with $|\pi| \leq m(b_x + 1)$, $L \leq \ell \leq 6L + O(m)$, so $\ell = \Theta(L)$.
- The engine of Section 2, and hence the y-digit extension of Theorem 2 and Theorem 7, counts word operations on words of length about the address length [22], that is $O(L)$ bits for the networks used here.
- Theorems 1 and 4–6 count bit operations of a random-access machine that can also draw independent uniform random bits. An operation in $\mathbb{F}_p$ or in $E(\mathbb{F}_p)$ costs $\mathrm{poly}(L)$ bit operations.

A random-access machine simulates one Turing-machine step, and one operation on a word of $O(L)$ bits, with $\mathrm{poly}(L)$ bit operations. So the three measures agree up to factors $\mathrm{poly}(L) = p^{o(1)}$,

which the $o(1)$ in every exponent absorbs; in particular the Lean bound gives the oracle (O) in time $p^{o(1)}$ in the model of Theorems 1 and 4–6. We never convert in the other direction, from a random-access bound to a Turing-machine bound, which can cost a polynomial in the running time, and we do not claim the bounds of Theorems 1 and 4–6 for Turing machines.

Running time is worst case over inputs; for Theorems 1 and 4–6, *expected time* is the expectation over the algorithm's random bits, for the worst input. *Las Vegas* means that the output is always correct: every output is checked, and only the running time is random. In every statement m is fixed and $p \to \infty$.

- *Time* $p^{o(1)}$ means: for every $\delta > 0$ there is a constant $C_{m,\delta}$ such that the time is at most $C_{m,\delta}\, p^{\delta}$ on every input. In Theorem 2 this is the form $C_k 2^{L/k}$ for every $k \geq 1$ (Lean: $C_k 2^{\ell/k}$, and $2^{\ell/k} \leq 2^{O(m)/k} p^{6/k}$ on reduced inputs).
- *Expected time* $p^{2/m+o(1)}$ means: for every $\delta > 0$ there is a constant $C_{m,\delta}$ such that for every prime p, every curve over $\mathbb{F}_p$ and every instance (P, Q) satisfying the hypotheses of the theorem, the expected time is at most $C_{m,\delta}\, p^{2/m+\delta}$.

The constants depend on m and δ only, not on the curve or the instance: the proofs of Theorems 1 and 4–6 bound the success probability of a round below by a function of L alone, and the cost of a round by a function of p and m. They are not explicit, because the constants C_k of Theorem 2 are not (Section 10).

2.2 Networks

Definition 2.1 ([22, Definition 1])
Let X, X' be finite sets and $c \geq 0$. A function $F : X \times X' \to \mathbb{Z}$ *factors through c bits* if there are $f : X \times \{0,1\}^c \to \mathbb{Z}$ and $f' : X' \times \{0,1\}^c \to \mathbb{Z}$ with $F(z, z') = \sum_{\theta \in \{0,1\}^c} f(z, \theta) f'(z', \theta)$ for all z, z'.

The equality test $[\pi(z) = \pi'(z')]$ of two c-bit strings factors through c bits, with $f(z, \theta) = [\pi(z) = \theta]$. All interactions in our networks are of this form.

Definition 2.2 ([22, Definition 2])
Let $P, B, \tau \geq 1$. A *network* on a finite node set J with parameters P, B, τ consists of: for each node a a set $S_a \subseteq \{0,1\}^{P_a}$ of settings, $P_a \leq P$, and a weight $W_a : S_a \to \mathbb{Z}$; for each unordered pair $\{a, b\}$ a symmetric $\Psi_{a,b} : S_a \times S_b \to \mathbb{Z}$, an integer $c_{a,b} \geq 0$ and a factorization of $\Psi_{a,b}$ through $c_{a,b}$ bits; such that (i) $\sum_{b \neq a} c_{a,b} \leq P$ for every a; (ii) all values of W_a, $\Psi_{a,b}$ and of the factorizations are below 2^B in absolute value; (iii) membership in S_a and all these values are computable in τ operations. Its *value* is

$$Z = \sum_{z \in \prod_a S_a} \prod_{a \in J} W_a(z_a) \prod_{\{a,b\}} \Psi_{a,b}(z_a, z_b),$$

and its *total shared information* is $U = 2 \sum_{\{a,b\}} c_{a,b} \leq |J| P$.

A network on $[g]$ is a network whose nodes have been merged into g clusters; P then bounds both the bits held by a cluster and the bits it shares. The value is an exact integer, and costs are counted in word operations with words of length about the address length [22].

The engine reduces every network to the following special network, the clique, and evaluates cliques by recursion. We quote the definitions in full.

Definition 2.3 ([22, Definition 3])
Let $K \geq 2$ and $\lambda, B \geq 1$ be integers. The (K, λ)-*clique with tables* $W_1, \ldots, W_K$ is the following network on $[K]$. A setting z_w of node w is a $(K-1)$-tuple of strings $z_w(w') \in \{0,1\}^\lambda$, one for each $w' \in [K] \setminus \{w\}$, and $S_w = (\{0,1\}^\lambda)^{K-1}$. For $w \neq w'$ and $\theta \in \{0,1\}^\lambda$,

$$c_{w,w'} = \lambda, \qquad \Psi_{w,w'}(z_w, z_{w'}) = [z_w(w') = z_{w'}(w)], \qquad f_{w,w'}(z_w, \theta) = [z_w(w') = \theta].$$

The *tables* $W_w : S_w \to \mathbb{Z}$ are arbitrary functions with $|W_w| < 2^B$; we say the tables are *bounded by* 2^B. They are not written out: an algorithm queries them by naming w and z_w and receives $W_w(z_w)$ at the cost of one operation. The *value of the clique* is its value as a network (Definition 2.2): the sum, over all choices of one string in $\{0,1\}^\lambda$ per edge $\{w, w'\}$, of the product over w of W_w evaluated at the strings on the edges at w.

Definition 2.4 ([22, Definition 7])
Let $\frac{1}{2} \leq \rho < 1$ be real and $c \geq 1$ an integer. *Pieces with parameters* (ρ, c) consist, for every $\lambda \geq 1$, of an integer $d \geq 1$, a finite set I of integers and functions $A_i : (\{0,1\}^\lambda)^4 \to \mathbb{Z}$, $i \in I$, such that: (a) $\sum_{i \in I} A_i(x, y, u, v) = d\,[x = v][y = u]$; (b) A_i factors through c_i^{row} bits as a function of (x, y) and (u, v), and through c_i^{col} bits as a function of (x, u) and (y, v), where $c_i^{\mathrm{row}} + c_i^{\mathrm{col}} \leq 4\rho\lambda + c \log_2(2\lambda)$; (c) $|I| \leq (2\lambda)^c$; the number d, the elements of I and all values of the A_i and of their factorizations are below $2^{(2\lambda)^c}$ in absolute value; and one algorithm computes each of them, and I, c_i^{row}, c_i^{col}, from λ, i and the arguments within $2^{c\lambda}$ operations.

Pieces are used only inside the proof of the Engine Theorem; we need them only to state its hypothesis.

2.3 Evaluation

The engine has two parts. The first reduces a network to a clique problem; the second solves clique problems with a recursion whose exponent e_ℓ tends to 0.

Engine Lemma ([22, Lemma 8])
Let $K, g \geq 2$ be integers and $0 < \zeta \leq 1$. Take a network on $[g]$ with parameters P, B, τ and total U, and assume that (i) $g = 3K^8$ and $\zeta = 1/(2K)$, or (ii) K divides g, $c_{a,b} \leq P/g$ for all $a \neq b$, and $2\zeta^2 g \geq K \ln(2gK)$. Put $L_{\mathrm{cl}} = \lceil (U + 2\zeta g P)/K^2 \rceil$ and $B' = \lceil 2gP/K \rceil + g^2 B$. If an algorithm outputs the value of every (K, L_{cl})-clique with tables bounded by $2^{B'}$ within T operations, then the value of the network is computed within $K^g + T \cdot 2^{2gP/K} \left(\tau\, g(P+B)\right)^{C_1}$ operations, by one procedure, the same for all K, g, ζ and all networks, which calls that algorithm.

Here C_1 is an absolute constant: Appendix A of [22], where Lemma 8 is stated, fixes the convention that "C_1 is an absolute constant". We use only case (i). If the exponent C_1 applies to $g(P+B)$ alone and τ enters as a factor, every bound below is unchanged, since only $\log_2$ of this factor enters and both readings give $O(\log_2(\tau\, g(P+B)))$.

Engine Theorem ([22, Theorem A])
Suppose there are pieces with parameters (ρ, c) and let $\alpha' = \log_2(1/\rho)$. Let $x_0 \geq 18$ be an integer with $2^{x_0} \geq c + 5$, and for $\ell \geq 0$ let

$$K_0 = 2^{x_0}, \quad \zeta_\ell = 2^{-\ell-5}, \quad g_\ell = 2^{3\ell+x_0} K_\ell, \quad K_{\ell+1} = g_\ell^2, \quad e_0 = \tfrac{1}{2}, \quad e_{\ell+1} = (\rho + 3\zeta_\ell) e_\ell + 3/K_\ell.$$

(a) $\log_2 K_\ell = (3x_0+6)2^\ell - 6\ell - 2x_0 - 6$, so $K_\ell < K_{\ell+1} \le K_\ell^{10}$; and $\frac{1}{2}\rho^\ell \le e_\ell \le 0.85\rho^\ell$, where $\rho^\ell \le \big((3x_0+6)/\log_2 K_\ell\big)^{\alpha'}$.

(b) One deterministic algorithm, given $\ell \ge 0$, $\lambda, B \ge 1$ and query access to tables bounded by 2^B, outputs the value of the (K_ℓ, λ)-clique within $2^{e_\ell K_\ell^2 \lambda}(K_\ell(\lambda+B))^{C_0 K_\ell^2}$ operations.

The g_ℓ of the Engine Theorem is internal to its recursion. Everywhere else g is the number of clusters in the Engine Lemma, $g = 3K^8$.

The paper provides pieces with $\rho = 11/12$ in its main text ($\alpha' = \log_2(12/11) \approx 0.1255$) and with $\rho = 1/\sqrt{2}$ in its Appendix B ($\alpha' = 1/2$, with a longer proof). The constant C_0 depends only on c; no numerical value is given. The paper gives no numerical value of c for the pieces of Appendix B either, since c depends on an unnamed constant in the Hardy–Ramanujan bound for the partition function. With explicit partition bounds (the upper bound $p(n) < e^\pi\sqrt{2n/3}$ from the generating function, and the lower bound from exact values of $p(n)$ for $n \le 16000$ together with Lehmer's remainder estimate as stated by DeSalvo and Pak [27]) the clauses of Definition 2.4 hold with $c = 12$. So $x_0 = 18$ is admissible, since it needs only $c \le 2^{18} - 5$. In the Anthropic Lean development the corresponding constant is far larger, because its partition lower bound has an inexplicit constant; with a sharper base case and the bound $2^k \le p(n)$ for $k(k+1) \le n$, both proved in Lean, its formula for c gives $c \le 137000$, which also admits $x_0 = 18$. The running-time clause of the pieces is checked only to the extent that it holds for any cost exponent up to about $3 \cdot 10^4$. The authors stress that (b) is needed in this uniform form, "because K grows with n" [22]. Our use is the same: K grows with p.

Hypothesis (E)
The Engine Lemma and parts (a) and (b) of the Engine Theorem hold as stated, and pieces with some fixed parameters (ρ, c), $\rho < 1$, exist (Definition 2.4).

Formal status of (E). In the Anthropic Lean development the uniform recursion is the theorem `recursionSolves`, whose statement quantifies the constant C_0 existentially and depends on c only; the grouping step corresponding to the Engine Lemma appears in the paper index with caveats on state sizes and unary parameters. We have read these statements but have not rebuilt that development. Rudich and Rousseau re-prove in Lean the part of the engine their factoring statement needs, and import nothing from the Anthropic development [24]. Our own Lean pipeline (Section 8) builds on their re-proof. Its final theorem has no hypotheses, so for the search over the x-string with sign bits (E) is discharged in Lean. (E) remains the hypothesis of the deterministic y-digit extension of Theorem 2 (both parts, once the prefix goes beyond the sign bits) and of Theorem 7. The counting statement Theorem 2(i) for prefixes over the x-string with sign bits is a Lean theorem and does not use (E) (Section 8). The ECDLP algorithms of Theorems 1 and 4–6 use only the Lean-proved search, so they do not depend on (E).

Exact counts. The Rudich–Rousseau variant of the engine works with counts modulo small integers and tests for zero via a bound. Our counts are at most $|F|^m \le 2^{L+O(m)}$, so they fit in $O(L)$ bits and can be recovered exactly from residues; in the Lean pipeline this is the lemma $Z < 2^{\text{input length}}$ (Section 8).

3 The decomposition network

This section builds two networks that count decompositions. The *edge network* (Section 3.3) is the one Theorem 2 uses: its value and its size bounds are proved in Lean. The network of Sections 3.1–3.2 has the same witnesses and the same kinds of gadgets with a hand-optimized topology and smaller constants; its value is proved on paper, not in Lean. Appendix A gives two smaller networks still.

Fix a prime $p > 3$, a curve $E : y^2 = x^3 + ax + b$ over $\mathbb{F}_p$ with $4a^3 + 27b^2 \neq 0$, an integer $m \geq 2$, and put

$$L = \lceil \log_2 p \rceil, \qquad B_x = \lceil p^{1/m} \rceil, \qquad b_x = \lfloor \log_2 B_x \rfloor + 1,$$

so b_x is the number of binary digits of B_x itself. Every $x < B_x$ has at most b_x binary digits; when B_x is a power of 2 the leading one of the b_x bits is always 0. This is the convention of the Lean statement (Section 8), which writes each $x(P_i)$ with `Nat.size B` bits, where $B = \min\{B : p \leq B^m\} = B_x$ and `Nat.size B` $= b_x$. Since $b_x \leq L/m + 2$, we have $mb_x = L + O(m)$. Elements of $\mathbb{F}_p$ are identified with integers in $[0, p)$. Since $B_x - 1 < p^{1/m} \leq \sqrt{p}$, every $x < B_x$ satisfies $x^2 < p$.

Definition 3.1
An m-tuple $(P_1, \ldots, P_m)$ of affine points of $E(\mathbb{F}_p)$ is a *generic decomposition* of an affine point R if $P_1 + \cdots + P_m = R$ and, with $S_1 = P_1$ and $S_k = S_{k-1} + P_k$, every S_{k-1} $(2 \leq k \leq m)$ is affine with $x(S_{k-1}) \neq x(P_k)$. For a bit string π let $G(R, \pi)$ be the set of generic decompositions $(P_1, \ldots, P_m)$ of R with $0 \leq x(P_i) < B_x$ for every i whose string $\sigma(P_1, \ldots, P_m)$ begins with π, where, with $x_i = x(P_i)$ written with b_x bits, $y_i = y(P_i)$ written with L bits and the sign bit $s_i = [y_i > (p-1)/2]$,

$$\sigma(P_1, \ldots, P_m) = x_1 s_1 \| \cdots \| x_m s_m \| y_1 \| \cdots \| y_m.$$

We call σ the *string* of the tuple and its first $m(b_x + 1)$ bits $x_1 s_1 \| \cdots \| x_m s_m$ its *x-string*. Since (x_i, s_i) determines P_i, distinct tuples have distinct x-strings.

Genericity excludes doubling steps, steps $S_{k-1} = -P_k$, partial sums equal to O, and $R = O$. Every step of a generic decomposition is a chord addition, which is what the network checks.

Example. Take $p = 13$, $E : y^2 = x^3 + x + 1$ and $m = 2$. Then $B_x = 4$ and $b_x = 3$, so an x-string has $2(3 + 1) = 8$ bits, and the sign bit is $s_i = [y_i > 6]$. The points with $x < 4$ are $(0, 1)$, $(0, 12)$, $(1, 4)$ and $(1, 9)$; for $x = 2, 3$ the value $x^3 + x + 1$ is a non-residue. Let $R = (8, 1)$. The chord through $(0, 1)$ and $(1, 4)$ has slope 3, and $x_3 = 9 - 0 - 1 = 8$, $y_3 = 3(0 - 8) - 1 \equiv 1$, so $(0, 1) + (1, 4) = R$ is a generic decomposition. With its reverse it is the only one, so $|G(R, \varepsilon)| = 2$. The two x-strings are $000\,0\,001\,0$ and $001\,0\,000\,0$, and $\mathrm{lexLeast}_2 = 00000010$. For $R = (8, 12)$ the least string is 00010011, from $(0, 12) + (1, 9)$, and $R = (0, 1)$ has no generic decomposition, so the answer is "none". These three values are decided in the Lean kernel for the Lean statement of Theorem 2 (module `L105_Examples`, theorems `E1`, `E2`, `E4`) and agree with a brute-force search (Section 8.5).

3.1 Construction

Witnesses. In this paragraph we index the points as $P_0, \ldots, P_{m-1}$, with $S_0 = P_0$ and $S_k = S_{k-1} + P_k$, so that $S_{m-1} = R$. A generic decomposition is described by canonical integers, each

uniquely determined by the tuple: the coordinates $x_i, y_i \in [0, p)$ of P_i; $u_i = x_i^2$; for the k-th addition ($1 \le k \le m-1$) the inverse w_k of $x(P_k) - x(S_{k-1})$, the slope λ_k and the partial sum $S_k = (s_k^x, s_k^y)$, where the last partial sum is the constant R; and for every reduction modulo p an integer quotient. The identities checked are

$$\begin{aligned} &u_i = x_i^2, && y_i^2 \equiv u_i x_i + a x_i + b, \\ &w_k(x_k - s_{k-1}^x) \equiv 1, && \lambda_k(x_k - s_{k-1}^x) \equiv y_k - s_{k-1}^y, \\ &\lambda_k^2 \equiv s_{k-1}^x + x_k + s_k^x, && \lambda_k(s_{k-1}^x - s_k^x) \equiv s_k^y + s_{k-1}^y, \end{aligned}$$

all modulo p, with $(s_0^x, s_0^y) = (x_0, y_0)$ and $(s_{m-1}^x, s_{m-1}^y) = R$. The equation for w_k forces $x(P_k) \neq x(S_{k-1})$. Each congruence $\mathcal{E} \equiv 0 \pmod p$, with $|\mathcal{E}| \le k_{\mathcal{E}}\, p$, is replaced by the integer identity $\mathcal{E} + Op = qp$ with an offset O and a quotient $q \in [0, N_q)$; for $k_{\mathcal{E}} \le O$ and $O + k_{\mathcal{E}} < N_q$ the congruence holds if and only if such a q exists, and then q is unique. There are two identities per point and four per addition, $6m - 4$ in all.

Gadgets. Each identity $\sum_t \varepsilon_t v_{l_t} v_{r_t} + (\text{linear and constant terms}) = 0$ is checked by one gadget in the style of the multiplication checks of Rudich and Rousseau [24]. Write each variable in base 2^d as a polynomial in a variable ξ with at most D digits. The identity holds over $\mathbb{Z}$ if and only if the polynomial $\mathcal{E}(\xi) = \sum_t \varepsilon_t V_{l_t}(\xi) V_{r_t}(\xi) + \cdots$ vanishes at $\xi = 2^d$. The gadget computes the coefficients of $\mathcal{E}$ by a Schönhage–Strassen transform: in the ring $\mathbb{Z}/(2^M + 1)$ the element $\omega = 2^{M/D}$ has order $T = 2D$, the product polynomials have degree below T, and M is chosen so that every coefficient of $\mathcal{E}$ is below $2^M/2$ in absolute value, so the cyclic convolution modulo $2^M + 1$ equals the integer convolution. The transform is done in two layers $T = A_1 A_2$. Four kinds of nodes carry it out: FA nodes (forward transform of a slice of one variable's digits), MID nodes (the whole quadratic form, combined in the transform domain), IB nodes (one inverse transform per identity) and CAR nodes (a carry chain that evaluates $\mathcal{E}(2^d)$ and accepts if and only if it is 0). Range conditions $x_i < B_x$ and $v < p$ are checked by digit-comparison chains (CH nodes), and the prefix π by the nodes holding the digits of the x_i, y_i (the sign bit s_i is one more digit comparison on y_i).

Every node stores only the values it receives and its own free digits, so a setting of a node is the tuple of its incoming messages, and the interaction $\Psi_{a,b}$ is equality of the message sent by a with the copy received by b. It factors through the message width (Definition 2.1). The weights W_a are $\{0, 1\}$-valued local checks. With all witnesses forced by their definitions, the network value is the number of assignments of the free variables (the x_i, y_i and the witnesses) that pass every local check, which is $|G(R, \pi)|$.

Parameters. We use the following explicit choice. Let D be the smallest power of two with $D \ge 4$ and $2D^2 \ge L$, $k = \log_2 D$, $T = 2D$, $A_1 = 2^{\lceil (k+1)/2 \rceil}$, $A_2 = T/A_1$, $h = D/A_1$ and $d = \lceil (L+2)/D \rceil$. With $\beta = 6D(2^d - 1)^2 + 2^d$ (a bound on the coefficients of $\mathcal{E}$ for at most six terms), put $M = D\lceil (\mathrm{bitlen}(\beta) + 2)/D \rceil$, message width $f_w = M + 1$ and carry width $c_b = M - d + 3$.

3.2 Size bounds

The *weight* of a node is the number of bits it holds plus the widths of its incident edges. Let U be twice the total edge width (Definition 2.2), H the total number of held bits and $W = U + H$ the total weight. The topology is explicit, and U, H and the number of nodes have exact closed

forms in $(L, m, D, A_1, A_2, d, M)$. The variables, their bit lengths, the number of gadgets using them, and whether they are range-checked are: x_0 (b_x bits, 6 gadgets, checked); x_k, $k \geq 1$ (b_x, 5, checked); y_0 (L, 3, checked); y_k (L, 2, checked); u_i ($2b_x$, 2); curve quotients ($L+2$, 1); w_k (L, 1, checked); λ_k (L, 3, checked); four quotients per addition ($L+2$, 1 each); s_k^x (L, 6, checked) and s_k^y (L, 3, checked) for $k < m-1$. Every gadget involves at most six variables, no variable lies in more than six gadgets, and the total number of (gadget, variable) incidences is $27m - 24$. With $\mathrm{nd}_v = \lceil \mathrm{bits}_v/d \rceil$ the number of digits of v,

$$U = 2\Big[A_2 S f_w + G(2Tf_w + (T-1)c_b) + \textstyle\sum_{v \text{ checked}} (\mathrm{nd}_v d + \mathrm{nd}_v - 1)\Big], \tag{1}$$

$$H = d \textstyle\sum_v \mathrm{nd}_v + A_2 S f_w + G(Tf_w + T(f_w + c_b)) + \sum_{v \text{ checked}} \mathrm{nd}_v (2d+1), \tag{2}$$

where $G = 6m - 4$ is the number of gadgets and $S = \sum_{\text{gadgets}} \sum_{v \in \text{gadget}} \min(A_1, \mathrm{nd}_v)$.

Lemma 3.2
For all $L \geq 16$:

(S1) $\sqrt{L/2} \leq D < \sqrt{2L}$, so $k < \frac{1}{2}\log_2(2L)$;

(S2) $d \leq (L+2)/D + 1$;

(S3) $\mathrm{bitlen}(\beta) \leq 2d + k + 3$, hence $f_w \leq F := 2d + D + k + 5$; also $c_b < f_w$ and $f_w \geq 2d + 3$;

(S4) $Tf_w \leq 15.3\, L$;

(S5) $L + 2 + d + D \leq 2L$, $L + 2 + d \leq 1.6L$ and $D \leq 0.36L$;

(S6) $F \leq 6.97\sqrt{L}$;

(S7) $\sqrt{2D} \leq A_1 \leq 2\sqrt{D}$, $A_2 \leq A_1$, $h \leq A_1$, and $\mathrm{nd}_v \leq D$ for every variable.

Proof. (S1) The lower bound is the definition. If $D > 4$ then $D/2$ failed, so $D^2 < 2L$; if $D = 4$ then $16 < 2L$. (S2) is immediate. (S3) $\beta = 6D2^{2d} - 12D2^d + 6D + 2^d \leq 6D2^{2d} < 2^{2d+k+3}$, and $M < \mathrm{bitlen}(\beta) + 2 + D$. (S4) $Tf_w \leq 2DF = 2D(2d+D+k+5) \leq 4(L+2) + 4D + 2D^2 + 2D(k+5) < 8L + E(L)$ with $E(L) = 8 + \sqrt{2L}(\log_2(2L) + 14)$, using (S1) and (S2). The function $E(L)/L$ is decreasing for $L \geq 16$ and $E(16)/16 < 7.3$. (S5) and (S6) follow from (S1) and (S2) in the same way: the differences are increasing in L for $L \geq 16$ and the inequalities hold at $L = 16$. (S7) $A_1^2 \geq 2^{k+1} = T$ and $A_1^2 \leq 2^{k+2} = 4D$; $\mathrm{nd}_v \leq \lceil (L+2)/d \rceil \leq D$ because $dD \geq L + 2$. □

The monotonicity claims in (S4)–(S6) are elementary calculus. We also evaluated every inequality of Lemma 3.2 and of the proofs below for every integer $L \in [16, 2^{20}]$.

Proposition 3.3 (shared information)
For all $L \geq 16$ and $m \geq 2$, $U \leq 1410\, mL$; asymptotically $U \leq (744 + o(1))\, mL$.

Proof. By (S7), $S \leq A_1(27m - 24)$, so $A_2 S f_w \leq (27m-24) T f_w$. Since $c_b < f_w$, $G(2Tf_w + (T-1)c_b) \leq 3(6m-4)Tf_w$. Together these are at most $(45m-36)Tf_w \leq 45m \cdot 15.3L$ by (S4). There are $6m-6$ checked variables, and each contributes $\mathrm{nd}_v d + \mathrm{nd}_v - 1 < \mathrm{bits}_v + d + D \leq L + 2 + d + D \leq 2L$ by (S5), so the chains contribute at most $12mL$. Hence $U \leq 2(688.5 + 12)mL < 1410\, mL$. Asymptotically $Tf_w \leq 8L + o(L)$ by (S4), which gives $2(45 \cdot 8 + 12) = 744$. □

Proposition 3.4 (total weight)
For all $L \geq 16$ and $m \geq 2$, $H \leq 730\, mL$ and $W = U + H \leq 2140\, mL$.

Proof. There are $12m-10$ variables and $\mathrm{nd}_v d < \mathrm{bits}_v + d \le L+2+d \le 1.6L$, so $d\sum_v \mathrm{nd}_v \le 19.2\,mL$. As in Proposition 3.3, $A_2 S f_w + G(T f_w + T(f_w + c_b)) \le 45m \cdot 15.3L = 688.5\,mL$. Finally $\sum_{\text{checked}} \mathrm{nd}_v(2d+1) < (6m-6)(2(L+2+d)+D) \le 21.4\,mL$ by (S5). □

Proposition 3.5 (largest node)
Every node has weight at most $13A_1 f_w \le 216\,L^{3/4}$ and holds at most $6A_1 f_w \le 100\,L^{3/4}$ bits.

Proof. An FA node holds at most hd bits and has at most $6A_2$ edges of width f_w and h chain edges of width d, so its weight is at most $2hd + 6A_2 f_w \le 7A_1 f_w$ (using $2hd \le 2A_1 d \le A_1 f_w$). A MID node of a gadget with $n_g = \sum_v \min(A_1, \mathrm{nd}_v) \le 6A_1$ holds $n_g f_w$ bits and has weight $(2n_g + A_1)f_w \le 13A_1 f_w$. IB nodes have weight $3A_2 f_w$, CAR nodes at most $5f_w$, CH nodes $3d+3$. By (S7), (S6) and (S1), $13A_1 f_w \le 26\sqrt{D}\,F \le 26(2L)^{1/4} \cdot 6.97\sqrt{L} < 216L^{3/4}$, and $6A_1 f_w < 100L^{3/4}$. □

The bound $L^{3/4}$ is sharp for this construction: a MID node of a gadget with two full-length variables holds at least $2A_1 f_w \ge 2\sqrt{2D} \cdot 2d = \Omega(L^{3/4})$ bits. This is why the cluster count is limited to $g = O(L^{1/4})$ below.

Proposition 3.6 (explicit clustering)
Order the nodes in construction order, let $\mathrm{acc}(a)$ be the total weight of the nodes before a, and put node a in cluster $\lfloor g\,\mathrm{acc}(a)/W \rfloor \in \{0, \dots, g-1\}$. Then the merged network on $[g]$ has $P < W/g + 216L^{3/4}$, and for $1 \le g \le c_3 L^{1/4}$

$$gP \le (2140 + 108\,c_3)\,mL.$$

In particular $gP \le 2250\,mL$ for $g \le L^{1/4}$ and $gP \le 2464\,mL$ for $g \le 3L^{1/4}$.

Proof. The nodes of cluster c have $\mathrm{acc} \in [cW/g, (c+1)W/g)$. If they are $a_1, \dots, a_s$ in order, their total weight is $(\mathrm{acc}(a_s) - \mathrm{acc}(a_1)) + w(a_s) < W/g + \max_a w(a)$. Both the bits held by a cluster and the total width of its cross-cluster edges are at most its weight. Hence $P < W/g + 216L^{3/4}$ and $gP < W + 216\,gL^{3/4} \le 2140\,mL + 216c_3 L \le (2140 + 108c_3)mL$ for $m \ge 2$. □

The proved constants are loose. On the instances we built, $U/(mL)$ is between 180 and 330 for $L \ge 4096$ and the largest node weight is between $25L^{3/4}$ and $40L^{3/4}$. Proposition 3.3 has no factor $\log L$, and this is the property the proof of Theorem 2 needs; the edge network below has it too, with larger constants. Appendix A describes a variant network with $U \le 125\,mL$ and g up to $\sqrt{L}/\log_2 L$, conditional on a hypothesis about primes in arithmetic progressions.

3.3 The edge network and Proposition N

The *edge network* is the network of our Lean development (Section 8). It checks the same $6m-4$ identities with the same witnesses, one Schönhage–Strassen gadget per identity, and the same kinds of nodes: source nodes holding the digits of the variables, and FA, MID, IB and CAR nodes. It differs from the network of Section 3.1 in that every message is sent on its own edge to one child ("child-specific messages"), and in its parameters, which are chosen for a simple proof rather than for small constants. Every node has its free digits and the messages it receives as its setting, every weight W_a is a $\{0,1\}$-valued local check, and every interaction $\Psi_{a,b}$ tests equality of the message computed from the parent's setting with the copy held by the child; so all values lie in $\{0,1\}$ and the value bound is $B = 1$.

Proposition 3.7 (edge network; Lean)
For every $m \geq 2$, every input (p, a, b, R) and every bit string π of length at most $m(b_x + 1)$ (it need not begin any x-string; then both sides of (i) are 0):

(i) the value of the edge network for π is $|G(R, \pi)|$;

(ii) for every $g \geq 1$ with $g^4 \leq L$, its nodes can be merged into g clusters, without changing the value, so that $U \leq 549504\, mL$ and $gP \leq 555732\, mL$.

Proof. Both parts are Lean theorems (Section 8). (i) is `edge_value` and `net_eq_ncard_edge`, specialized to the reduced instances by `edgeCount_real_eq_Zc`; there the counted objects are the x-strings with sign bits, which are in bijection with the tuples of $G(R, \pi)$. (ii) is `ecdlp_engine_conditions`: for any slot width L' such that every constant of the instance is below $2^{L'}$, and any g with $g^4 \leq L'$, it gives a clustering with $U \leq 183168\, mL'$ and $gP \leq 185244\, mL'$. Our instantiation takes $L' = 2L + 3 \leq 3L$ (for $L \geq 3$), and $g^4 \leq L$ implies $g^4 \leq L'$. The total weight of the unclustered network is bounded in Lean as well (`wt_htot`, against the bound U_c of module `L65`). The statement of (ii) bounds only the shared bits of a cluster. The setting-length bound P that Definition 2.2 also requires comes from the held-bit bounds of the Lean development: the weight `wtB` (shared plus free bits) and the `EdgeCore` fields `hheld`, `hmax` and `htot` (`wt_htot`) bound the number of settings of a node, hence its setting length, by its weight, and so for a cluster by the cluster's weight. This is the bound on P_A used with Proposition N. □

The constants of Proposition 3.7 are about 400 times (for U) and 230 times (for gP) those of Proposition 3.3 and Proposition 3.6, and the cluster condition $g^4 \leq L$ replaces $g \leq 3L^{1/4}$. Theorem 2 needs only $U = O(mL)$ and $gP = O(mL)$, so either network serves for it. We base Theorem 2 on the edge network because its value is proved.

Lean proves (i) for bit strings of length at most $m(b_x + 1)$, that is, for prefixes of the x-string with sign bits. The search of Theorem 2(ii) also uses prefixes that extend into the y-digits. Write $n_\sigma = m(b_x + 1 + L)$ for the length of a string σ.

To state the extension we need the layout of the edge network (Lean module `L56_Edge`). Every variable and every constant of the identity system has a *slot* of $L_s = dD \geq L$ bits, D digits of d bits (in Lean $L_s \geq L'$, the slot-width parameter of Proposition 3.7). A slot value V is written in base 2^d: digit r holds bits $rd, \ldots, rd + d - 1$ of V. The digits of a slot are split into A slices (in Lean, digit r lies in slice r mod A), and slice s of slot v is the free variable of exactly one node, the *source node* $\mathrm{src}(v, s)$. The weight of a source node is a $\{0, 1\}$-valued check of its own slice, the *source check*: in Lean, $W_{\mathrm{src}(v,s)}(z) = [\mathrm{ok}(v, s, z)]$, and for the prefix π_x the check ok is the predicate `okC` for a list of (bit position, bit value) pairs per slot. That list pins the constants, pins the sign slot when no sign is required, and lists the bits of x_i given by π_x; for the slot of y_i it is empty. All other weights are 1, except those of the CAR nodes, which check carries.

Lemma 3.8 (prefixes of the full string)
Let π be a bit string with $m(b_x + 1) < |\pi| \leq n_\sigma$, and let π_x be its first $m(b_x + 1)$ bits. The bits of π that fall in the y_i-block of σ prescribe the top t_i bits of y_i, for some $0 \leq t_i \leq L$: bit q of the block ($0 \leq q < t_i$) is bit $L - 1 - q$ of the integer y_i. Let N_π be the edge network for π_x in which the list of the slot of each y_i is enlarged from the empty list to the pairs $(L - 1 - q, \pi\text{-bit})$, $q < t_i$. Then:

(i) the value of N_π is $|G(R, \pi)|$ (both sides are 0 if π begins no string);

(ii) N_π has the same nodes, settings, edges and edge widths as the edge network for π_x; hence the bounds of Proposition 3.7(ii) hold for N_π, with the same clustering.

Proof. Alignment. A prescribed bit $j = L - 1 - q$ of y_i lies in digit $\lfloor j/d \rfloor$ of its slot, at position $j \bmod d$, and that digit lies in exactly one slice, so exactly one source node tests it. A digit with only some of its bits prescribed is compared under a mask: only the prescribed positions are tested. Bits at positions $L, \dots, L_s - 1$ are never prescribed. In every assignment that passes all checks they are 0, because the range identity of y_i (Lean identity `rng`, bound p) forces $V < p \le 2^L$. All prescribed positions are below $L \le L_s$, as `okC_iff` requires.

(i) Lean's `edge_value` holds for an arbitrary family of $\{0,1\}$-valued source checks: the value of the edge network equals the number of digit assignments that pass every source check and satisfy every identity. Lean's `okC_iff`, which holds for arbitrary lists, turns the source checks of a slot into the condition that the slot value has the listed bits. So the value of N_π is the number of slot valuations that satisfy every identity, the conditions of π_x, and the prescribed bits of the y_i. The valuations satisfying the identities and the conditions of π_x are in bijection with $G(R, \pi_x)$ (Proposition 3.7(i)). Under this bijection the slot of y_i holds the integer $y(P_i) \in [0, p)$: the Lean correspondence (module `L44`) maps a valuation to the integer assignment whose point coordinates are the coordinate slots, and the witnesses are unique (`chain_unique`, `zero_iff_window`). The prescribed bits therefore keep exactly the tuples of $G(R, \pi_x)$ whose y-bits agree with π, which is $G(R, \pi)$.

(ii) The source check enters only the weight of a source node, a function of that node's own slice. The node set, the settings, the messages, the interactions and the widths are defined without it. They depend on the prefix only through the sign requirements (Lean `sgOf`), which π_x fixes. The bounds of Proposition 3.7(ii) and the clustering that attains them (Lean `clus`, computed from node weights, `exists_cluster`) are functions of these data alone, so they hold for N_π unchanged. □

Status of Lemma 3.8. The x-and-sign case, $|\pi| \le m(b_x + 1)$, is Lean-verified: it is Proposition 3.7, and the search over it is the end-to-end Lean theorem of Section 8. The extension to the y-digits is a paper proof. It uses Lean lemmas that hold for arbitrary source checks (`edge_value`, `okC_iff`) and two observations read off the Lean construction: that the y_i slot holds $y(P_i)$ in the bijection, and that the size bounds do not involve the source checks. Lean's `ecdlp_engine_conditions` is stated for the x-string checks only, and N_π is not itself formalized. The cost of the added checks is part of Lemma 3.9.

Lemma 3.9 (cost of the local functions)
Count costs in word operations on words of $O(\log(mL))$ bits, and let every local function read the input (p, a, b, R, π), which has $O(mL)$ bits since $|\pi| \le n_\sigma$. Let N be the edge network for (p, a, b, R) and a bit string π with $|\pi| \le n_\sigma$ (the network N_π of Lemma 3.8 when $|\pi| > m(b_x + 1)$), and for $g \ge 1$ with $g^4 \le L$ let $N^{(g)}$ be the network on $[g]$ obtained from N by the clustering of Proposition 3.7(ii). Then:

(i) membership in each S_A, every value of W_A, $\Psi_{A,B}$ and of their factorizations, and every $c_{A,B}$ of $N^{(g)}$ are computable within $\tau \le (mL)^{O(1)}$ operations;

(ii) a deterministic algorithm outputs $N^{(g)}$ (its nodes with their clusters, setting lengths, edges and widths) in time $(mL)^{O(1)}$.

For the edge network, $\tau \le C\, m\, L^{7/4}$ bit operations for an absolute constant C. The exponents in $O(1)$ are absolute; in particular they do not depend on m. The same holds for the network of Section 3.1 with the clustering of Proposition 3.6.

Proof. Nodes of the edge network. The Lean parameters (`par_efficient`) give D digits per slot of $d \le D$ bits, with $D^2 \le 16 L_s$, and the ring $\mathbb{Z}/(2^M+1)$ with $M = wD$ for a bounded integer w ($w = 5$ in `par_efficient`). So ring elements have $M+1 = O(\sqrt{L})$ bits. The nodes and their local functions are (module `L56_Edge`):

- $\mathrm{src}(v, s)$: holds slice s of slot v; weight the source check, a comparison of at most L_s bits with bits of the input; sends its slice to the FA nodes that read it.
- $\mathrm{FA}(c, j, s)$, for identity c, variable j of c and slice s: receives the slice; sends to each MID child of c one ring element, a partial forward transform, which is a sum over the slice of digits multiplied by powers of $\omega = 2^{M/D}$ (cyclic shifts with negation); weight 1.
- $\mathrm{MID}(c, k)$: receives one ring element per (variable, slice) of c; sends to each IB child one ring element, obtained by summing over slices and evaluating the quadratic form of c (at most six products of ring elements, with constants read from the input); weight 1.
- $\mathrm{IB}(c, j_1)$: receives MID elements; sends to its CAR children one ring element of the inverse transform, again a sum of shifted elements; weight 1.
- $\mathrm{CAR}(c, j)$: receives one IB element and the previous carry; weight 1 if the sum is divisible by 2^d (and, for the last, the final carry is 0), else 0; sends a carry of $M+3$ bits.

Every interaction is $\Psi_{a,b}(z_a, z_b) = [\mathrm{msg}_{a\to b}(z_a) = \text{copy in } z_b]$ for the parent a and child b, with factorization $f(z_a, \theta) = [\mathrm{msg}_{a\to b}(z_a) = \theta]$, $f'(z_b, \theta) = [\text{copy in } z_b = \theta]$, through $c_{a,b}$ bits, the message width. All values lie in $\{0, 1\}$.

Settings and node count. A node's setting is its free slice (if any) and the copies of its incoming messages. It has at most $P \le c_2 mL$ bits, since a node holds no more than its cluster (Proposition 3.7(ii) with $g = 1$). For the parameters of the Lean pipeline the sharper, m-free bound $O(L^{3/4})$ is a Lean theorem, stated for arbitrary source checks and so covering N_π (`edge_bridge_setting`). The number of nodes is at most $828\, mX^2$ with $X^4 = \Theta(L)$ (`edge_bridge_gates`, module `L65_Bridge2`), so $O(m\sqrt{L})$; the number of edges is $O(m\sqrt{L})$ as well (counted by hand from the definitions of the nodes above: each identity has $O(A^2) = O(D) = O(\sqrt{L})$ edges, and there are $O(m)$ identities).

Unclustered cost. Each message and each weight above is computed from the node's setting and the input by at most $O(mL)$ additions, shifts, products of two ring elements and bit comparisons, on integers of $O(\sqrt{L})$ bits (ring elements have $M+1 = O(\sqrt{L})$ bits). That is $\mathrm{poly}(mL)$ bit operations, hence $(mL)^{O(1)}$ word operations. The same holds for $\mathrm{msg}_{a\to b}$, f, f' and $\Psi_{a,b}$. Membership in S_a is a range test on each component of the setting (a digit below 2^d, a ring element below 2^M+1, a carry below 2^{M+3}), linear in the setting length.

Clusters. The clustering is Lean's `clus`: nodes in construction order, node a in cluster $\lfloor g\,\mathrm{acc}(a)/W \rfloor$, as in Proposition 3.6. It is computed from the node weights in time linear in the number of nodes. The merged network is Lean's `Net.coarsen`: for clusters $A \ne B$, $S_A = \prod_{a\in A} S_a$, $W_A(z_A) = \prod_{a\in A} W_a(z_a) \prod_{\{a,b\}\subseteq A} \Psi_{a,b}(z_a, z_b)$, $\Psi_{A,B} = \prod_{a\in A, b\in B} \Psi_{a,b}$ and $c_{A,B} = \sum_{a\in A, b\in B} c_{a,b}$. The factorization of $\Psi_{A,B}$ runs over $\theta = (\theta_{a,b})_{a\in A, b\in B}$, the concatenation of the members' strings, with $f_A(z_A, \theta) = \prod_{a,b} f_{a,b}(z_a, \theta_{a,b})$ and f'_B likewise; expanding the product of sums proves the

identity. All values stay in $\{0,1\}$, so $B = 1$. The merged $\Psi_{A,B}$, W_A and $f_{A,B}$ are products of member factors (Lean's `coarsen_F_eval`), so a query to a cluster is the product of its member factors: one member weight per member, one member Ψ or f per edge inside or crossing the cluster. For the edge network there are $O(m\sqrt{L})$ nodes and $O(m\sqrt{L})$ edges, so a query multiplies $O(m\sqrt{L})$ member factors, and $\tau \le (mL)^{O(1)}$. (Non-adjacent pairs contribute the factor 1, and the algorithm enumerates edges, not pairs.) For the hand-optimized network we argue the same count with the same node kinds, but this is not independently reviewed.

Explicit bound. The per-node counts (a message of the largest node costs $O(L^{5/4})$ bit operations, dominated by the MID message; a source check costs $O(L^{3/4})$) are read from the Lean definitions of the nodes by hand, were independently recomputed, and are not formalized. The instance data (p, a, b, R, π) is part of the query input. The formal statement of the engine does not need a preparation step: it treats the network as an oracle with cost τ per query, and any preparation the algorithm makes once (tabulating the clustering, its members, offsets and edges, and the constants of the instance, in time $O(mL \log(mL))$) is absorbed in the engine's polynomial factor. Multiplying the $O(m\sqrt{L})$ member factors, each costing $O(L^{5/4})$ at most, gives, for the edge network,

$$\tau = O(m\, L^{7/4}) \text{ bit operations,}$$

uniformly in m, with an absolute constant. For the hand-optimized network we claim the same bound; this is argued and not independently reviewed. Either bound gives $\tau \le (mL)^{O(1)}$, which is all that is used in the other sections.

Construction. The network is produced by explicit loops over the $O(m)$ identities, their at most six variables, the slices and the transform indices, with parameters computed from L (`par_efficient`), followed by the node weights and acc. It has $O(mL)$ nodes, so this takes time $(mL)^{O(1)}$.

Uniformity. m enters only through the number of identities, the number of nodes and $|\pi| \le n_\sigma$, each $O(mL)$; all exponents above are absolute.

The network of Section 3.1. It has the same node kinds plus CH nodes (digit-comparison chains, comparisons of d-bit digits). Settings have at most $100L^{3/4}$ bits (Proposition 3.5), ring elements have $f_w = O(\sqrt{L})$ bits (Lemma 3.2), and the clustering of Proposition 3.6 is the same cumulative-weight rule. The argument is unchanged. □

Lemma 3.9 is a paper proof. It is needed for the paper statements that rest on (E): the deterministic y-digit extensions of Theorem 2 and Theorem 7. The Lean theorems of Section 8 do not use it: there the engine and its cost on the edge network for x-string prefixes are formalized.

Proposition N
There are absolute constants $c_1 = 549504$ and $c_2 = 555732$ and a deterministic algorithm that, given (p, a, b, R, π) with π a bit string of length at most n_σ, and $g \ge 1$ with $g^4 \le L$, outputs in time $(mL)^{O(1)}$ a network on $[g]$ with $U \le c_1 mL$, $gP \le c_2 mL$, value bound $B = 1$ and $\tau \le (mL)^{O(1)}$, whose value is $|G(R, \pi)|$. The constants and the exponents in $O(1)$ do not depend on m.

Proof. For $|\pi| \le m(b_x + 1)$, Proposition 3.7; for longer π, Lemma 3.8. In both cases Lemma 3.9 gives τ and the construction time for the clustered network. □

We write (N) for Proposition N. For the hand-optimized network of Section 3.1 the size bounds are proved (Propositions 3.3–3.6, with $c_1 = 1410$, $c_2 = 2464$ and $g \le 3L^{1/4}$), and Lemma 3.9 applies to it. That its value equals $|G(R, \pi)|$ is proved on paper, after three repairs

to the construction as first described: (a) each y_i gets a comparison chain that tests its sign bit, which the earlier checks never exercised; (b) each x_i uses b_x bits, the bit length of B_x as in Definition 3.1, not that of $B_x - 1$; and (c) the witness u_i has $\min(2b_x, L)$ bits, since $2b_x$ bits overflow the degree bound for $m = 2$. The repairs change the bounds to $U \le 1405\,mL$, $H \le 733\,mL$ and $W \le 2138\,mL$, within the constants above, and the asymptotic constant of U from 744 to 748. The proof is a direct analysis of the gadgets as in Proposition 3.7, extended to sign bits and y-digit prefixes. It was checked as follows:

- *Size bounds.* A script recomputes the exact closed forms (1)–(2) against the constructed topology for 290 values of $L \in [16, 65536]$ and $m \in \{2, 3, 5, 8, 16\}$, and checks the cluster assignment on real topologies.
- *Value equals count.* Checked against brute-force enumeration for x-prefixes (without sign bits): on 168 instances with the parameters of our first construction ($m = 2, 3, 4$, p from 37 to 16381, targets with zero, one and many decompositions, empty, partial, full and wrong prefixes) and on 172 instances with the parameters above, every network value equals the brute-force count; about 40,000 perturbed witnesses were all rejected; every gadget was checked exhaustively for $p = 7, 13$. An independent re-implementation matched 30 further instances. After the repairs, an implementation written from the text alone matched brute force on 240 instances and 1452 counts, 461 of them with y-digit prefixes, and detected 8 of 8 injected faults; an independent counter matched 210 further instances.

So Theorem 2 holds with this network in place of the edge network, as a paper proof. Its constants enter the crossover estimates of Section 10. In addition, the cumulative-weight clustering satisfies $P < W/g + 216L^{3/4}$ for every $g \ge 1$, not only for $g \le 3L^{1/4}$, so the level condition of Section 10 improves from $K_\ell \le L^{1/32}$ to $K_\ell \le L^{1/28}$; this was checked on 17 constructed networks for all g up to the number of gates (largest ratio 0.905).

4 Fast decomposition

Theorem 2 (fast decomposition)
For every fixed $m \ge 2$ and every $k \ge 1$ there is a constant $C_k = C_k(m)$ and a deterministic algorithm that, given a prime p, $a, b \in \mathbb{F}_p$ with $4a^3 + 27b^2 \ne 0$, an affine point $R \in E(\mathbb{F}_p)$ and a prefix π of length at most $m(b_x + 1 + L)$:

(i) (counting) outputs $|G(R, \pi)|$ within $C_k 2^{L/k}$ operations; for prefixes over the x-string with sign bits this needs no hypothesis, and over the y-digits it assumes (E);

(ii) (search; no hypotheses) if $G(R, \varepsilon) \ne \emptyset$, where ε is the empty prefix, outputs the lexicographically least x-string with sign bits (Remark 4.1) of a tuple of $G(R, \varepsilon)$, and otherwise reports that there is none, within $C_k 2^{L/k}$ operations. This is the Lean theorem `subexponentialRate_all` (Section 8). Assuming (E), continuing the descent over the y-digits (Lemma 3.8) outputs the tuple itself, y_i included, within $(m(b_x + 1 + L) + 1)\, C_k 2^{L/k}$ operations.

Both parts are deterministic; no square roots modulo p are computed. In particular, the bounded generic decomposition problem is solved in time $p^{o(1)}$ for each fixed m.

Here $mb_x = L+O(m)$, so the search for the x-string with sign bits uses $m(b_x+1)+1 = L+O(m)$ counts, and the descent over the full string σ of Definition 3.1 uses $(m+1)L + O(m)$. The tuple the full descent finds is also the tuple whose x-string is lexicographically least: all strings have the same length, the x-string is a prefix of σ, and it determines the tuple. So the descent over the y-digits changes nothing about which tuple is found; it only reads off the y_i deterministically. Without (E), the y_i follow from the labels by square roots (Remark 4.1).

Proof. This is the paper proof. It assumes (E) only for prefixes that reach into the y-digits (the y-digit extension of (i) and (ii)). For the counting in (i) and the search in (ii) over the x-string with sign bits it is superseded by the Lean proofs (see the paragraph after the proof).

Step 1 (choice of K). Let ℓ be the largest index with $K_\ell \le (L/81)^{1/32}$, and put $K = K_\ell$ and $g = 3K^8$, so that $g^4 = 81K^{32} \le L$. This requires $K_0 = 2^{x_0} \le (L/81)^{1/32}$; the finitely many inputs with smaller L are handled by a table, whose cost goes into C_k. Proposition N supplies a network on $[g]$ with $U \le c_1 mL$, $gP \le c_2 mL$ and $B = 1$ (here $c_1 = 549504$ and $c_2 = 555732$). By the Engine Theorem (a), $K_{\ell+1} \le K^{10}$ and $K_{\ell+1} > (L/81)^{1/32}$, so $K > (L/81)^{1/320}$ and $\log_2 K \ge (\log_2 L - 7)/320$.

Step 2 (cost of one engine call). The Engine Lemma, case (i), with $\zeta = 1/(2K)$ reduces the network to (K, L_{cl})-cliques with $K^2 L_{\mathrm{cl}} \le U + gP/K + K^2$ and $B' \le 2gP/K + g^2 B + 1$, which the Engine Theorem (b) evaluates. Taking logarithms,

$$\log_2 T_{\mathrm{eng}} \le 1 + \max\{\, g \log_2 K,\ \Lambda \,\},$$

where

$$\begin{aligned}\Lambda = e_\ell\Big(U + \frac{gP}{K} + K^2\Big) + C_0 K^2 \log_2\big(K(L_{\mathrm{cl}} + B')\big) \\ + \frac{2gP}{K} + C_1 \log_2\big(\tau\, g(P+B)\big).\end{aligned}$$

Substituting the bounds of Proposition N,

$$\log_2 T_{\mathrm{eng}} \le e_\ell\, c_1 mL\,(1+o(1)) + \frac{2c_2 mL}{K} + C_0 K^2 \log_2(K\, c\, mL) + g \log_2 K + O(\log(mL)). \quad (3)$$

Step 3 (asymptotics). We have $g \log_2 K \le L^{1/4} \log_2 L$, which is $o(L)$; the term $C_0 K^2 \log_2(\cdot)$ is $O(L^{1/16} \log L) = o(L)$; $2c_2 mL/K \le 2c_2 m\, 81^{1/320} L^{1-1/320} = o(L)$ for fixed m; and by the Engine Theorem (a)

$$e_\ell \le 0.85\rho^\ell \le 0.85\Big(\frac{3x_0+6}{\log_2 K_\ell}\Big)^{\alpha'} \le 0.85\Big(\frac{320(3x_0+6)}{\log_2 L - 7}\Big)^{\alpha'} \to 0.$$

Hence $\log_2 T_{\mathrm{eng}} = o(L)$ for fixed m, that is $T_{\mathrm{eng}} \le C_k 2^{L/k}$ for every k. This proves (i).

Step 4 (search from counts). Write $Z(\pi) = |G(R,\pi)|$ and $n_\sigma = m(b_x + 1 + L)$ for the length of a string. For every prefix π shorter than n_σ, $Z(\pi) = Z(\pi 0) + Z(\pi 1)$, since the next bit partitions the counted set. Starting from $\pi = \varepsilon$ with $Z(\varepsilon) > 0$, set $\pi \leftarrow \pi 0$ if $Z(\pi 0) > 0$ and $\pi \leftarrow \pi 1$ otherwise, n_σ times. Every prefix visited has $Z > 0$, so the final string is the string of a tuple in $G(R,\varepsilon)$, and it is the lexicographically least such string. This takes $n_\sigma + 1$ counts, each by (i) with the network of Lemma 3.8 once π passes the x-string (by Proposition N, which covers every bit string π of length at most n_σ). The tuple is read off the string: x_i and y_i are written in it. □

The argument is a reduction: the combinatorial Steps 3 and 4 are elementary (Step 4 is proved in Lean, Section 8), while the inputs are (E) and Proposition N. For the search over the x-string with sign bits (Remark 4.1) the dependence on (E) is discharged in Lean: that search is the theorem `subexponentialRate_all` of Section 8, which has no hypotheses because the engine itself is formalized there. The same holds for the counting statement (i) over the x-string with sign bits, which is the theorem `countingRate_all`. (E) remains the hypothesis only of the extension of (i) and (ii) to the y-digits (Lemma 3.8). Theorem 2 assumes nothing about how points are distributed on E. The constants C_0 and x_0 of the engine are existential, so C_k is not explicit.

Remark 4.1 (square roots and sign bits)
The search runs over the y-digits because no deterministic polynomial-time algorithm for square roots modulo p is known without a quadratic non-residue. Stopping the descent after the x-string costs only $m(b_x + 1) + 1$ counts and gives the labels (x_i, s_i), which determine the points; the y_i then follow by square roots, deterministically given a non-residue and otherwise by a Las Vegas algorithm (Tonelli–Shanks or Cipolla) in expected polynomial time. This truncated search is proved in Lean for every m, with no hypotheses (Section 8): a Turing machine outputs the lexicographically least x-string with sign bits, or reports that there is none, within $C_k 2^{\ell/k}$ steps on inputs of length ℓ, for every $k \geq 1$. For π of length $O(L)$ the input length is $\ell = O(L)$, so this is time $p^{o(1)}$. In the ECDLP algorithms below a factor-base point only needs a label, and (x_i, s_i) is one; randomized square roots would be harmless there in any case.

Remark 4.2 (the full decomposition problem)
Theorem 2 extends to all ordered m-tuples of affine points with $x(P_i) < B_x$ summing to any $R \in E(\mathbb{F}_p)$, $R = O$ included, with doublings, cancellations and partial sums O allowed. We state this extension with weaker evidence than Theorem 2, and the ECDLP results do not use it. Classify step k ($2 \leq k \leq m$) of the walk $S_k = S_{k-1} + P_k$ as A (generic addition), D (doubling: $P_k = S_{k-1}$ with $y \neq 0$), C (cancellation: $P_k = -S_{k-1}$, including 2-torsion) or S (start: $S_{k-1} = O$). For affine S_{k-1} and equal x-coordinates, $y(P_k) = \pm y(S_{k-1})$, so the types are exhaustive and exclusive, and the word in $\{A, D, C, S\}^{m-1}$ is a function of the tuple. Type S occurs exactly after C, and the last type is C exactly when $R = O$. So for fixed m the words form a finite set: $2, 5, 12, 29, 70, 169, 408$ words for affine R and $m = 2, \ldots, 8$, about 2.4^m. Each word has a network of the same kind. A D step uses six gadgets (two equalities, the inverse of $2y$, the tangent slope, and the two sum identities), a C step two, and an S step none. The full count is the sum of the word counts, and prefix descent works because the next bit partitions the whole set. Status: the word classification is proved by the case analysis above. That the word networks count correctly is checked by brute force: exhaustively for $m = 2$ and $p \in \{7, 11, 13, 17\}$; on 312 instances with $p \leq 199$ for $m = 2, 3$ and $p \leq 61$ for $m = 4$, where 46,944 tuples (14,490 of them non-generic) were each accepted by exactly their own word and the word sums equal the direct counts for every R, $R = O$ included; and at gate level on 66 decompositions, with 1,794 perturbed witnesses all rejected. Each word has at most $8m - 6$ gadgets, at most $27m - 24$ variable incidences and at most $6m - 6$ range-checked variables. Repeating the proof of Proposition 3.3 with these counts gives $U \leq$ about $1600\, mL$; this step is argued, not written out, and measured values are at most 1.18 times those of the generic network. Lemma 3.9 applies to the word networks without change, and nothing here is in Lean.

5 Index calculus and coverage

Throughout this section $p > 3$, $E : y^2 = x^3 + Ax + B$ is nonsingular over $\mathbb{F}_p$, $n = |E(\mathbb{F}_p)| \le p + 1 + 2\sqrt{p} \le 2p$, and $m \ge 2$ is fixed. The decomposition oracle is Theorem 2(ii):

Oracle (O)
A deterministic algorithm $\mathcal{D}$ that, given (p, a, b) and an affine $R \in E_{a,b}(\mathbb{F}_p)$, returns the labels (x_i, s_i) (Remark 4.1) of a generic decomposition $R = P_1 + \cdots + P_m$ with $x(P_i) < \lceil p^{1/m} \rceil$ whenever one exists, and reports that there is none otherwise, in time $p^{o(1)}$ for fixed m. Theorem 2(ii) provides it with no hypotheses: it is the Lean theorem `subexponentialRate_all` with the empty prefix (Section 8). The points P_i follow from their labels by a Las Vegas square root modulo p (Tonelli–Shanks or Cipolla) in expected polynomial time. The algorithms below need only the labels, and the coins of the square root never enter a row of the relation matrix. So Theorems 1 and 4–6 do not depend on (E).

The oracle accepts any model (a, b). The algorithms below use this freedom.

5.1 From decompositions to discrete logarithms

Semaev's index calculus [17] with $\mathcal{D}$ as relation finder works as follows. Sample $R = \alpha P + \beta Q$, ask $\mathcal{D}$ for a decomposition, record the relation, and solve a sparse linear system once there are more relations than factor-base points. Two things have to be proved: that a constant fraction of targets R decompose, and that the linear algebra yields d. The standard analysis assumes both. Under the assumption that a random point decomposes with probability at least $c/m!$ for an absolute $c > 0$, and that the relation matrix behaves like a random sparse matrix, the method runs in time $p^{2/m+o(1)}$ given (O). This is only heuristic, and Theorem 4 below replaces both assumptions by proofs.

The second assumption is easy to remove; this is standard.

Lemma 5.1 (rank-free Las Vegas step)
Let $\langle P \rangle$ have prime order q and $Q = dP$. Let (α_i, β_i), $i = 1, \dots, N_r$, be independent and uniform in $(\mathbb{Z}/q)^2$, $R_i = \alpha_i P + \beta_i Q$, and let $\mathcal{M}$ be any matrix-valued function of $(R_1, \dots, R_{N_r})$ (and of randomness independent of the β_i). If $\lambda \ne 0$ is a function of $\mathcal{M}$, then $\Pr[\sum_i \lambda_i \beta_i = 0] = 1/q$.

Proof. Given $R_i = (\alpha_i + \beta_i d)P$, the pair (α_i, β_i) is uniform on the line $\{\alpha + \beta d = \text{const}\}$, so β_i is uniform on $\mathbb{Z}/q$, independently of the R_j and hence of $\mathcal{M}$ and λ. A nonzero linear form in independent uniform variables over $\mathbb{F}_q$ is uniform. □

If the rows e_i of $\mathcal{M}$ record decompositions $R_i = \sum_F e_{iF} F$ over a factor base $\mathcal{F}$ and $\lambda^\top \mathcal{M} = 0$ modulo q, then $\sum_i \lambda_i R_i = 0$ in $\langle P \rangle$ (when $\mathcal{F} \subseteq \langle P \rangle$), so $\sum_i \lambda_i \alpha_i + d \sum_i \lambda_i \beta_i \equiv 0$, and d follows whenever $\sum_i \lambda_i \beta_i \ne 0$. With $N_r = |\mathcal{F}| + 1$ rows a nonzero λ exists. No assumption on the rank of $\mathcal{M}$ is needed.

The first assumption, that a constant fraction of points decompose, is the real issue. A fixed box $\{x < X\}$ on a fixed model cannot be handled with the tools we have. The relevant character sums over an interval of length X are nontrivial only for $X \ge p^{1/2+\varepsilon}$ (Kohel and Shparlinski [28]), and $X = p^{1/m}$ is below that for every $m \ge 2$. We therefore average over models.

5.2 Summation polynomials

Theorem S (Semaev [17, Theorem 1])
Let $\mathbb{K}$ be a field of characteristic $\neq 2, 3$ and $E : y^2 = x^3 + Ax + B$ an elliptic curve over $\mathbb{K}$. Define $f_2 = X_1 - X_2$,

$$f_3 = (X_1 - X_2)^2 X_3^2 - 2\big((X_1 + X_2)(X_1X_2 + A) + 2B\big)X_3 + \big((X_1X_2 - A)^2 - 4B(X_1 + X_2)\big),$$

and $f_n = \mathrm{Res}_X\big(f_{n-1}(X_1, \dots, X_{n-2}, X), f_3(X_{n-1}, X_n, X)\big)$ for $n \geq 4$. For $x_1, \dots, x_n \in \overline{\mathbb{K}}$, $f_n(x_1, \dots, x_n) = 0$ if and only if there are $y_i \in \overline{\mathbb{K}}$ with $(x_i, y_i) \in E$ and $\sum_i (x_i, y_i) = O$. For $n \geq 3$, f_n is symmetric of degree 2^{n-2} in each variable, and its coefficients are polynomials in $\mathbb{Z}[A, B]$.

We use Theorem S over $\mathbb{K} = \mathbb{F}_p$ and over the function field $\mathbb{K} = \overline{\mathbb{F}}_p(s)$; its statement allows any field of characteristic $\neq 2, 3$. As an independent sanity check of the equivalence we tested it numerically over $\mathbb{F}_{p^2}$ (15,500 cases, no mismatch).

Lemma 5.2 (weights)
Give X_i, A, B the weights $2, 4, 6$. Then f_n is weighted homogeneous of weight $w_n = (n-1)2^{n-1}$.

Proof. $w_3 = 8$ by inspection. If $F, G \in R[z]$ have exact z-degrees d_F, d_G and weighted homogeneous coefficients (with z of weight 2) of total weights w_F, w_G, then $\mathrm{Res}_z(F, G)$ is weighted homogeneous of weight $d_G w_F + d_F w_G - 2d_F d_G$: the Sylvester determinant is homogeneous of degree d_G in the coefficients of F and d_F in those of G, and the coefficient of z^i has weight $w - 2i$. With $F = f_{n-1}(\dots, X)$ ($d_F = 2^{n-3}$) and $G = f_3$ ($d_G = 2$, $w_G = 8$) this gives $w_n = 2w_{n-1} + 2^{n-1}$, solved by $w_n = (n-1)2^{n-1}$. □

5.3 Isomorphic models and the second moment

For $u \in \mathbb{F}_p^*$ let $E_u : y^2 = x^3 + u^4Ax + u^6B$ and $\phi_u(x, y) = (u^2x, u^3y)$, a group isomorphism $E \to E_u$ over $\mathbb{F}_p$. Let $X = \lfloor p^{1/m} \rfloor$, $\mathcal{I} = [1, X] \subset \mathbb{F}_p^*$, and

$$F^{(u)} = \{P \in E_u(\mathbb{F}_p) \setminus \{O\} : x(P) \in \mathcal{I}\}, \qquad f^{(u)} = |F^{(u)}|.$$

Pulled back to E, $F^{(u)}$ is the dilated box $\{P : u^2x(P) \bmod p \in [1, X]\}$. The m-fold additive energy of a finite set F in an abelian group is

$$E_m(F) = \#\{(P_1, \dots, P_m, Q_1, \dots, Q_m) \in F^{2m} : P_1 + \cdots + P_m = Q_1 + \cdots + Q_m\}.$$

A $2m$-tuple $t \in \mathcal{I}^{2m}$ is *trivial* if every value occurs in it an even number of times. For t write $g_t(s) = f_{2m}(st_1, \dots, st_{2m}; A, B) \in \mathbb{F}_p[s]$.

Lemma 5.3 (degree and scaling)
$\deg g_t \leq D'_{2m} := (2m-1)2^{2m-2}$, and $f_{2m}(t; u^4A, u^6B) = u^{w_{2m}}\, g_t(u^{-2})$.

Proof. Every monomial $x^e A^\alpha B^\beta$ of f_{2m} (with x^e a monomial of total degree e) has $2e + 4\alpha + 6\beta = w_{2m}$ by Lemma 5.2, so $e \leq w_{2m}/2 = D'_{2m}$. The scaling identity is weighted homogeneity applied to $(u^{-2}t, A, B) \mapsto (t, u^4A, u^6B)$. □

The key input is that g_t is not identically zero. We need two facts.

Lemma 5.4 (squares)
Let $\ell(x) = x^3 + Ax + B$ with $4A^3 + 27B^2 \neq 0$ and $r \in \mathbb{F}_p^*$, $r \neq 1$. Then $\ell(x)\ell(rx)$ is a constant times a square in $\overline{\mathbb{F}}_p[x]$ if and only if $A(r - r^3) = 0$ and $B(1 - r^3) = 0$; that is, if and only if $B = 0$ and $r = -1$, or $A = 0$ and $r^3 = 1$.

Proof. ℓ has three simple roots, so $\ell(x)\ell(rx)$ is a constant times a square if and only if the root sets $\{\rho\}$ and $\{\rho/r\}$ coincide, if and only if $\ell(rx) = r^3\ell(x)$, which compares coefficients to $Ar = Ar^3$ and $B = Br^3$. If $A \neq 0$ then $r^2 = 1$, so $r = -1$ and $B = 0$. If $A = 0$ then $B \neq 0$ and $r^3 = 1$. □

Lemma 5.5 (constant torsion)
Let E be an elliptic curve over $\mathbb{F}_p$, let $\mathcal{L}$ be a finitely generated extension of $\overline{\mathbb{F}}_p$ of transcendence degree 1, and let $P \in E(\mathcal{L})$ be a torsion point. Then $P \in E(\overline{\mathbb{F}}_p)$.

Proof. Let C be the smooth projective curve over $\overline{\mathbb{F}}_p$ with function field $\mathcal{L}$. The point P is a rational map $C \to E$, which extends to a morphism φ because C is smooth and E is projective. If $NP = O$, then $[N] \circ \varphi$ is constant, so $\varphi(C) \subseteq [N]^{-1}(O)$, a finite set of closed points (also when $p \mid N$, since $[N]$ is finite). As C is irreducible, φ is constant. □

Lemma N′ (nondegeneracy in the dilation)
Assume $A \neq 0$, or $A = 0$, $B \neq 0$ and $X < p/3$. Let $t \in \mathcal{I}^{2m}$ be nontrivial, where $\mathcal{I} = [1, X]$ with $X < p/2$. Then $g_t \neq 0$ in $\mathbb{F}_p[s]$.

Proof. Let $t_1, \ldots, t_k$ be the distinct values of t, occurring $c_1, \ldots, c_k$ times, with c_{i_0} odd. Suppose $g_t = 0$. Let $\mathcal{K} = \overline{\mathbb{F}}_p(s)$, $Y_i^2 = \ell(st_i)$, $\mathcal{L} = \mathcal{K}(Y_1, \ldots, Y_k)$ and $P_i = (st_i, Y_i) \in E(\mathcal{L})$.

Relation. By Theorem S over $\mathcal{K}$ there are signs with $\sum_j (st_j, \pm Y) = O$; grouping equal x-coordinates gives $\sum_i n_i P_i = O$ with $n_i \equiv c_i \pmod 2$, so n_{i_0} is odd and nonzero.

Characters. $\Gamma = \mathrm{Gal}(\mathcal{L}/\mathcal{K})$ is an elementary abelian 2-group with $\gamma(Y_i) = \chi_i(\gamma)Y_i$, so $\gamma(P_i) = \chi_i(\gamma)P_i$ in $E(\mathcal{L})$, because $-(x, y) = (x, -y)$. By Kummer theory, $\chi_i = \chi_j$ if and only if $\ell(st_i)\ell(st_j)$ is a square in $\mathcal{K}$, if and only if $\ell(x)\ell(rx)$ is a constant times a square with $r = t_j/t_i$. Since $t_i + t_j < p$, $r \neq -1$. Each χ_i is nontrivial: as a polynomial in s, $\ell(st_i)$ is a cubic with three distinct roots, so it is not a square in $\mathcal{K}$.

Case $A \neq 0$. By Lemma 5.4 the χ_i are pairwise distinct. Applying $\sum_\gamma \chi_{i_0}(\gamma)\gamma$ to $\sum_i n_i P_i = O$ kills every term with $\chi_i \neq \chi_{i_0}$ and gives $|\Gamma|\, n_{i_0} P_{i_0} = O$. So P_{i_0} is torsion, hence constant by Lemma 5.5; but $x(P_{i_0}) = st_{i_0}$ is transcendental over $\mathbb{F}_p$. Contradiction.

Case $A = 0$, $B \neq 0$. Now $\chi_i = \chi_j$ exactly when $t_j/t_i \in \{\omega, \omega^2\}$, ω a primitive cube root of unity in $\mathbb{F}_p$ (if $p \equiv 2 \bmod 3$ there is none and the previous case applies verbatim). A class $\{a, \omega a, \omega^2 a\}$ cannot lie in $[1, X]$: $\omega^2 a \equiv -(a + \omega a)$, whose representative $p - a - \omega a$ exceeds $p - 2X > X$. So each character class has at most two elements. Let $\zeta(x, y) = (\omega x, y)$, an automorphism of E with $1 + \zeta + \zeta^2 = 0$. If $t_j = \omega^e t_i$ with $e = \pm 1$, then $Y_j^2 = Y_i^2$ and $P_j = \epsilon\, \zeta^e P_i$ with $\epsilon = \pm 1$. Applying the projector of the class of i_0 gives $\psi P_{i_0} = O$ with $\psi = n_{i_0}$ or $\psi = n_{i_0} + \epsilon n_j \zeta^e$. Since $(1, \zeta)$ is a $\mathbb{Z}$-basis of $\mathbb{Z}[\zeta]$ and $\zeta^{-1} = -1 - \zeta$, $\psi = 0$ would force $n_{i_0} = 0$; so $\psi \neq 0$. A nonzero endomorphism has finite kernel, so P_{i_0} is torsion, and we conclude as before. □

The hypothesis $X < p/3$ is needed when $A = 0$. If t contains a full orbit $(a, \omega a, \omega^2 a)$ then $P + \zeta P + \zeta^2 P = O$ and g_t vanishes. For instance, at $p = 1033$, $B = 3$ the relation holds for 243 values of s, far more than $D_6' = 80$. For $A \neq 0$ we also confirmed Lemma 5.4 by direct computation over all nonsingular (A, B) and all r for seven primes up to 43 (161,084 cases).

Proposition 5.6 (energy over the dilations of one curve)
Under the hypotheses of Lemma N′, with $\Theta_m = (2m-1)!!\,4^m$,

$$\frac{1}{p-1}\sum_{u\in\mathbb{F}_p^*} E_m(F^{(u)}) \le \mu := \Theta_m X^m + \frac{2\cdot 4^m D'_{2m} X^{2m}}{p-1}.$$

Proof. Count $(u; P_1,\dots,P_m,Q_1,\dots,Q_m)$ with all points in $F^{(u)}$ and $\sum P_i = \sum Q_j$, and let $t = (x(P_1),\dots,x(Q_m)) \in \mathcal{I}^{2m}$. Trivial t: there are at most $(2m-1)!!\,X^m$ of them (a perfect matching of the $2m$ positions and one value per pair), each with at most 4^m sign choices, for each of the $p-1$ values of u. Nontrivial t: $\sum P_i - \sum Q_j = O$ on E_u, so $f_{2m}(t; u^4A, u^6B) = 0$ by Theorem S over $\mathbb{F}_p$, i.e. $g_t(u^{-2}) = 0$ (Lemma 5.3). As $g_t \neq 0$ (Lemma N′) has degree at most D'_{2m}, there are at most D'_{2m} values of s and at most $2D'_{2m}$ values of u; each has at most 4^m sign choices, and there are at most X^{2m} tuples t. □

Lemma F (size of the factor base)
Let $p \ge 100$, $X \le \sqrt{p}$, and $A \neq 0$, or $A = 0$, $B \neq 0$. Then

$$\frac{1}{p-1}\sum_{u\in\mathbb{F}_p^*} (f^{(u)} - X)^2 \le 13X \ (A\neq 0), \qquad \le 13X+2 \ (A = 0),$$

so $\Pr_u[f^{(u)} \le X/2] \le 52/X$, respectively $\le 56/X$ for $X \ge 2$.

Proof. Let χ be the quadratic character. Since $x^3 + u^4Ax + u^6B = u^6\ell(u^{-2}x)$, $f^{(u)} - X = \sum_{t\in\mathcal{I}} \chi(\ell(st))$ with $s = u^{-2}$, and $\sum_{u\in\mathbb{F}_p^*} \Phi(u^{-2}) = \sum_{s\neq 0}(1+\chi(s))\Phi(s)$. Expanding the square, a diagonal pair $t = t'$ contributes at most $p-1$. For $t \neq t'$ put $r = t'/t$ and $x = st$:

$$\sum_{s\neq 0}(1+\chi(s))\chi(\ell(st)\ell(st')) = \sum_{x\neq 0}\chi(\ell(x)\ell(rx)) + \chi(t)\sum_{x\neq 0}\chi\big(x\,\ell(x)\ell(rx)\big).$$

If $A \neq 0$, neither $\ell(x)\ell(rx)$ (Lemma 5.4, $r \neq -1$) nor $x\ell(x)\ell(rx)$ (the factor x is simple if $B \neq 0$, and occurs to the third power if $B = 0$) is a constant times a square. By Weil's bound [29] ($|\sum_{x\in\mathbb{F}_p}\chi(h(x))| \le (\delta-1)\sqrt{p}$ for h not a constant times a square, with δ distinct roots), accounting for the at most two common roots of $\ell(x)$ and $\ell(rx)$ and for the excluded term $x = 0$, the two sums are at most $5\sqrt{p}+3$ and $6\sqrt{p}+2$. Hence

$$\frac{1}{p-1}\sum_u (f^{(u)} - X)^2 \le X + \frac{X(X-1)(11\sqrt{p}+5)}{p-1} \le X\Big(12 + \frac{5\sqrt{p}+11}{p-1}\Big) \le 12.62\,X$$

for $X \le \sqrt{p}$ and $p \ge 100$. If $A = 0$, the pairs with $r \in \{\omega, \omega^2\}$ have no Weil bound and contribute at most $p-1$ each. There are at most two such ordered pairs in $[1,X]^2$: the vectors $(a,b) \in [1,X]^2$ with $b \equiv \omega a$ form a lattice of index p; two of them have $ab' - a'b \equiv 0$ and $|ab' - a'b| < X^2 \le p$, so they are parallel; and $a^2 + ab + b^2$ is a positive multiple of p but less than $3p$, which forces the primitive vector. Chebyshev's inequality gives the probabilities. □

Exact enumeration over all u for $p \le 20011$ (generic, $j = 0$ and $j = 1728$ curves) gives $\frac{1}{p-1}\sum_u (f^{(u)} - X)^2/X \le 1.39$. The constant 13 is loose.

Theorem 3 (enough points decompose)

Let $E : y^2 = x^3 + Ax + B$ be nonsingular over $\mathbb{F}_p$; let $m \geq 2$, $X = \lfloor p^{1/m} \rfloor \geq \max(12m, 2)$, $p \geq 100$ and $\kappa \geq 1$. With $c'_m = \Theta_m + 3 \cdot 4^m D'_{2m}$ and $\eta = 52$ ($A \neq 0$) or $\eta = 56$ ($A = 0$), with probability at least $1 - 1/\kappa - \eta/X$ over u uniform in $\mathbb{F}_p^*$: $f^{(u)} \geq X/2$, $E_m(F^{(u)}) \leq \kappa\mu \leq \kappa c'_m p$, and the set G_u of points $R \in E(\mathbb{F}_p)$ such that $\phi_u(R)$ has a generic decomposition into points of $F^{(u)}$ satisfies

$$\frac{|G_u|}{n} \geq \frac{(f^{(u)})^{2m}}{4E_m(F^{(u)})\, n} \geq c_m(\kappa) := \frac{1}{8 \cdot 16^m \kappa c'_m}.$$

Theorem 3 uses no engine and no hypothesis. Its inputs are Proposition 5.6 (through Lemma N′) and Lemma F.

Proof. Markov's inequality on Proposition 5.6, Lemma F, and a union bound give the first two properties. Also $\mu \leq c'_m p$ since $X^m \leq p$, $X^{2m} \leq p^2$ and $p^2/(p-1) \leq 1.5p$. Let $f = f^{(u)} \geq X/2 \geq 6m$. The non-generic m-tuples of $F^{(u)}$ number at most $(3m-2)f^{m-1} \leq f^m/2$ (as $f \geq 6m-4$): for each $i \in \{2, \ldots, m\}$, at most f^{m-1} tuples have $S_{i-1} = O$ and at most $2f^{m-1}$ have $S_{i-1} = \pm P_i$, and at most f^{m-1} have final sum $S_m = O$. Let $r_g(R)$ count generic tuples with sum R; it vanishes at $R = O$. Then $\sum_R r_g(R) \geq f^m/2$ and $\sum_R r_g(R)^2 \leq E_m$, so by Cauchy–Schwarz the number of R with $r_g(R) > 0$ is at least $f^{2m}/(4E_m)$. Finally $f^{2m} \geq (X/2)^{2m} \geq (p^{1/m}/4)^{2m} = p^2/16^m$ and $n \leq 2p$. □

For $m = 2$: $\Theta_2 = 48$, $D'_4 = 12$, $c'_2 = 624$, and $c_m(\kappa) = 1/(1.28 \cdot 10^6 \kappa)$. The truth is far better. Over all dilations at $p \in \{1009, 4099, 10007\}$ the mean coverage is 0.39–0.42 and $E_2/X^2 \approx 4$; for $y^2 = x^3 + B$ the numbers are the same.

5.4 Discrete logarithms for prime or squarefree order

Theorem 4

Let $\mathcal{D}$ be the decomposition oracle (O), provided by Theorem 2(ii), a Lean theorem with no hypotheses. Fix $m \geq 2$. Let $E/\mathbb{F}_p$, $p > 3$, be any nonsingular curve $y^2 = x^3 + Ax + B$ such that $E(\mathbb{F}_p) = \langle P \rangle$ is cyclic of prime or squarefree order n, and let $Q = dP$. Assume $X = \lfloor p^{1/m} \rfloor \geq \max(12m, 208)$. Then the Las Vegas algorithm below finds d in expected time $p^{2/m+o(1)}$ and always verifies its answer. The constants are uniform over all such curves, with no averaging over curves.

Algorithm. Compute n by Schoof's algorithm [30] and factor n (in time $p^{o(1)}$, for instance by the rigorous algorithm of Lenstra and Pomerance [31]). Fix $\kappa = 4$. Repeat:

1. Draw u uniformly from $\mathbb{F}_p^*$ and compute $P' = \phi_u(P)$, $Q' = \phi_u(Q)$.

2. Compute $f' = |F'|$ exactly, where F' is the oracle's factor base $\{x < \lceil p^{1/m} \rceil\}$ on E_u (Euler's criterion, time $X \cdot \operatorname{polylog} p$); $F^{(u)} \subseteq F'$ and $f' \leq f^{(u)} + 2$. Put $N_r = f' + 1$.

3. Relation phase, with a cap of $2N_r/c_m(\kappa)$ oracle calls: draw (α, β) uniform in $(\mathbb{Z}/n)^2$, put $R = \alpha P' + \beta Q'$, call $\mathcal{D}(R)$ on E_u; if it returns a decomposition, record the row $e \in \mathbb{Z}^{F'}$ of multiplicities and the pair (α, β). Stop at N_r rows. If the cap is reached first, go to step 1.

4. For each prime $q \mid n$, compute a nonzero $\lambda^{(q)} \in \mathbb{F}_q^{N_r}$ with $\lambda^{(q)\top}\mathcal{M} \equiv 0 \pmod q$ by sparse linear algebra ($\tilde{O}(mf'^2)$ operations, see below); if $\sum_i \lambda_i^{(q)}\beta_i \not\equiv 0$, put
$$d_q = -\Big(\sum_i \lambda_i^{(q)}\alpha_i\Big)\Big(\sum_i \lambda_i^{(q)}\beta_i\Big)^{-1} \bmod q;$$
otherwise go to step 1.

5. Combine the d_q by the Chinese remainder theorem; if $dP = Q$ output d, otherwise go to step 1.

Factor-base points are labeled by (x, s) with the sign bit of Remark 4.1. For the linear algebra we use Wiedemann's algorithm [32], whose preconditioned (singular-case) analysis bounds the success probability below only over fields that are large compared with the matrix [32, §§III, V]. For a prime q with $q \le f'^2$ we therefore run it over an extension $\mathbb{F}_{q^k}$ with $q^k > f'^2$, $k = O(\log f')$: a nonzero kernel vector over $\mathbb{F}_{q^k}$, written in an $\mathbb{F}_q$-basis of $\mathbb{F}_{q^k}$, gives k vectors over $\mathbb{F}_q$ in the kernel, at least one of them nonzero. This costs a factor $\tilde{O}(k^2) = \mathrm{polylog}(p)$, and the result is checked, so failures only cause a repetition. Alternatively one can cite input (W′) of Section 6.

Proof. Correctness. Since n is squarefree, $\langle P'\rangle = E_u(\mathbb{F}_p) \cong \prod_q \mathbb{Z}/q$, and modulo each q the argument after Lemma 5.1 applies to the projections of all points. Any d that is output satisfies $dP = Q$, whatever u was drawn.

Probability that a round succeeds. Call u good if the conclusions of Theorem 3 hold with $\kappa = 4$. This has probability at least $1 - 1/4 - 56/X \ge 0.48$ for $X \ge 208$. Since P' generates $E_u(\mathbb{F}_p)$, each R is uniform, and for a good u the oracle succeeds whenever $\phi_u^{-1}(R) \in G_u$, which has probability at least $c = c_m(4)$; it may succeed more often, since $F' \supseteq F^{(u)}$. The number of successes among $2N_r/c$ calls dominates a binomial with mean $2N_r$, so by Chernoff's bound the cap is reached with probability at most $e^{-N_r/4}$. Given that N_r rows were collected, Lemma 5.1 applies modulo each q: only the rows of the current round are used, $\mathcal{M}$ is a function of the R_i (the oracle is deterministic and the stopping rule depends only on the R_i), and by the Chinese remainder theorem the $\beta_i \bmod q$ are uniform and independent across q given the R_i. So all $\sum_i \lambda_i^{(q)}\beta_i$ are nonzero with probability $\prod_{q\mid n}(1 - 1/q) = \varphi(n)/n$. A round therefore succeeds with probability at least $0.45\,\varphi(n)/n$, and $n/\varphi(n) = O(\log\log n)$ by the bound of Rosser and Schoenfeld [33]. For prime n the factor is $1 - 1/n$.

Cost. A round costs at most $2N_r/c_m(4)$ oracle calls of cost $p^{o(1)}$, plus $O(mf'^2)\,\mathrm{polylog}\,p$ for the linear algebra over at most $\log_2 n$ primes, plus $X\,\mathrm{polylog}\,p$. With $f' \le 2X + 2 \le 3p^{1/m}$, this is $p^{1/m+o(1)} + p^{2/m+o(1)}$. The expected number of rounds is $O(\log\log p)$. Schoof's algorithm takes $\mathrm{poly}(L)$ and factoring $p^{o(1)}$. □

Remarks. (1) The statement is asymptotic: $X \ge 208$ makes the probability bounds non-vacuous, and the $o(1)$ hides the oracle cost. Rho's exponent $1/2$ is beaten for $m \ge 5$, and only past the threshold of Section 10. (2) The algorithm randomizes over models of the given curve. Whether the plain box $u = 1$ already has coverage bounded below is open (Section 10); on all our data, coverage at $u = 1$ equals the average over u. (3) Theorem 4 covers every curve of cryptographic shape, including curves with $j = 0$ such as $y^2 = x^3 + 7$, for which the repair in Lemma N′ is needed. (4) An earlier version of the argument averages over all curves $(A, B) \in \mathbb{F}_p^2$ instead of over the models of one curve. There, the nondegeneracy of $f_{2m}(t; A, B)$ as a polynomial in (A, B) is proved by degenerating to the nodal curve $(A, B) = (-3u^2, 2u^3)$ and a valuation

argument, and the energy is bounded by Schwartz–Zippel. To transfer it to prime-order curves, one restricts to the subfamily of prime-order pairs, which needs a hypothesis on primes in short intervals around $p+1$ (true for almost all p by Guth and Maynard [34]). Theorem 4 supersedes it; we mention it because its coverage chain is formalized (Section 8).

6 Arbitrary group orders

Theorem 4 needs $E(\mathbb{F}_p)$ cyclic of squarefree order. This section removes that restriction. Notation: $G = E(\mathbb{F}_p) \cong \mathbb{Z}/n_1 \times \mathbb{Z}/n_2$ with $n_2 \mid n_1$, $N = |G| \le 2p$, $\exp(G) = n_1$; for a prime $\ell \mid N$ (in this section ℓ always denotes a prime, not the engine level of Section 2), G_ℓ is the ℓ-Sylow subgroup and $\pi_\ell : G \to G_\ell$ the projection; $P \in G$ is arbitrary and $Q \in \langle P \rangle$. The inputs are:

(O) the oracle of Section 5, provided by Theorem 2(ii), a Lean theorem with no hypotheses (Section 8);

(C1) Theorem 3 (coverage on a fixed curve in a random model, including $A = 0$);

(C2) Schoof's algorithm for N [30];

(C3) factoring N in time $N^{o(1)}$: either the proved bound of Rudich and Rousseau [24] (time $C_k 2^{n/k}$ per prime factor, for every k, by their Theorem 1, which is formalized in Lean; complete factorization is their Theorem 2; an unrefereed preprint) or the rigorous Las Vegas algorithm of Lenstra and Pomerance [31], expected time $L_N[1/2, 1+o(1)]$;

(C4) the Rosser–Schoenfeld bound $n/\varphi(n) \le e^\gamma \ln\ln n + 3/\ln\ln n$ for $n \ge 3$ [33].

6.1 What goes wrong, and the fix

The index calculus of Section 5 uses four things that fail in general. (F1) It needs logarithms of factor-base points to base P, which exist only inside $\langle P \rangle$; the oracle returns points of all of G. (F2) It needs targets uniform on a set that the coverage bound controls; $R = aP + bQ$ is uniform on $\langle P \rangle$, which can be tiny, while Theorem 3 is about G. (F3) A kernel vector modulo $\mathrm{ord}(P)$ gives a valid relation only if it is a kernel vector modulo a multiple of $\exp(G)$. (F4) Pohlig–Hellman projection of relations lands in G_ℓ, where $\langle \pi_\ell(P) \rangle$ need not be a direct summand.

The fix is to compute the lattice of relations among a generating tuple of G, not logarithms. Draw two random points H_1, H_2 and sample $R = a_1 H_1 + a_2 H_2 + a_3 P + a_4 Q$ with $a \in (\mathbb{Z}/N)^4$ uniform. When $\langle H_1, H_2 \rangle = G$, R is uniform on G and (C1) applies. The relation lattice contains $(0, 0, d, -1)$, and d can be read off it.

6.2 Algorithm

Fix $\kappa = 4$. Compute N (C2) and its factorization (C3). Repeat rounds:

1. Draw $u \in \mathbb{F}_p^*$ uniformly. Draw H_1, H_2: pick $x \in \mathbb{F}_p$ uniformly and accept if $x^3 + Ax + B$ is a nonzero square (a zero is accepted with probability $1/2$), with a random square root as y. Every point of $G \setminus \{O\}$ then has probability at least $1/(4N)$. Put $(V_1, V_2, V_3, V_4) = (H_1, H_2, P, Q)$.

2. Compute the oracle factor base F_u on E_u (pulled back to E), $f_u = |F_u|$, and put $N_r = f_u + 12$.

3. Relation phase, with a cap of $2N_r/c_m(4)$ samples: draw $a \in (\mathbb{Z}/N)^4$, put $R = \sum_j a_j V_j$; if $R \neq O$, call $\mathcal{D}$ on $\phi_u(R)$; on success record the row $e_i \in \mathbb{Z}^{F_u}$ and $a_i = a$. Stop at N_r rows; if the cap is hit, end the round.

4. For each prime $\ell \mid N$, with $v = v_\ell(N)$ and $R_\ell = \mathbb{Z}/\ell^v$: (a) compute $s \geq 12$ vectors $\Lambda_1, \ldots, \Lambda_s \in R_\ell^{N_r}$ with $\Lambda_j^\top \mathcal{M} = 0$ (Lemma 6.1); (b) put $\mu_j = \sum_i \Lambda_{ji} a_i \in R_\ell^4$; (c) decide whether some $(0, 0, d', -1)$ lies in the R_ℓ-span of the μ_j (Lemma 6.5); if so let $d_\ell = d'$, else end the round.

5. Combine the d_ℓ by the Chinese remainder theorem to d''; output d'' if $d''P = Q$.

Any output satisfies $d''P = Q$, i.e. $d'' \equiv d \pmod{\text{ord}(P)}$, which is all the problem determines. Correctness never depends on any hypothesis; the hypotheses only bound the running time. The algorithm uses N and its factorization, but neither (n_1, n_2) nor $\text{ord}(P)$.

6.3 Lemmas

Let $\text{Rel}_\ell = \{\mu \in R_\ell^4 : \sum_j \mu_j \pi_\ell(V_j) = 0\}$, the ℓ-relation lattice.

Lemma 6.1 (elimination over $\mathbb{Z}/\ell^v$)
Given an $N_r \times f$ matrix $\mathcal{M}$ over R_ℓ, row operations with pivots of minimal ℓ-adic valuation produce an invertible U such that the rows of U belonging to the $s \geq N_r - f$ non-pivot rows, collected in Λ, satisfy $\Lambda\mathcal{M} = 0$, and Λ has a right inverse over R_ℓ. The cost is $O(N_r^2(f + N_r))$ ring operations.

Proof. In the active submatrix choose an entry $\ell^k w$ of minimal valuation, w a unit. Every other entry of its column is divisible by ℓ^k, so subtracting multiples of the pivot row clears the column exactly. Mark the pivot row and column inactive and repeat until the active submatrix is zero; there are at most f pivots. The transformation U is invertible, each non-pivot row of $U\mathcal{M}$ is zero, and the corresponding columns of U^{-1} give a right inverse of Λ. □

Lemma 6.2 (relations are relations)
If $\lambda \in R_\ell^{N_r}$ and $\lambda^\top \mathcal{M} \equiv 0 \pmod{\ell^v}$, then $\mu = \sum_i \lambda_i a_i \in \text{Rel}_\ell$. Hence $(0, 0, d, -1) \in \text{Rel}_\ell$, and every $(0, 0, d', -1) \in \text{Rel}_\ell$ satisfies $d'\pi_\ell(P) = \pi_\ell(Q)$.

Proof. $\sum_i \lambda_i R_i = \sum_F (\lambda^\top \mathcal{M})_F F$, and every coefficient on the right is divisible by ℓ^v. Applying π_ℓ kills the right side, because ℓ^v annihilates G_ℓ; the left side is $\sum_j \mu_j \pi_\ell(V_j)$. □

Lemma 6.3 (uniformity)
Assume $\langle V_1, \ldots, V_4 \rangle = G$ and condition on the sequence of accepted R_i. Then $a_i \bmod \ell^v = \alpha_0(R_i) + k_i$ for a fixed section α_0, with k_i independent and uniform on Rel_ℓ. If Λ $(s \times N_r)$ is a function of the R_i and of randomness independent of the tags a_i, satisfies $\Lambda\mathcal{M} = 0$ over R_ℓ and has rank s modulo ℓ, then $\mu_1, \ldots, \mu_s$ are independent and uniform on Rel_ℓ.

Proof. For a section α_0 of the surjection $(\mathbb{Z}/N)^4 \to G$, $a \mapsto (R(a), a - \alpha_0(R(a)))$ is a bijection onto $G \times \ker$, so R and the kernel part are independent and uniform; the kernel reduces onto Rel_ℓ (lift by the Chinese remainder theorem). Rank s modulo ℓ gives W_0 with $\Lambda W_0 = I + \ell\mathcal{N}$, invertible since $\ell\mathcal{N}$ is nilpotent, so Λ has a right inverse and $k \mapsto \Lambda k$ maps $\text{Rel}_\ell^{N_r}$ onto Rel_ℓ^s. The image of a uniform element is uniform. By Lemma 6.2 each $(\Lambda\alpha_0)_j \in \text{Rel}_\ell$, and a fixed element plus a uniform element of Rel_ℓ is uniform. □

The tags supply the randomness. The kernel vectors themselves need not be random: any Λ of full rank modulo ℓ works, including a structured basis.

Lemma 6.4 (generation)
A finite abelian ℓ-group with at most 4 generators is generated by s independent uniform elements with probability at least $1 - 2\ell^{3-s}$. For $s = 12$, the failure probabilities summed over all primes are below 0.0045.

Proof. By the Frattini argument, the elements generate if and only if their images span $H/\ell H \cong \mathbb{F}_\ell^k$, $k \le 4$, which happens with probability $\prod_{i<k}(1-\ell^{i-s})$. $\mathrm{Rel}_\ell \le R_\ell^4$ has at most 4 generators. The sum is $2\sum_\ell \ell^{-9} < 2(\zeta(9)-1) < 0.0045$. □

Lemma 6.5 (reading off d)
Given generators of a subgroup $\mathcal{S}_0 \le R_\ell^4$, one can decide in polynomial time whether $(0,0,d',-1) \in \mathcal{S}_0$ for some d', and find such d'.

Proof. For column $c = 1, 2$ in turn: take a generator π whose c-th entry $\ell^k w$ has minimal valuation; replace every other generator r by $r - (r_c/\ell^k)w^{-1}\pi$; add $\ell^{v-k}\pi$ and delete π. An element $\alpha\pi + \sum_r \beta_r r'$ has c-th entry $\alpha\ell^k w$, which is 0 if and only if $\ell^{v-k} \mid \alpha$, so the new set spans $\mathcal{S}_0 \cap \{z_c = 0\}$. After both columns, $(0,0,d',-1)$ is in the span if and only if some generator has a unit fourth entry r_4, and then $d' = -r_3/r_4$. □

6.4 Every curve: exponent $3/m$

Theorem 5
Let the oracle (O) be the one provided by Theorem 2(ii), and use the inputs (C1)–(C4). Fix $m \ge 2$ and let $X = \lfloor p^{1/m} \rfloor \ge \max(12m, 300)$. For every nonsingular curve $y^2 = x^3 + Ax + B$ over $\mathbb{F}_p$, with $G = E(\mathbb{F}_p)$ of arbitrary order and structure, and all $P \in G$, $Q \in \langle P \rangle$, the algorithm above outputs d with $dP = Q$, always correctly, in expected time $p^{3/m+o(1)}$.

Proof. Correctness is Lemma 6.2 and the final check. A round succeeds if: (a) $\langle H_1, H_2 \rangle = G$; (b) u is good in the sense of Theorem 3 with $\kappa = 4$, which has probability at least $1 - 1/4 - 56/X \ge 0.56$ and is independent of (a); (c) the cap is not hit, which given (a) and (b) fails with probability at most $e^{-N_r/4}$, since R is uniform on G and each sample is accepted with probability at least $c_m(4)$; (d) the μ_j generate every Rel_ℓ, with probability at least 0.995 by Lemmas 6.1, 6.3 and 6.4. Then Lemma 6.5 finds d_ℓ at every ℓ.

For (a): a uniform pair generates G if and only if its images span $G/\ell G$ for every $\ell \mid N$, which has probability $\prod_{\ell \mid N,\ \ell \nmid n_2}(1-\ell^{-2}) \prod_{\ell \mid n_2}(1-\ell^{-1})(1-\ell^{-2}) \ge (6/\pi^2)\,\varphi(n_2)/n_2$. Since $n_2^2 \le 2p$, (C4) gives $\varphi(n_2)/n_2 \ge 1/(e^\gamma \ln L + 3)$ up to constants. The near-uniform sampling of H_1, H_2 loses a factor 16. So $\sigma_{\mathrm{gen}} \ge (6/\pi^2)/(16(e^\gamma \ln L + 3))$, and a round succeeds with probability at least $\sigma_{\mathrm{gen}}/2$.

A round costs $(2N_r/c_m(4))(T_{\mathcal{D}} + \mathrm{poly}(L))$ for the relation phase, which is $p^{1/m+o(1)}$, plus dense elimination $O(\log N \cdot N_r^3)\,\mathrm{poly}(L) = p^{3/m+o(1)}$, with $N_r \le 2X + 15 \le 3p^{1/m}$. The expected number of rounds is $O(\log L)$, and Schoof's algorithm and factoring cost $p^{o(1)}$. □

6.5 Squarefree exponent or $m \le 8$: exponent $2/m$

The cubic term comes from dense elimination. Two facts remove it. First, kernel vectors need only be independent modulo ℓ (Lemma 6.3). Second, uniform kernel samples can be produced by solving consistent sparse systems.

Lemma 6.6 (uniform kernel samples)
Let $\mathcal{A}$ be a matrix over a finite field and σ any (possibly randomized, independently of y) procedure returning a solution of $\mathcal{A}x = b$ for consistent b. For y uniform, $z = y - \sigma(\mathcal{A}y)$ is exactly uniform on $\ker \mathcal{A}$.

Proof. On each coset $y_0 + \ker \mathcal{A}$ the right-hand side $\mathcal{A}y$ is constant, so $z = (y_0 - \sigma(\mathcal{A}y_0)) + (y - y_0)$ is a fixed kernel element plus a uniform one. □

Input (W′)
A Las Vegas algorithm that solves a consistent linear system with a sparse matrix of rank r over any finite field, using $O(r)$ products by the matrix or its transpose and $O(rN \log^2 N)$ further field operations. We cite this from Chen, Eberly, Kaltofen, Saunders, Turner and Villard [35, §6], whose introduction states that Wiedemann's sparse preconditioners solve such systems over small finite fields without binary search. We have read their Proposition 6.2 and Corollary 6.3, but not the reduction in their Section 2 on which the operation count rests.

Lemma 6.7 (enough kernel vectors)
Let $\dim \ker(\mathcal{M}^\top \bmod \ell) \geq 12$. Twenty-four independent uniform samples contain 12 vectors with independent reductions modulo ℓ with probability at least $1 - 2\ell^{-12}$, and the resulting μ_j generate Rel_ℓ with probability at least $1 - 2\ell^{-9}$. Summed over all primes, the failure probability is below 0.0045.

Proof. Project onto 12 coordinates of a basis of the kernel: 24 uniform vectors of $\mathbb{F}_\ell^{12}$ span it with probability at least $1 - 2\ell^{-12}$. Then apply Lemmas 6.3 and 6.4. □

Theorem 6
Let the oracle (O) be the one provided by Theorem 2(ii), use the inputs (C1)–(C4) and (W′), and let p, m, X, E, P, Q be as in Theorem 5.

(i) If $\exp(E(\mathbb{F}_p))$ is squarefree (any order and structure, including non-cyclic groups all of whose Sylow subgroups are elementary abelian of rank at most 2), then for every fixed $m \geq 2$ a Las Vegas algorithm solves the instance in expected time $p^{2/m+o(1)}$.

(ii) For every curve, of any group order and structure, and every m with $2 \leq m \leq 8$, a Las Vegas algorithm solves the instance in expected time $p^{2/m+o(1)}$.

Proof. (i) Replace step 4 of the algorithm, for each prime $\ell \mid N$ with $v_\ell(e_B) = 1$ (where $e_B = \mathrm{lcm}$ of the orders of H_1, H_2, P, Q; on the event $\langle H_1, H_2 \rangle = G$, $e_B = \exp(G)$), by: draw 24 samples of $\ker(\mathcal{M}^\top \bmod \ell)$ with Lemma 6.6 and (W′), keep 12 with independent reductions, form the μ_j modulo ℓ, and apply Lemma 6.5 with $v = 1$. Lemma 6.2 holds with modulus ℓ because $\ell G_\ell = 0$. Lemma 6.7 replaces the probability 0.995 by 0.99. The matrix has $N_r \leq 3p^{1/m}$ rows with at most m nonzero entries each, so one sample costs $O(f)$ products of cost $O(mN_r)$ plus $O(fN_r \log^2 N_r)$, i.e. $\tilde{O}(mp^{2/m})$ operations over $\mathbb{F}_\ell$, for at most L primes. There is no cubic step. The algorithm does not need to know that the exponent is squarefree: rounds with $\langle H_1, H_2 \rangle \neq G$ may produce garbage, which the final check rejects.

(ii) Pohlig–Hellman [36] needs no structure of G: for $\ell \mid N$ and $a_\ell = v_\ell(\mathrm{ord}\, P)$, the points $P_\ell = (N/\ell^{v_\ell(N)})P$ and Q_ℓ satisfy $Q_\ell = dP_\ell$ in a cyclic group of order ℓ^{a_ℓ}, and $d \bmod \ell^{a_\ell}$ is found digit by digit with baby-step giant-step in $O(L\sqrt{\ell})$ group operations. If $\ell^2 \mid N$ then $\ell \leq \sqrt{2p}$, so this costs $p^{1/4+o(1)} \leq p^{2/m+o(1)}$ for $m \leq 8$. Use Pohlig–Hellman for every prime with $\ell^2 \mid N$;

every other prime divides N exactly once, so $v_\ell(\exp G) = 1$ and the field path of (i) applies. The Chinese remainder theorem combines the residues, and the final check makes the algorithm Las Vegas. □

For $m \le 8$, the range $5 \le m \le 8$ is where $p^{2/m}$ beats $p^{1/2}$. More generally the proof of (ii) gives $p^{2/m+o(1)}$ for every m and every curve such that no prime $\ell > 2p^{4/m}$ has $\ell^2 \mid \exp(G)$. For the remaining curves ($m \ge 9$ and a prime $2p^{4/m} < \ell \le \sqrt{2p}$ with $\ell^2 \mid \exp(G)$), Pohlig–Hellman costs $\sqrt{\ell} > p^{2/m}$. Lifting kernel vectors of $\mathcal{M}$ from $\mathbb{Z}/\ell$ to $\mathbb{Z}/\ell^2$ is obstructed by the ℓ-torsion of $\mathbb{Z}^f/(\text{row module of } \mathcal{M})$, and this obstruction is real. An oracle that satisfies (O) can return, whenever it has a choice, decompositions orthogonal modulo ℓ to a hidden character. In experiments with $\ell = 41$ and $f = 80, 100$, such an oracle made the kernel non-liftable in 20 of 20 rounds. So an argument about the matrix the oracle happens to produce need not work from our hypotheses; we have not proved that it cannot. Theorem 1 changes the sampling instead.

6.6 Every curve, every m, by a planted diagonal

The idea is to plant a random multiple of each factor-base point on the diagonal. The i-th of the first f rows is sampled from $R_i = a_i \cdot V + \nu F_i$, where $F_1, \dots, F_f$ are the factor-base points and ν is one random integer for the whole round. The target R_i is still uniform on G, so the oracle's answers carry no information about ν, and the top $f \times f$ block of the relation matrix is $T' - \nu I$, where T' does not depend on ν. Its determinant is $\det(T' - \nu I) = (-1)^f \det(\nu I - T')$, up to sign the characteristic polynomial of T' evaluated at ν, which is monic, so for every large prime it vanishes modulo ℓ for few values of ν. With this block invertible, kernel vectors modulo ℓ^v come from nonsingular sparse solves.

Input (W″)
Wiedemann's algorithm [32, §§II, VI] for a nonsingular sparse system over a finite field $\mathbb{F}_\ell$ of size f: the minimal polynomial of a Krylov sequence, computed by Berlekamp–Massey [37], yields the solution with $O(f)$ products by the matrix and $O(f^2)$ further operations. The Krylov step succeeds with probability at least $1 - O(f)/\ell$, and success is verified by one more product. The solution is lifted from $\mathbb{Z}/\ell$ to $\mathbb{Z}/\ell^v$ by ℓ-adic lifting as in Dixon [38]: solve modulo ℓ, subtract, divide the right-hand side by ℓ, repeat. Wiedemann's success bound is used with $\ell > 4f^2$, within its hypotheses.

Algorithm H″. As the algorithm of Theorem 5, with the following changes in each round. Let $f = f_u$, $w = \omega(N)$ (the number of prime factors of N), and $L_0 = \nu_0 = \max(8fw, 4f^2)$. Draw ν uniformly from $\{0, \dots, \nu_0 - 1\}$.

(i) For every prime $\ell \mid N$ with $\ell \le L_0$, compute $d \bmod \ell^{a_\ell}$ by Pohlig–Hellman, as in the proof of Theorem 6(ii).

(ii) Planted rows: for $i = 1, \dots, f$, draw a uniform in $(\mathbb{Z}/N)^4$, put $R = a \cdot V + \nu F_i$, redraw a if $R = O$, and query the oracle on $\phi_u(R)$ until it accepts. Record the row $e^{(i)} = e_{\mathcal{D}(R_i)} - \nu\, \mathbf{e}_i$ (with $\mathbf{e}_i$ the i-th unit vector) and the tag a_i. Then draw 12 ordinary rows ($R = a \cdot V$) as before. All draws share the cap $2(f + 12)/c_m(4)$.

(iii) For every prime $\ell \mid N$ with $\ell > L_0$ and $v = v_\ell(N)$: let $\mathcal{B}$ be the top $f \times f$ block, and for $k = 1, \dots, 12$ let r_k be the k-th ordinary row. Compute $x_k = -r_k \mathcal{B}^{-1}$ over $\mathbb{Z}/\ell^v$ by (W″),

put $\lambda_k = (x_k, \mathbf{e}_k) \in R_\ell^{f+12}$ and $\mu_k = \sum_i \lambda_{k,i} a_i$, and apply Lemma 6.5. If $\mathcal{B}$ is singular modulo ℓ (detected by the solver), end the round.

(iv) Combine all residues by the Chinese remainder theorem; output d'' if $d''P = Q$.

Each planted row is a true relation: $\sum_j e_j^{(i)} F_j = \mathcal{D}(R_i)\text{-sum} - \nu F_i = R_i - \nu F_i = a_i \cdot V$. So Lemma 6.2 holds for the whole matrix $\mathcal{M}$, and $\lambda_k \mathcal{M} = x_k \mathcal{B} + r_k = 0$.

Lemma 6.8 (the oracle learns nothing about ν)
Assume $\langle H_1, H_2 \rangle = G$. Let the oracle be arbitrary: deterministic, randomized with coins independent of ν, or stateful and adaptive. Then the joint law of all queried points, all oracle answers, the stopping and cap decisions, and hence of $T' = (e_{\mathcal{D}(R_i)})_{i \le f}$, does not depend on ν. Conditional on all of these and on ν, the tag of each row is $\alpha_0(R - \nu F_i) + k$ (or $\alpha_0(R) + k$ for ordinary rows) with k uniform on $\ker((\mathbb{Z}/N)^4 \to G)$, independent across rows and of everything else.

Proof. Induction over the transcript. At draw t the algorithm forms $R^{(t)} = a^{(t)} \cdot V + \nu F_{i(t)}$, where the row index $i(t)$ is a function of the history of answers and $a^{(t)}$ is fresh and uniform. On $\langle H_1, H_2 \rangle = G$, $a^{(t)} \cdot V$ is uniform on G, so given ν and the history, $R^{(t)}$ is a translate of a uniform variable by a fixed point, hence uniform on G with a law that does not depend on ν. So the conditional law of the next answer given $(\nu, \text{history})$ does not depend on ν, and by the chain rule neither does the law of the whole transcript. Given $R^{(t)}$ and ν, the tag is uniform on the coset $\{a : a \cdot V = R^{(t)} - \nu F_{i(t)}\}$, and the oracle never sees it. □

Lemma 6.9 (invertibility at all large primes)
On the event of Lemma 6.8, with probability at least $7/8$ over ν, the block $\mathcal{B} = T' - \nu I$ is invertible over $\mathbb{Z}/\ell^v$ for every prime $\ell \mid N$ with $\ell > L_0$.

Proof. $\det(\nu I - T')$ is the characteristic polynomial of T' evaluated at ν: monic of degree f, with integer coefficients independent of ν (Lemma 6.8). For $\ell > \nu_0$ the ν_0 admissible values of ν are distinct modulo ℓ, and a nonzero polynomial of degree f has at most f roots modulo ℓ, so $\Pr_\nu[\ell \mid \det \mathcal{B}] \le f/\nu_0$. A union bound over the at most w such primes gives $wf/\nu_0 \le 1/8$. A determinant that is a unit modulo ℓ is a unit modulo ℓ^v. □

Lemma 6.10 (uniformity with planted rows)
On the events of Lemmas 6.8 and 6.9, for each $\ell > L_0$, the vectors $\mu_1, \ldots, \mu_{12}$ are independent and uniform on Rel_ℓ.

Proof. The tag of ordinary row $f + k$ is $\alpha_0(R_{f+k}) + k_{f+k}$ with k_{f+k} uniform on the kernel and independent of the R's, of ν and of all other tags (Lemma 6.8). The vector x_k, the matrix $\mathcal{B}$ and the planted tags are functions of the R's, of ν and of the other kernel parts, so $\mu_k = c_k + (k_{f+k} \bmod \ell^v)$ with c_k independent of k_{f+k}. Lemma 6.2 gives $\mu_k \in \mathrm{Rel}_\ell$, so $c_k \in \mathrm{Rel}_\ell$. A fixed element of Rel_ℓ plus an independent uniform element of Rel_ℓ is uniform. Conditioning on the invertibility of $\mathcal{B}$, a function of the R's and ν, does not affect k_{f+k}. □

Theorem 1 (main theorem)
Let the oracle (O) be the one provided by Theorem 2(ii), and use the inputs (C1)–(C4) and (W″). Fix $m \ge 2$ and let $X = \lfloor p^{1/m} \rfloor \ge \max(12m, 300)$. For every nonsingular curve $y^2 = x^3 + Ax + B$ over $\mathbb{F}_p$, of any group order and structure, and all $P \in G$, $Q \in \langle P \rangle$, Algorithm H″ finds d with $dP = Q$, always correctly, in expected time $p^{2/m + o(1)}$.

Proof. Correctness. Pohlig–Hellman residues are exact, the lattice residues are correct by Lemma 6.2, and the final check rejects anything else.

Success probability. Let Π be the lower bound for a round of Theorem 5: the generation event, the goodness of u, the cap, and generation of the lattices (Lemma 6.4). The planted draws are uniform on G (Lemma 6.8), so the coverage and Chernoff arguments are unchanged. ν is independent of the generation event and of u, and given these events the transcript is independent of ν. So Lemma 6.9 multiplies the success probability by $7/8$, and Lemmas 6.10, 6.4 and 6.5 give d_ℓ at every $\ell > L_0$. A round succeeds with probability at least $\frac{7}{8}\Pi \geq \frac{7}{16}\sigma_{\mathrm{gen}}$, and the expected number of rounds is $O(\log L)$.

Cost. The relation phase makes at most $2(f+12)/c_m(4)$ oracle calls and one scalar multiplication per draw: $p^{1/m+o(1)}$. Pohlig–Hellman for $\ell \leq L_0 = p^{2/m+o(1)}$ (since $f \leq 3p^{1/m}$ and $w \leq \log_2(2p)$) costs $O(L\sqrt{\ell}) = p^{1/m+o(1)}$ per prime, for at most L primes. For each of the at most L primes $\ell > L_0$, there are 12 solves; each has $v \leq \log_\ell N$ lifting levels and costs $O(f)$ products by $\mathcal{B}^\top$ (at most $m+1$ nonzero entries per row, so $O(mf)$ operations each) plus $O(f^2)$ for Berlekamp–Massey. Since $\ell > 4f^2$, the Krylov step fails with probability $o(1)$ and failures are detected, so the expected number of attempts is $1 + o(1)$. The total is $\tilde{O}(L^3 m f^2) = p^{2/m+o(1)}$. Schoof's algorithm and factoring cost $p^{o(1)}$. □

Theorem 1 contains Theorem 6 and removes both of its restrictions. It also needs no null-space sampler, only a nonsingular solver. Theorem 6 is kept because its proof is independent and its toy checks are more extensive.

Memory. The algorithms of Theorems 1 and 4 use space $p^{1/m+o(1)}$. The oracle runs in time $p^{o(1)}$, hence in space $p^{o(1)}$. The relation matrix has at most $3p^{1/m}+12$ rows with at most $m+1$ nonzero entries each, and the tags have four entries; all entries have $O(L)$ bits (over $\mathbb{F}_{q^k}$ in Theorem 4, $O(kL)$ bits with $k = O(\log f')$). Wiedemann's algorithm keeps a constant number of vectors of length f and a sequence of $O(f)$ scalars, Dixon lifting keeps $O(1)$ vectors per level, and the solves are done one after another. In Theorem 1, baby-step giant-step for the primes $\ell \leq L_0 = p^{2/m+o(1)}$ stores $O(\sqrt{\ell}) = p^{1/m+o(1)}$ points. Schoof's algorithm and factoring run in time, hence space, $p^{o(1)}$. Theorem 5 stores a dense $N_r \times N_r$ matrix and uses space $p^{2/m+o(1)}$.

Checks. These tests exercise the algebra (relation lattices, Lemmas 6.3 and 6.6, Pohlig–Hellman and the Chinese remainder step), not the asymptotic cost, the sparse solver, or (C1). We ran toy index calculi with $m = 2$, a brute-force oracle, random dilations and p between 10^3 and 10^4, on prime-order, cyclic composite, prime-power, and non-cyclic groups (for example $\mathbb{Z}/228 \times \mathbb{Z}/6$), with P of maximal, tiny and random order. Three independently written implementations recovered $d \bmod \mathrm{ord}(P)$ in 70/70, 210/210 and 144/144 instances. As a negative control, linear algebra modulo ℓ on primes with $\ell^2 \mid N$, without Pohlig–Hellman, failed in 13 of 48 instances. A chi-square test of Lemma 6.3 on real oracle data with a structured Λ was consistent with uniformity. For Theorem 1, two independent implementations used elliptic curves and abstract groups with $\ell^2 \mid N$ (ℓ between 41 and 73), brute-force oracles, and three oracle policies: lexicographic, adversarial (orthogonal to a hidden character), and stateful (trying to guess the planted index). Over about 9,800 planted rounds with $\mathcal{B}$ invertible, $d \bmod \ell^2$ was recovered in every round, and no wrong answer was produced. Rounds with $\mathcal{B}$ singular were detected and occurred at rates below the bound f/ν_0 in the regime of Lemma 6.9. With the adversarial oracle, the unplanted matrix was non-liftable in every round. A sparse Wiedemann–Dixon solve of $\mathcal{B}$ modulo ℓ^3 for

f up to 800 used $3f$ products per lifting level. Chi-square tests of Lemma 6.10, and of the independence of the transcript from ν under a stateful oracle, showed no dependence.

7 Growing m: time $p^{o(1)}$

For fixed m the algorithms above take time $p^{2/m+o(1)}$ or $p^{3/m+o(1)}$. Letting m grow with p trades the linear-algebra exponent against the engine cost, which grows linearly in m. In this section, unlike the rest of the paper, m grows with p. This needs a bound on the engine cost that is uniform in m. For each fixed m the oracle is Lean-proved (Theorem 2(ii)), but the Lean theorem does not give uniformity in m: its constants C_k depend on m, and the Lean engine of Rudich and Rousseau [24] on which it rests is stated for each k separately. We use the engine of (E) instead, whose constants are absolute (Proposition 7.2). Everything in this section is therefore conditional on two inputs: (E), the engine of [22], whose Lean statements we have read but not rebuilt (Section 2), and (N), the network of Proposition N in the weak form of Lemma 7.1. Theorem 7 is a theorem about the algorithm under (E) and (N), and is not claimed unconditionally. We write $\lambda = \log_2 L$ throughout.

Machine model. The cost model is word operations on a random-access machine, as in (E), with words long enough to address the memory used. Converting to bit operations costs a factor polynomial in the word length. The engine's word length is at most its running exponent, $O(\log T_{\mathrm{eng}}) = O(L)$ bits (for the m used below, $O(L\lambda^{-\alpha'/2})$), so the factor is $2^{O(\log L)}$ and is absorbed in the $O(\log L)$ that is added to every logarithm of a running time below. The word length is therefore not $O(\log(mL))$ once m grows with L, but the polynomial conversion costs nothing at the scale $2^{o(L)}$.

7.1 The node cost

Proposition N states $\tau \le (mL)^{O(1)}$ and construction time $(mL)^{O(1)}$, with exponents independent of m, through Lemma 3.9, a paper proof. Theorem 7 needs far less than this: it only uses that the node functions are evaluated at a cost whose logarithm is $O(L^{1-\delta})$, because τ enters the engine bound through $\log_2 \tau$ alone. We therefore state the hypothesis (N) in the weak form below, which Lemma 3.9 implies with room to spare, so that the section does not depend on the sharp exponent 7/4.

Lemma 7.1 (node cost, weak form)
There are absolute constants C and $\delta > 0$ such that for all $2 \le m \le L$, all sufficiently large L, every input (p, a, b, R, π) with $|\pi| \le m(b_x + 1)$ and every $g \ge 1$ with $g^4 \le L$, the clustered edge network of Proposition N is output within $2^{L^{1-\delta}}$ word operations, and each of its local functions is evaluated within $2^{L^{1-\delta}}$ word operations; that is, $\log_2 \tau \le L^{1-\delta}$.

Proof. Lemma 3.9 gives $\tau \le C\, m\, L^{7/4}$ bit operations and construction time $(mL)^{O(1)}$ with exponents independent of m. For $m \le L$ this is $2^{O(\log L)} \le 2^{L^{1-\delta}}$ for any $\delta < 1$ and large L (bit operations convert to word operations at no loss). □

The lemma is a consequence of a paper proof, with the per-node counts read from the Lean definitions by hand and independently recomputed; it is not formalized. Only $\log_2 \tau = o(mL\lambda^{-\alpha'})$ enters below, and this weak form suffices for it since $mL\lambda^{-\alpha'} \ge 2L/\lambda$.

7.2 The uniform engine bound

Proposition 7.2 (uniform engine bound)
Assume (E) and (N). Let (ρ, c) be the parameters of the pieces of (E), $\alpha' = \log_2(1/\rho)$, x_0 as in the Engine Theorem, c_1, c_2 the constants of Proposition N, and

$$a_\star = 0.85\, c_1 \big(320(3x_0+6)\big)^{\alpha'}.$$

One deterministic procedure, given m and (p, a, b, R, π) as in Proposition N with $|\pi| \le m(b_x+1)$, constructs the network of Proposition N and outputs its value $|G(R,\pi)|$ within $T_{\mathrm{eng}}(m, L)$ word operations of the model of (E), where for every $\varepsilon > 0$ there is an L_ε, independent of m, such that

$$\log_2 T_{\mathrm{eng}}(m, L) \le (a_\star + \varepsilon)\, mL\, (\log_2 L)^{-\alpha'} \qquad \text{for all } L \ge L_\varepsilon \text{ and } 2 \le m \le L. \qquad (\star)$$

Proof. Step 0 (the constants are absolute). The proof composes the Engine Lemma with the Engine Theorem (b), as Step 2 of the proof of Theorem 2 does. What is new is that no constant may depend on m. The Engine Lemma is stated for "one procedure, the same for all K, g, ζ and all networks", with C_1 absolute. In the Anthropic Lean development the Engine Lemma is the statement `ExplicitCost.GroupingExplicit` and the Engine Theorem (b) is the statement `RecursionSolves`. Their quantifiers are

$$\exists\, C_1\ \forall\, (\text{machine}, K, g, \text{network}, P, B, \tau, \dots) \qquad \text{and} \qquad \forall\, c\ \exists\, C_0\ \forall\, (\rho, x_0, \text{pieces}, \ell, \lambda, B, \text{tables}),$$

so C_1 is chosen before the network and C_0 depends only on c. Both statements are proved, as the theorems `groupingExplicit` and `recursionSolves`. The pieces, and with them ρ, c, x_0, C_0 and C_1, are fixed once and for all, so none of these depends on m, on the network or on L. The Anthropic paper composes the two statements for an arbitrary network with parameters P, B, τ and total U in its display (70) (proof of its Theorem D(a), Step 4); we repeat that composition with the sizes of Proposition N. We have read these Lean statements but not rebuilt the development (Section 2).

Step 1 (level). As in Step 1 of the proof of Theorem 2, let ℓ be the largest index with $K_\ell \le (L/81)^{1/32}$, $K = K_\ell$ and $g = 3K^8$, so $g^4 \le L$, $K > (L/81)^{1/320}$ and $\log_2 K \ge (\log_2 L - 7)/320$. The index ℓ is computed from the closed form of $\log_2 K_\ell$ in time $\mathrm{poly}(L)$. Nothing here depends on m. Proposition N gives the network on $[g]$ with $U \le c_1 mL$, $gP \le c_2 mL$ and $B = 1$, for every $m \ge 2$; by Lemma 7.1 it is output in $2^{L^{1-\delta}}$ word operations and has $\log_2 \tau \le L^{1-\delta}$.

Step 2 (one engine call). The display in Step 2 of the proof of Theorem 2 holds for every m, since no step of its derivation uses that m is fixed. Bounding the maximum by the sum and substituting $K^2 L_{\mathrm{cl}} \le U + gP/K + K^2$ and the bounds of Step 1,

$$\log_2 T_{\mathrm{eng}} \le e_\ell\, c_1 mL + \mathcal{R},$$

$$\mathcal{R} = \frac{(2+e_\ell)\, c_2 mL}{K} + e_\ell K^2 + C_0 K^2 \log_2\big(K(L_{\mathrm{cl}} + B')\big) + g \log_2 K + C_1 \log_2\big(\tau\, g(P+1)\big) + 1,$$

where T_{eng} now also includes the construction of the network and the computation of ℓ.

Step 3 (main term). By the Engine Theorem (a) and Step 1,

$$e_\ell \le 0.85\rho^\ell \le 0.85\Big(\frac{3x_0+6}{\log_2 K}\Big)^{\alpha'} \le 0.85\Big(\frac{320(3x_0+6)}{\log_2 L - 7}\Big)^{\alpha'},$$

so $e_\ell c_1 mL \le a_\star\, mL(\log_2 L - 7)^{-\alpha'} = a_\star\, mL(\log_2 L)^{-\alpha'}(1+o(1))$, with an $o(1)$ that depends on L only.

Step 4 (the rest). Let $2 \le m \le L$ and write $u = mL(\log_2 L)^{-\alpha'}$. Since $L_{\mathrm{cl}} \le (c_1 + c_2)mL + 1$, $B' \le 2c_2 mL + 1 + g^2 \le 2c_2 mL + L^{1/2} + 1$, $K \le L^{1/32}$ and $P \le c_2 mL$, every logarithm in $\mathcal{R}$ other than $\log_2 \tau$ is $O(\log L)$ (this uses $m \le L$). The terms of $\mathcal{R}$, and the construction cost of Step 1, are bounded as follows:

- $(2 + e_\ell)c_2 mL/K \le 3 \cdot 81^{1/320} c_2\, mL^{1-1/320}$, which is at most $3.1\, c_2 (\log_2 L)^{\alpha'} L^{-1/320} \cdot u$;
- $e_\ell K^2 + C_0 K^2 \log_2(K(L_{\mathrm{cl}} + B')) = O(C_0 L^{1/16} \log L)$, $g \log_2 K \le L^{1/4} \log_2 L$ and $C_1 \log_2(g(P + 1)) + 1 = O(C_1 \log L)$, each at most $L^{1/4}(\log L)^{O(1)}$, while $u \ge 2L(\log_2 L)^{-1}$;
- $C_1 \log_2 \tau$ and the logarithm of the construction time are at most $(C_1 + 1)L^{1-\delta}$ by Lemma 7.1, which is $o(u)$ since $u \ge 2L/\lambda$;
- computing ℓ costs $\mathrm{poly}(L)$, that is $O(\log L)$ in the logarithm.

Each ratio to u tends to 0 as $L \to \infty$ and depends on L only (through the absolute constants c_2, C_0, C_1 and α'), not on m. With Step 3 this gives $(\star)$. The bound is in the word operations of (E). The explicit constant $a_\star$ is meaningful only in that model: the Lean predicate of [22] counts the steps of a machine whose step is an arbitrary function of its state, so in the Lean model the exponent holds up to a multiplicative machine constant, which we do not track. Bit operations add $O(\log L)$ to the logarithm (Machine model above), which is also $o(u)$. □

For fixed m, Proposition 7.2 is Step 3 of the proof of Theorem 2; only the uniformity in m is new. The proposition also holds without the restriction $m \le L$ if $\log m = O(L)$, since then all logarithms in $\mathcal{R}$ are still $O(L)$ and are divided by mL. With the constants of the network of Section 3.1 ($c_1 = 1410$, $c_2 = 2464$, $g \le 3L^{1/4}$) the same derivation gives $a_\star \approx 4.1 \cdot 10^3$ for $\rho = 11/12$ and $x_0 = 18$, but that network's value is checked, not proved.

7.3 Theorem 7

Theorem 7

Conditional on (E) and (N). Assume (E), and (N): Proposition N (the network's correctness and its size bounds, uniform in m) together with Lemma 7.1. Use also the inputs (C1), (C2) and (C4). Let α' be the rate exponent of the pieces of (E) and $m = m^*(L) = \lfloor \lambda^{\alpha'/2} \rfloor$. There is L_0, depending only on the absolute constants of (E) and (N), with $\log_2 L_0 \ge 2^{2/\alpha'}$ (so that $m^* \ge 2$; this is $\log_2 L_0 \ge 6.25 \cdot 10^4$ for $\alpha' = \log_2(12/11)$), such that for all $L \ge L_0$, every nonsingular curve $y^2 = x^3 + Ax + B$ over $\mathbb{F}_p$, of any group order and structure, and all P and $Q \in \langle P \rangle$, the algorithm of Theorem 5 with $m = m^*$, with the oracle (O) realized by the search from counts of Theorem 2 on the engine of (E), and using the Lenstra–Pomerance algorithm to factor N, finds d with $dP = Q$, always correctly, in expected time (bit operations on a random-access machine)

$$2^{O(L(\log L)^{-\alpha'/2})} = p^{O((\log L)^{-\alpha'/2})} = p^{o(1)}.$$

The implied constant is absolute but not explicit: it contains C_0, C_1 of (E) and, in the Lean model, a machine constant. The exponent is meaningful as stated only in the word-operation model of (E); in the formal model it holds up to that machine constant. In the word-operation model of (E) the proof gives the exponent $\max(1 + a_\star, 3)\, L\lambda^{-\alpha'/2}(1 + o(1))$.

This is time $p^{o(1)} = 2^{o(L)}$, subexponential in the input length L, but it is not of the form $L_p[\alpha, c] = \exp(c(\log p)^{\alpha}(\log\log p)^{1-\alpha})$ with $\alpha < 1$ that "subexponential" usually means for index calculus (Diem's bounds over extension fields are of that form). With the fully proved $\alpha' = \log_2(12/11)$ the bound is $2^{O(L/(\log L)^{0.063})}$.

Proof. Oracle. Step 4 of the proof of Theorem 2, stopped after the x-string with sign bits, finds the labels of a decomposition, or reports that there is none, with $m(b_x+1)+1 = L+O(m)$ counts. Each count is the value of the network of Proposition N for a prefix of length at most $m(b_x+1)$, which is $|G(R,\pi)|$ for every m (Proposition 3.7(i), a Lean theorem), and Proposition 7.2 computes it. So the oracle costs $T_{\mathcal{D}} = (L + O(m))\, T_{\mathrm{eng}}(m, L)$.

Side conditions. Theorem 5 needs $X = \lfloor p^{1/m} \rfloor \geq \max(12m, 300)$ and, through (C1), $X \leq \sqrt{p}$, $X < p/3$ and $p \geq 100$. We prove them for every $L \geq 2^{64}$ with $m^* \geq 2$ (which the choice of L_0 gives; for $2^{64} \leq L < 2^{6.25\cdot 10^4}$ one has $m^* = 1$ at $\alpha' = \log_2(12/11)$ and Theorem 5 does not apply). Since $L = \lceil \log_2 p \rceil$ we have $p > 2^{L-1}$, hence $X \geq p^{1/m} - 1 > 2^y - 1$ with $y = (L-1)/m$. Now $m^* \leq \lambda^{\alpha'/2} \leq \lambda$ because $\alpha' < 1$ and $\lambda \geq 64$, so $y \geq (L-1)/\lambda \geq L/(2\lambda) = 2^{\lambda-1}/\lambda \geq 2^{57}$, as $2^{\lambda-1}/\lambda$ is increasing for $\lambda \geq 2$. Also $m^* \leq \lambda^{0.0628}$ gives $12m^* + 1 < 2^y$ trivially (the left side is below 12λ, the right side above $2^{2^{57}}$), and $301 < 2^y$. Hence $X \geq 2^y - 1 \geq \max(12m^*, 300)$. The conditions $X \leq \sqrt{p}$ and $X < p/3$ hold for $m \geq 2$ and $p \geq 100$. (Note that $\alpha'/2 = \frac{1}{2}\log_2(12/11) \approx 0.0628$ exceeds $1/16$; the argument uses only $m^* \leq \lambda$.) A numerical spot check for $L = 2^k$, $8 \leq k < 400$, agrees, but the proof does not rely on it.

The constants of (C1) do not depend on m, except through $c_m(4)$. Lemma F, Markov's inequality, the Chernoff bound and Lemma 6.4 are m-free, and

$$\log_2(1/c_m(4)) = \log_2(8 \cdot 16^m \cdot 4 \cdot c'_m) = m \log_2 m + O(m),$$

since $\log_2 \Theta_m = \log_2((2m)!\, 2^m/m!) = m \log_2 m + O(m)$ and $\log_2(3 \cdot 4^m D'_{2m}) = O(m)$. This is the same order as the $\log_2 m!$ of the decomposition heuristic.

Total. By the proof of Theorem 5, the expected number of rounds is $O(\log L)$, and a round costs $(2N_r/c_m(4))(T_{\mathcal{D}} + \mathrm{poly}(L))$ plus dense linear algebra, with $N_r \leq 3p^{1/m}$. Schoof's algorithm costs $\mathrm{poly}(L)$, and the Lenstra–Pomerance algorithm factors N in expected time $L_N[1/2, 1+o(1)] = 2^{O(\sqrt{L \log L})}$ [31, main theorem]. Hence

$$\log_2 \mathbb{E}[T] \leq \max\left\{ \frac{L}{m} + \log_2 \frac{1}{c_m(4)} + \log_2 T_{\mathrm{eng}}(m, L),\ \frac{3L}{m},\ O(\sqrt{L \log L}) \right\} + O(\log L).$$

At $m = m^*$: $L/m^* = L(\log_2 L)^{-\alpha'/2}(1+O(1/m^*))$; by $(\star)$, for every $\varepsilon > 0$ and large L, $\log_2 T_{\mathrm{eng}} \leq (a_\star + \varepsilon)\, m^* L (\log_2 L)^{-\alpha'} \leq (a_\star + \varepsilon) L (\log_2 L)^{-\alpha'/2}$; the coverage term is $O((\log L)^{\alpha'/2} \log\log L)$; and $\sqrt{L \log L}$ is $o(L(\log L)^{-\alpha'/2})$, because the ratio of the two is $(\log L)^{(1+\alpha')/2}/\sqrt{L} \to 0$. So

$$\log_2 \mathbb{E}[T] \leq \max(1 + a_\star, 3)\, L (\log_2 L)^{-\alpha'/2}(1 + o(1)).$$

Correctness is that of Theorem 5: every output is checked. □

Remarks. (1) The rate is the one the decomposition heuristic predicts with the same engine bound. The coverage loss $2^{m \log m + O(m)}$ plays the role of $m!$. Theorem 7 uses the dense Theorem 5; the sparse path of Theorem 6 would only change the constant 3. (2) The choice of m^* has slack. The engine cost stays $p^{o(1)}$ as long as $m = o(\lambda^{\alpha'})$. With the exact recursion for K_ℓ and e_ℓ (Engine Theorem, $e_0 = \frac{1}{2}$, $e_{\ell+1} = (\rho + 3\zeta_\ell)e_\ell + 3/K_\ell$), $\rho = 11/12$, $x_0 = 18$, the ratio $e_\ell c_1 \lambda^{\alpha'}$ of the main

engine term to $mL\lambda^{-\alpha'}$ lies, for $10^3 \le \log_2 L \le 10^{300}$, in $[1.6 \cdot 10^3, 2.5 \cdot 10^3]$ for the constants $c_1 = 1410$ of the network of Section 3.1 and in $[6 \cdot 10^5, 9.5 \cdot 10^5]$ for the constants $c_1 = 549504$ of the proved edge network; it does not depend on m. These ranges are loose on purpose: the ratio oscillates with the level ℓ (it is not monotone in L), and the quoted intervals are enclosures of a numerical scan with step 0.01 in $\log_{10} \log_2 L$, not asserted bounds. The sharp bound is $a_\star$ of Proposition 7.2, $4.1 \cdot 10^3$ and $1.6 \cdot 10^6$ respectively. The ratio of the total exponent of Theorem 7 to $L\lambda^{-\alpha'/2}$ at $m = m^*$ lies in $[1.5 \cdot 10^3, 2.5 \cdot 10^3]$ (resp. $[5 \cdot 10^5, 9.5 \cdot 10^5]$) for $10^5 \le \log_2 L \le 10^{300}$, below $\max(1 + a_\star, 3)$. (3) Theorem 7 takes the factoring step from Lenstra–Pomerance. The proved bound of Rudich and Rousseau, $C_k 2^{n/k}$ for each fixed k, does not give a rate uniform in m; for fixed m either reference suffices. (4) Two different crossovers must not be confused. Say the exponent of the algorithm beats generic $\sqrt{p}$ when $L/m + e_\ell c_1 mL < L/2$ (the coverage and factoring terms are lower order). With $\alpha' = \log_2(12/11)$, the only value fully proved in the main text of [22], $m^*(L) \ge 2$ requires $\log_2 L \ge 2^{2/\alpha'} \approx 6.25 \cdot 10^4$. Scanning $\log_2 L$ with the exact e_ℓ recursion (recall $L = \lceil \log_2 p \rceil$):

	$m = m^*$ of Theorem 7	best integer m
network of Section 3.1 ($c_1 = 1410$)	$10^{58.4}$	about $10^{36.4}$
edge network ($c_1 = 549504$)	$10^{99.6}$	about 10^{57}

The bound of Theorem 7 itself, with $m = m^*$, therefore beats $\sqrt{p}$ only for $\log_2 L \gtrsim 10^{58}$ (network of Section 3.1) or 10^{99} (edge network), where L is itself astronomically large. The smaller figure, about $10^{36.4}$ for the network of Section 3.1, is the crossover of the optimized $m \approx (e_\ell c_1)^{-1/2}$, which is not the m^* of the theorem. It is $2.5 \cdot 10^{36}$ with the exact recursion for e_ℓ; Section 10 quotes $6.3 \cdot 10^{35}$ for the fixed-m threshold with the same pieces, because there e_ℓ is taken at its lower bound $\frac{1}{2}\rho^\ell$ and the exact e_ℓ is larger by a factor up to 1.29. The choice m^* is made for the clean $p^{o(1)}$ statement, not for the crossover. All these figures are in the word-operation model of (E); in the formal model they hold up to a machine constant. With $\alpha' = 1/2$ (Appendix B of [22]) the corresponding thresholds are smaller, near $\log_2 L \approx 10^{11.7}$ for the optimized m and the network of Section 3.1 ($10^{16.8}$ for the proved edge network; same scan, $\rho = 2^{-1/2}$). None of this is the fixed-m threshold of Section 10; the quantities are different and only happen to be of similar size.

8 Formal verification

Parts of the argument are formalized in Lean 4 (toolchain v4.35.0, with Mathlib and the Lean development of Rudich and Rousseau [24] as dependencies). "Proved in Lean" below means: the module builds, contains no `sorry`, `admit` or new `axiom`, and `#print axioms` of the named theorems reports only `propext`, `Classical.choice` and `Quot.sound`. The one deliberate `sorry` is the target statement `ECDLP.subexponentialRate` in module `L35_Challenge`; it is never used as a lemma, and its type is proved separately below. All modules are in the namespace `ECDLP`. The builds were run in our working environment (the last full rebuild compiled 9608 jobs); Table 3 summarizes the status.

8.1 The main Lean theorem

The audit statement `subexponentialRate m` (module `L35_Challenge`, which imports only Mathlib) says: there is one Turing machine (Mathlib's `TM2ComputableInTime`, with finite alphabets)

that, given (p, a, b, x_R, y_R, π), outputs $\mathrm{lexLeast}_m$, the lexicographically least x-string with sign bits of a generic decomposition $R = P_1 + \cdots + P_m$ with $x(P_i) < B$, $B = \min\{B : p \le B^m\} = B_x$, beginning with π, or reports that there is none; and it does so within $C_k 2^{\ell/k}$ steps on inputs of length ℓ, for every $k \ge 1$. Membership in the set of strings includes primality of p and nonsingularity of the curve. The theorem

```
theorem ECDLP.L104R.subexponentialRate_all :
  ∀ m : ℕ, type_of% (ECDLP.subexponentialRate m)
```

(module `L104_Final`) proves this for every m, with no hypotheses. Unfolded, its statement is

```
∀ m : ℕ, ∃ M : TM2ComputableInTime enc encOut (lexLeast m),
  (∀ k, Finite (M.tm.Γ k)) ∧
  ∀ k : ℕ, 1 ≤ k → ∃ C : ℕ, ∀ ℓ, M.time ℓ ≤ C * 2 ^ (ℓ / k)
```

The axiom check reports only Lean's three standard axioms:

```
#print axioms ECDLP.L104R.subexponentialRate_all
-- [propext, Classical.choice, Quot.sound]
```

The cases $m = 0$ and $m = 1$ are degenerate and are proved separately inside the proof, in modules `L77` and `L102` (for $m = 1$ the only candidate point is R). For $m \ge 2$ the machine is prefix search driven by counts of the edge network (Proposition 3.7), evaluated by the engine.

The engine is not assumed. Its correctness and running time inside the pipeline come from the Lean proofs of Rudich and Rousseau, which our development imports and which we generalized to a symbolic depth budget. So the search over the x-string with sign bits, for every m, is machine-checked and conditional on nothing beyond Lean and Mathlib. In particular this part of Theorem 2(ii), and the oracle (O) in the label form used in Sections 5 and 6, do not depend on (E). An independent audit found the statement faithful: for $m = 2$ and $p = 13$, the values of $\mathrm{lexLeast}_2$ on three instances (two decompositions, one "none") were decided in the Lean kernel and match a brute-force search; the input encoding is binary (Mathlib's `encodeNat`), so the input length is $\Theta(\log p)$; and the time bound is worst case over all inputs of each length. We note the points where the Lean statement and the paper differ in form:

- The output is the x-string with sign bits, as in Theorem 2(ii), not the tuple (x_i, y_i). The two determine each other (Definition 3.1), and the lexicographically least string selects the same tuple as the y-digit extension (the remark after Theorem 2); without (E) the y_i follow by square roots (Remark 4.1).
- Each x_i takes `Nat.size` $B_x = b_x$ bits, the convention of Section 3.
- Time is counted in machine steps and in the total input length ℓ, which includes π. For inputs with $a, b, x_R, y_R < p$, $\ell = O(L + |\pi|)$, so the bound is $p^{o(1)}$ for $|\pi| = O(L)$.
- The inputs are natural numbers read modulo p, and invalid inputs (non-prime p, singular curve) give the answer "none".

8.2 The counting theorem

The counting form of Theorem 2 is also a Lean theorem. The audit statement `countingRate m` (module `L109_CountChallenge`, which imports only Mathlib) says: there is one Turing machine that, given (p, a, b, x_R, y_R, π), outputs in binary the number of generic decompositions $R = P_1 + \cdots + P_m$ with $x(P_i) < \lceil p^{1/m} \rceil$ whose x-string with sign bits begins with π, within $C_k 2^{\ell/k}$

steps for every $k \geq 1$. Since a point is determined by its x-coordinate and sign bit, this is the number of such decompositions. The theorem

```
theorem ECDLP.L132.countingRate_all :
  ∀ m : ℕ, type_of% (ECDLP109.countingRate m)
```

(module `L132_Final`) proves it for every m, with no hypotheses, and its axiom check reports only Lean's three standard axioms. The machine runs the engine of Rudich and Rousseau on the edge network, computes the count modulo small integers and recovers it exactly from the bound $Z < 2^{\ell}$ (`count_le`, `zc_lt_len`); short inputs use a lookup table. An independent audit checked that the statement is the one in the audit file (an `rfl` example ties the two), that the proof does not use the `sorry` of the audit file, and that the counted set is the set $G(R, \pi)$ of Definition 3.1 restricted to x-and-sign prefixes. So Theorem 2(i), for prefixes over the x-string with sign bits, no longer depends on (E).

Inside both machines the network value = count step is a theorem (`edgeCount_real_eq_Zc`, module `L93`), and so are the network's size bounds (`wt_htot`, module `L94`), the frame invariant of every engine call (`frameIn_fr`, module `L96`), and the value and cost of the engine on this network family (`engFullHF_val`, `engFullHF_cost`, module `L104_Engine`). Earlier versions of this pipeline carried a hypothesis (`OffFrame`) and a bound $m \leq 2^{19000}$; both are gone.

The network in Lean is the edge network of Section 3.3. Its size constants ($U \leq 183168\, mL'$, $gP \leq 185244\, mL'$, with $L' \leq 3L$) are about 400 times cruder for U and about 230 times for gP than those of the hand-optimized network of Sections 3.1–3.2, which is not formalized.

8.3 What is proved

Encoding. For a short Weierstrass curve over $\mathbb{Z}/p$ (Mathlib's `WeierstrassCurve.Affine`), the four constraints of one chord addition hold if and only if the x-coordinates differ and the witnesses are the inverse, the slope and the sum, and the witnesses are unique (`stepC_iff`). The chained system is satisfiable if and only if the tuple is generic with sum R (`chain_iff`, `chain_unique`). Congruences are equivalent to integer identities with a unique quotient in a window (`mul_modEq_iff`, `zero_iff_window`). Finally, the number of integer solutions of the whole system equals the number of generic decompositions (`card_int_eq_card_generic`).

Search from counts. For any count oracle satisfying $c(\pi) = c(\pi 0) + c(\pi 1)$ with 0/1 values on full strings, prefix descent returns the lexicographically least string of the set with n queries (`search_spec`, `search_abstract`). This is Step 4 of Theorem 2.

Gadgets and contraction. A Schönhage–Strassen gadget (square transforms, under an explicit size condition) accepts if and only if the combined integer identity holds (`Par.accept_iff`, `NPar.good_iff`). The value of a forced network (each node has a free variable and receives messages computed by its parents) equals the number of free assignments passing all local checks (`forced_value`; with child-specific messages, `pforced_value`). As networks in the sense of Definition 2.2, a gadget's value is the number of digit assignments satisfying its identity (`gadget_value`, `system_value`).

Network value equals decomposition count. For a Lean network with one gadget per identity and child-specific messages (the *edge network*), the value for prefix π equals the number of decomposition strings extending π, and prefix search over its values returns lexLeast (`edge_`

value, main_count_edge, net_eq_ncard_edge, search_computes_lexLeast_exists_edge; also edgeCount_real_eq_Zc for the reduced instances used by the driver). Earlier variants (a packed network and a broadcast network) are also proved correct, but they violate the engine's node-size condition, which we also proved. The edge network is the network on which Theorem 2 is based (Proposition 3.7); the hand-optimized network of Sections 3.1–3.2 is a sibling, not the same object, and is not formalized. For the edge network, the engine's size and cluster conditions are proved in Lean with larger constants than in Section 3.2: total shared information at most $183168\, mL'$ and $gP \le 185244\, mL'$ for $g^4 \le L'$, where L' is the slot width (exists_cluster, ecdlp_engine_conditions), and the total node weight is at most the bound U_c of module L65 (wt_htot, module L94).

Coverage chains. The probabilistic skeleton of Theorems 3 and 4, and of the average-over-curves variant mentioned in Remark (4) after Theorem 4, is proved with the algebraic lemmas as named hypotheses. Average over curves: Cauchy–Schwarz coverage from energy (sq_card_le_sums_mul_energy), the count of non-generic tuples, the averaged energy bound via Mathlib's Schwartz–Zippel lemma, the rank-free Las Vegas step (las_vegas_success), and the link to Mathlib points and to the oracle's strings (coverage_oracle). Its single mathematical hypothesis is SemaevNondeg (Semaev's vanishing property together with the average-over-curves nondegeneracy of Remark (4) after Theorem 4 for a polynomial family of bounded degree). Theorems 3 and 4: the isomorphism $E \to E_u$ (dilEquiv), transfer of the discrete logarithm (dlp_transfer), and coverage for all u outside a bad set of the right size (corollary4_coverage). Its two hypotheses are DilationNondeg (Lemma N′ as a root count) and FactorBaseVar (Lemma F). For $A = 0$, DilationNondeg must be instantiated with $\mathcal{I} = [1, X]$, $X < p/3$, as in Lemma N′. The formalization of the energy count found an error in an earlier draft: the trivial tuples have 4^m sign choices, not 2^m.

The index-calculus reduction. The combinatorial and probabilistic core of Theorem 1 is proved in Lean with the analytic inputs as named hypotheses. The relation lattice and the recovery of the logarithm from it (relation_lattice, recover_from_span), the Chinese remainder step (crt_correct), the planted diagonal (planted_kernel, block_isUnit) and the final step (runFinal_sound, runFinal_complete) are theorems. The relation rows of the collection rule are a theorem for an adaptive, stateful oracle (algSpec_row_rel), and so is the Chernoff bound on the number of accepted draws (cap_bound). Lemma gen is proved without any hypothesis on the group: every finite abelian group is generated by at most $\log_2 |G|$ elements, and $s \ge \log_2 |A| + 1$ uniform elements generate A with probability at least $5/16$ (gen_log_explicit). So the reduction no longer assumes that $E(\mathbb{F}_p)$ is generated by two points. For the two-point form we also prove $n \le (1 + \log_2 n)\, \varphi(n)$ (totient_ge), which replaces the Mertens-type bound of the paper proof at the cost of a factor $O(\log p)$. The remaining hypotheses of the reduction are the coverage bound (Theorem 3, which needs the Weil bound), the group order and its factorization (Schoof's algorithm and factoring), the cost of the sparse linear algebra, and the loss from sampling points only nearly uniformly.

8.4 What is not in Lean

The following are paper proofs only: the deterministic extension of the search to the y-digits in Theorem 2(ii) (Lemma 3.8, whose x-and-sign case is Proposition 3.7); Lemma 3.9 (the cost τ as a paper statement about Definition 2.2; the Lean pipeline proves the engine's cost directly);

Semaev's theorem (Mathlib has no summation polynomials or reduction theory of elliptic curves); Lemmas 5.2, 5.4, 5.5, N′ and F; the $A = 0$ repair; the size bounds of Section 3.2 for the hand-optimized network (they are proved in Lean, with cruder constants, for the edge network); the restart and Chernoff analysis and the Chinese remainder step of Theorem 4; and Theorems 1, 5 and 6. These were checked by independent adversarial review and by the numerical checks reported in each section. The appendix results (the networks X and V8x, and genus 2) and the other networks of Section 3 are also not in Lean. The Anthropic engine statements (E) are Lean-stated in the authors' development (`recursionSolves`); we did not rebuild it, and our Lean pipeline does not use it.

8.5 Reproducibility

The repository accompanying this paper, `https://github.com/kaarelkaarelson/fast-ecdlp`, has two parts. `lean/` is the Lean project: `Challenge.lean` states `subexponentialRate` and imports only Mathlib; `Solution.lean` states the same declarations and proves them; `lean/comparator.json` names the two modules, the theorem and the permitted axioms for a mechanical comparison of the two files; and `Verify.lean` prints the axioms of `subexponentialRate_all` and of the theorems of Table 3 that are in the repository. The modules added for version 2 (the counting theorem `countingRate_all`, the index-calculus reduction, the Chernoff and generation modules, and with them `gen_log_explicit`) are in `v2/lean/`, a Lake project that builds on `lean/`; its `Verify.lean` prints their axioms. `scripts/` holds the numerical checks: each script recomputes a claim, can fail, and ends with a verdict line, and the recorded output sits next to it. Table 4 lists, for each claim, how to check it. Commands are run from the folder of the script; paths are relative to the repository root.

The repository does not contain the scripts for the Poisson fit of Section 9, the plain-box comparison ($u = 1$) of Section 10, or the appendix results (the networks X and V8x, and genus 2), or the computations behind Theorem 7, its Remarks, the block clustering and the c_1 bound; these claims cannot be checked from it.

9 Related work

Generic algorithms and lower bounds. Pollard's rho method [3] solves the discrete logarithm problem in a group of prime order r with heuristically $O(\sqrt{r})$ operations and little memory; baby-step giant-step [6] achieves $O(\sqrt{r})$ rigorously, with $\sqrt{r}$ memory. Nechaev [7] and Shoup [8] proved that $\Omega(\sqrt{r})$ operations are necessary for generic algorithms. Our algorithms are not generic: they read the bits of coordinates.

Precomputation. In a different model, an algorithm may first compute a large table that depends only on the group, and then use it for every instance. Bernstein and Lange [14, 15] showed that after such a precomputation each discrete logarithm in a group of order r costs about $r^{1/3}$ operations; for elliptic curves over $\mathbb{F}_p$ of prime order this is about $p^{1/3}$ per instance. Our algorithms are uniform and use no precomputation; their costs are not comparable with these online costs.

Special curves. Some classes of curves over prime fields have long been known to be weak. For anomalous curves ($|E(\mathbb{F}_p)| = p$, a prime order) Semaev [9], Satoh and Araki [10] and Smart [11]

Claim	In Lean	Main theorem
The decomposition algorithm (Theorem 2)		
Search in time $p^{o(1)}$, every m, engine included	Proved	`subexponentialRate_all`
Counting in time $p^{o(1)}$, every m, engine included	Proved	`countingRate_all`
Group law $\Leftrightarrow$ integer constraint system	Proved	`chain_iff`
Search from counts (prefix self-reduction)	Proved	`search_spec`
Gadget acceptance $\Leftrightarrow$ integer identity	Proved	`Par.accept_iff`
Contraction of forced networks	Proved	`forced_value`
Network value = decomposition count	Proved	`edgeCount_real_eq_Zc`
Engine size and cluster conditions	Proved[a]	`ecdlp_engine_conditions`
Index calculus (Theorem 1)		
Relation lattice, recovery, planted diagonal	Proved[c]	`runFinal_sound`
Relation rows, adaptive oracle; Chernoff cap	Proved	`algSpec_row_rel`, `cap_bound`
Random elements generate any finite abelian group	Proved	`gen_log_explicit`
Crossover thresholds (Section 10)		
Schedule arithmetic: block clustering ($e_0 = \frac{1}{2}$) and base case of Lemma 10.1	Proved[d]	`G1_arith_paper`, `E0Q_arith_free`, `E0Q_arith_paper`
Coverage		
Average over curves	Proved[b]	`coverage_oracle`
Random isomorphic model (Theorems 3 and 4)	Proved[b]	`corollary4_coverage`
Paper proofs only		
y-digit search (Lemma 3.8); cost τ (Lemma 3.9)	No	–
Lemmas N′, F and 5.2; Semaev's theorem; the case $A = 0$	No	–
Discrete-logarithm theorems (Theorems 1, 4–6); coverage (Theorem 3)	No	–
Theorem 7 (growing m) and Proposition 7.2	No	–

Table 3: Formal verification status. [a]For the network used in Lean, with constants about 400 times (for U) and 230 times (for gP) those of Section 3.2. [b]With the algebraic lemmas as named hypotheses (Semaev's vanishing property with the nondegeneracy of Remark (4) after Theorem 4 or Lemma N′, and the factor-base count of Lemma F), which the paper proves on paper. [c]With the coverage bound, the group order and its factorization, and the cost of the sparse linear algebra as named hypotheses. [d]Arithmetic only: every level condition and the final inequality of the recursion, checked by a Boolean checker over $\mathbb{Q}$ with a soundness proof and kernel evaluation; Theorem A itself, assumption (R1) and Lemma 10.1 are not formalized. The version 2 modules and theorems are listed in `v2/README.md`.

Claim	How it is established	Where in the repository
Theorem 2(ii): decomposition search in time $p^{o(1)}$, every m, engine included	Lean	`lean/`
Worked example ($p = 13$, $m = 2$, Section 3)	Lean (kernel-decided)	`lean/`
Network value equals the decomposition count	Lean; numerical check	`scripts/network/`
Size bounds (Section 3)	Paper proof; numerical check	`scripts/size_bounds/`
Full decomposition problem (Remark 4.2)	Numerical check	`scripts/full_semaev/`
Coverage: Theorem S, Lemmas 5.2, N′ and F, the case $A = 0$	Paper proof; numerical check	`scripts/coverage/`
Theorems 4–6 (discrete logarithms)	Paper proof; toy runs	`scripts/index_calculus/`
Theorem 1 (planted diagonal)	Paper proof; toy runs	`scripts/index_calculus/theorem1_planted/`
Crossover thresholds (Section 10)	Computation	`scripts/crossover/`
Schedules of Table 5 (block clustering, base case)	Computation; Lean (arithmetic)	`v2/scripts/schedules/`, `v2/lean/`

Table 4: How each claim is established. "Numerical check" and "toy runs" test instances; they are not proofs. Commands and expected outputs are listed in `docs/REPRODUCE.md`.

solve ECDLP in polynomial time. For curves of small embedding degree k (the order r divides $p^k - 1$), the pairing reductions of Menezes, Okamoto and Vanstone [12] and of Frey and Rück [13] transfer the problem to $\mathbb{F}_{p^k}^*$, where index calculus runs in subexponential time for small k; prime-order curves with small embedding degree exist [39]. These attacks use special structure of the group order. Our algorithms apply to every curve, and on curves outside these classes no algorithm was known to run in time $p^{1/2-\varepsilon}$; Pollard's rho method remains the best known for them [4, 5].

Index calculus on $E(\mathbb{F}_p)$ before summation polynomials. Miller [1] argued, when proposing elliptic-curve cryptography, that index calculus is unlikely to work on elliptic curves, because points do not have a notion of small height that behaves well under reduction. Silverman's xedni calculus [40] lifts points to curves over $\mathbb{Q}$ and seeks a dependency there; Jacobson, Koblitz, Silverman, Stein and Teske [41] showed that it fails, because the lifted points are almost always independent. Our method lifts nothing: decompositions are found over $\mathbb{F}_p$.

Summation polynomials. Semaev [17] introduced summation polynomials and the index calculus with the factor base of points with small x-coordinate, and observed that fast bounded decomposition would bring the cost below the square root. Our algorithm never computes a summation polynomial. The network checks the group law directly, with the y-coordinates as witnesses. Summation polynomials enter only the analysis, through Theorem S, in the proofs of Lemma N′ and Proposition 5.6.

Extension fields. Gaudry [18] gave an index calculus for abelian varieties of small dimension and applied it to Weil restrictions of elliptic curves over $\mathbb{F}_{q^n}$, with heuristic running time $\tilde{O}(q^{4/3})$ for $n = 3$ and $\tilde{O}(q^{3/2})$ for $n = 4$. Diem proved rigorously, for fixed n, a bound $\tilde{O}(q^{2-2/n})$ for discrete logarithms in class groups of curves over $\mathbb{F}_q$ [19], and that the ECDLP over $\mathbb{F}_{q^n}$ can be solved in subexponential expected time $(q_i^{n_i})^{o(1)}$ when $n_i \to \infty$ with $n_i/\log q_i \to 0$ [20], then with $n_i/(\log q_i)^2 \to 0$, and in time $e^{O(\log(q^n)^{3/4})}$ for $a(\log q)^{1/3} \le n \le b \log q$ [21]. These results use the algebraic structure of the decomposition problem over $\mathbb{F}_{q^n}$ (a polynomial system over $\mathbb{F}_q$ after Weil descent). Over prime fields there is no such structure: the factor base is an interval, and the only handle we found is the second moment over isomorphic models. The survey of Galbraith and Gaudry [4] reviews the many Gröbner-basis approaches and the heuristics about their complexity, including the estimate that a proportion of about $1/n!$ of points decomposes. Faugère, Perret, Petit and Renault [42] studied the polynomial systems that Weil descent gives over binary fields, and Petit and Quisquater [43] derived a subexponential bound for binary fields under a heuristic on the first fall degree. Huang, Kosters and Yeo [44] gave computational evidence against that heuristic. Faugère, Gaudry, Huot and Renault [45] used the symmetries of summation polynomials to speed up the Gröbner-basis step, and Joux and Vitse [46, 47] combined Weil descent with decompositions over covering curves and analyzed small-degree extension fields. All of these rely on the subfield structure of $\mathbb{F}_{q^n}$.

Prime fields. Semaev [48] replaced the summation polynomial by a system of cubic equations in auxiliary variables and, under a first fall degree assumption on Gröbner-basis computations, derived the heuristic bound $2^{c\sqrt{n \ln n}}$, $c \approx 1.69$, for curves over $\mathbb{F}_{2^n}$; for prime q he states that "it is not quite clear how the system ... may be resolved" [48, §4.2] and makes no complexity claim. Galbraith and Gaudry [4] regard its subexponential conjecture as not yet demonstrated,

and the first fall degree assumption is the heuristic questioned by Huang, Kosters and Yeo [44]. Petit, Kosters and Messeng [49] proposed factor bases over prime fields defined by rational maps, with algebraic rather than interval structure. Amadori, Pintore and Sala [25] proposed a variant of Semaev's method for prime fields that builds the factor base during the relation search and needs only one Gröbner-basis computation; they give experiments and a partial complexity analysis, and do not claim a bound below $p^{1/2}$ because the cost of the polynomial system is not estimated. Yokoyama, Yasuda, Takahashi and Kogure [5] bounded the complexity of Semaev's naive index calculus, estimated the success probability with the $m!$ symmetry factor, and argued, under statistical assumptions on Gröbner-basis computation, that the naive method cannot beat generic algorithms; our decomposition step does not use Gröbner bases. Mahalanobis and Mallick [50] gave a Las Vegas reduction of ECDLP, based on partitions of integers modulo p, to a linear-algebra problem, succeeding with probability about $1 - 1/e$; the whole algorithm succeeds with probability about $0.6(\log p)^2/p$ per pass. As far as we know, no previous work proves a lower bound on the proportion of decomposable points for an interval factor base over $\mathbb{F}_p$, even for $m = 2$. We have not checked this against the literature on additive energy and sumsets of x-coordinate sets of elliptic curves, which may contain related bounds. Our bound averages over isomorphic models and does not settle the question for a fixed model (Section 10). Exact counts on random curves with $p \le 2^{26}$ and $m \le 6$ fit a Poisson model for decompositions into distinct points to within about 1%. For $m \ge 4$ the $1/m!$ rate is not reached at these sizes, because of the many tuples with repeated or opposite points.

The October 2026 papers. The Anthropic [22] and OpenAI [23] papers count satisfying assignments of k-CNF formulas in time $2^{o(n)}$; we use only the network-evaluation engine of the first. Rudich and Rousseau [24] factor integers in time $N^{o(1)}$ by counting divisors with a given prefix, with a network of Schönhage–Strassen multiplication checks for the single product $de = N$. Our network follows their design. It differs in the m-fold composition of the group law, the reduction modulo p by explicit quotients, the range checks, and the size analysis that keeps U and gP at $O(mL)$. None of the three papers discusses discrete logarithms or elliptic curves, beyond mentions of Lenstra's elliptic-curve factoring method and of Shor's algorithm in the factoring paper. Our search for other applications of the engine to discrete logarithms covered the five days after the papers appeared and found none. In the older literature, the prime-field proposals above give either a conditional reduction (Semaev [17]) or algorithms with no complexity bound below $p^{1/2}$; none claims, even heuristically, time $p^{1/2-\varepsilon}$ for general curves over $\mathbb{F}_p$. To our knowledge, then, Theorems 1 and 4–6 give the first algorithm for general curves over prime fields that is asymptotically faster than the square root, and its bound is proved, not heuristic; we make no stronger priority claim. Williams [51] showed that improving exhaustive search for circuit satisfiability implies circuit lower bounds; the engine papers discuss which of these consequences follow from their algorithms.

Factoring and linear algebra. For Theorem 6 we use the sparse black-box solvers of Chen, Eberly, Kaltofen, Saunders, Turner and Villard [35]. We count points with Schoof's algorithm [30].

10 Limitations

The results are galactic. All statements are asymptotic in p. The algorithms of Theorems 1 and 4–6 cost $p^{1/m}$ engine calls for the relations plus $p^{2/m+o(1)}$ for the linear algebra. So for the proved guarantee to beat Pollard's $2^{L/2}$ it is necessary that $2/m < 1/2$, that is $m \geq 5$, and that $L/m + \log_2 T_{\mathrm{eng}} < L/2$. Since $\log_2 T_{\mathrm{eng}} \geq e_\ell c_1 m L(1+o(1))$ by (3), the second condition requires

$$e_\ell \, c_1 m < \tfrac{1}{2} - \tfrac{1}{m},$$

with $e_\ell \geq \frac{1}{2}\rho^\ell$ from the Engine Theorem (a) and ℓ the largest index allowed by the cluster condition of the network. We evaluated this condition, minimized over $m \geq 5$, with the pieces $\rho = 1/\sqrt{2}$ of Appendix B of [22] and $x_0 = 18$, which is admissible for them (Section 2). Table 5 lists the resulting thresholds on $\log_2 L$, with the status of each input. Here $L = \lceil \log_2 p \rceil$, so $\log_2 L \approx \log_2 \log_2 p$. All thresholds are rounded up in the last digit shown (exact: $2.01326 \cdot 10^9$, $4.02653 \cdot 10^9$, $5.15396 \cdot 10^{11}$, $6.75540 \cdot 10^{16}$, $1.69253 \cdot 10^8$, $6.29142 \cdot 10^7$, $4.37178 \cdot 10^7$). They were recomputed with an independent implementation of the recursion $e_{\ell+1} = (\rho + 3 \cdot 2^{-\ell-5}) e_\ell + 3/K_\ell$.

The size bounds of the first three rows rest on closed forms of the constructed networks, so the statement is not an enumeration. For V8x we proved on paper that $U_{\mathrm{gate}}/(5L) \leq 89.0075 + (0.6875j + 12)/2^j$ for every $j \geq 2$ and every $L \in [64 \cdot 4^j, 256 \cdot 4^j)$ ($D = 2^j$), hence $c_1 \leq 89.01$ for $L \geq 2^{34}$; the proof uses structural facts of the network (sizes of the digit and transform nodes), which we derived from its closed forms and checked on constructed networks up to $L = 2^{400}$. We have not reviewed this argument independently.

The block clustering for Lemma 8(ii) (new in version 2). The first two rows of Table 5 do not use $g = 3K^8$ clusters. Case (ii) of Lemma 8 in the text of [22] asks only that $K \mid g$, that the cross load satisfies $c_{ab} \leq P/g$ for $a \neq b$, and that $2\zeta^2 g \geq K \ln(2gK)$. For V8x we give an explicit deterministic clustering with these properties: the middle nodes are cut into $g/2$ blocks of equal size, the transform nodes, kept in atoms of at most two nodes, are cut sequentially into $g/2$ blocks, and the cluster parameter is $P^* = 2M f_w(\lceil 2S/g \rceil + 1)$ with M middle nodes, S transform nodes and edge width f_w. It has $c_{ab} \leq P^*/g$ and $gP^*/U \leq 7/3$, and it is valid for every power of two $g \leq \frac{1}{2}L^{1/4}$ because the number of middle nodes exceeds $L^{1/4}/4$. This is the effective degree $D_{\mathrm{eff}} = 4$, against $g = 3K^8 \leq \frac{1}{2}L^{1/4}$ in the headline row, and it is why the exponent $L \approx K^4$ replaces $L \approx K^{32}$. The hypotheses were checked on 221 constructed networks (1081 triples (L, m, g), 5820 pairs of block counts, no failure) and independently re-checked by a red team with its own implementation. With $\zeta_{\mathrm{top}} = 2^{-20}$ the factor $1 + 2\zeta g P^*/U$ raises c_1 to $c_1^{\mathrm{eff}} = 89.0104$.

With the recursion of the Anthropic paper and a schedule with $x_0 = 7$ and free level parameters s_ℓ, ζ_ℓ chosen by dynamic programming (all conditions checked in outward-rounded decimal arithmetic at each of the 20 levels, with $x_{20} = 42{,}313{,}122$ and $s_{\mathrm{top}} = 65$), the condition holds for $\log_2 L \geq 4(x_{20} + s_{\mathrm{top}} + 1) = 169{,}252{,}752$, that is $1.693 \cdot 10^8$ after rounding up. With the schedule of the Anthropic paper itself ($x_0 \geq 18$, $\zeta_\ell = 2^{-\ell-5}$) twenty levels are still needed and the threshold is $\log_2 L \geq 4 \cdot 62{,}914{,}398 + O(1) \approx 2.52 \cdot 10^8$.

Assumption (R1). The Engine Theorem (b) is proved in [22] for $x_0 \geq 18$ and $\zeta_\ell = 2^{-\ell-5}$. The first row uses a schedule with $x_0 = 7$. Assumption (R1) is that the proof of (b) goes through for this schedule if, at every level, the explicit conditions it uses hold: $K \mid g$, $2\zeta^2 g \geq K \ln(2gK)$, $1/g \leq \zeta$, $\zeta \leq 1$, $e_\ell \leq 1$, and $(c+5)K^2 \leq g^4/2$ with $c \leq 2^{18} - 5$; the base case $e_0 = \frac{1}{2}$ and the constant C_0, which depends only on c, do not depend on the schedule. We verified this by reading

Network and schedule	$\log_2 L \geq$	Status of the inputs
V8x, block clustering, base case of this section ($e_0 = \frac{1}{4} + \frac{1}{2K_0}$), $x_0 = 8$ ($\ell = 18$)	$4.372 \cdot 10^7$	As the next row, plus (R1). Base case proved on paper (Lemma 10.1); the schedule arithmetic, including $s_{\text{top}} = 63$, is a Lean theorem (`EOQ_arith_free`).
Same, schedule of the Anthropic paper ($x_0 = 18$, $\zeta_\ell = 2^{-\ell-5}$)	$6.292 \cdot 10^7$	As the row with $2.52 \cdot 10^8$, without (R1), plus Lemma 10.1. The schedule arithmetic is a Lean theorem (`EOQ_arith_paper`); $x_{18} = 15{,}728{,}490$, $s_{\text{top}} = 64$.
V8x, block clustering of Lemma 8(ii), $x_0 = 7$ ($m = 5$, $\ell = 20$, $c_1^{\text{eff}} = 89.01$)	$1.693 \cdot 10^8$	Valid under assumption (R1) below. Clustering proved on paper, checked on 221 constructed networks; c_1 bound proved on paper for $L \geq 2^{34}$ and checked exactly on $1.7 \cdot 10^7$ values of L; value of V8x and Definition 2.2 checked numerically only; the top-level overhead terms are bounded by size, not given with explicit constants.
Same, schedule of the Anthropic paper ($x_0 \geq 18$)	$2.52 \cdot 10^8$	As above, without (R1). The schedule ($x_{20} = 62{,}914{,}398$) was reproduced by a red team, with $e_{20}\, c_1^{\text{eff}} \cdot 5 = 0.280$; 19 levels fail (0.396).
V8x, $K_\ell \leq (L/1296)^{1/32}$, $c_1(5) \leq 91.8$	$2.014 \cdot 10^9$	Value checked numerically against brute force (346 instances) and Definition 2.2 (2.3 million setting pairs); no proof of the value in this paper. Size bounds: closed forms checked on constructed topologies up to $L = 2^{400}$; beyond, and hence at the threshold, extrapolated (argued).
X, $c_1 = 125$ ($\ell = 21$)	$4.027 \cdot 10^9$	Value proved on paper and checked on 292 instances. Size bounds as for V8x.
Compact network of Section 3.1, $c_1 = 1410$, $K_\ell \leq L^{1/32}$ ($\ell = 28$)	$5.154 \cdot 10^{11}$	Value proved on paper and checked; size bounds proved on paper and checked. $4.510 \cdot 10^{11}$ with the cluster bound for every g ($K_\ell \leq L^{1/28}$), $1.289 \cdot 10^{11}$ with its asymptotic constant 748.
Edge network, $c_1 = 549504$, $K_\ell \leq (L/81)^{1/32}$ ($\ell = 45$)	$6.756 \cdot 10^{16}$	Value and size bounds are Lean theorems.

Table 5: Crossover thresholds for $m = 5$ with the recursion of the Anthropic engine. All rows use $\rho = 1/\sqrt{2}$, and all use $c = 12$ for the pieces of Appendix B of [22], which is our own re-derivation (Section 2) and not a statement of that paper.

the proof line by line, and the conditions level by level in exact arithmetic. It is a hand audit, not a Lean check, and the Anthropic Lean development was not rebuilt. We also assume that the correctness of the V8x network and the modelling of the held bits in Definition 2.2 are as in Appendix A, and that the top-level overhead terms (K^g, $(K(L'+B'))^{C_0K^2}$ and $2^{2gP/K}$) are $2^{o(L)}$, which holds by size but was not derived with explicit constants. A red-team review of the clustering and of the schedule arithmetic found no failure and rated (R1) the main open risk; an independent review of the written audit of (R1) checked every schedule condition in exact arithmetic for $c = 12$ and $c = 2^{18} - 5$, found no failure, and found that any $x_0 \geq 3$ suffices with a free schedule. If (R1) fails, the best figure that does not use it is $6.292 \cdot 10^7$ (with the base case below), or $2.52 \cdot 10^8$ without it.

A faster base case (new in version 2). The Engine Theorem starts its recursion with $e_0 = \frac{1}{2}$: the base case evaluates a (K_0, λ)-clique by brute force over all $K_0(K_0-1)/2$ edge strings [22, l. 1613–1615]. The value of a clique is a sum over the edges of a product of node functions, each depending on the $K_0 - 1$ edges at its node, so it is a contraction along the line graph $L(K_{K_0})$, and it can be evaluated by variable elimination along a path decomposition of that graph.

Lemma 10.1
Let $\mathrm{tw}_K = \mathrm{tw}(L(K_K))$, which equals $(K-2)K/4 + K - 2$ for even K [52]. The value of a (K, λ)-clique with node functions bounded by 2^B can be computed with at most $(K(\lambda + B))^5\, 2^{\lambda(\mathrm{tw}_K+1)}$ operations in the cost model of [22, Definition 4]. Hence the base case of the Engine Theorem holds with

$$e_0 = \frac{\mathrm{tw}_{K_0}+1}{K_0^2} = \frac{1}{4} + \frac{1}{2K_0} - \frac{1}{K_0^2} \leq \frac{1}{4} + \frac{1}{2K_0} \qquad (K_0 \text{ even}).$$

Proof sketch. Harvey and Wood give an explicit path decomposition $A_1, \ldots, A_K$ of $L(K_K)$ of width tw_K [52, Lemma 14]; only this upper bound is used. Sweeping along it, keep for each assignment of the edges in the current bag the sum over the already eliminated edges of the product of the node functions whose edges are all eliminated or present. Each node function's $K-1$ edges lie in a common bag, so it is queried once per assignment of that bag. A bag has $2^{\lambda|A_i|} \leq 2^{\lambda(\mathrm{tw}_K+1)}$ assignments; there are K bags; the partial sums have at most $K(\lambda+B)+K^2\lambda$ bits. Counting queries, additions, multiplications, carries and table initialisation gives the stated bound. The base case of [22, Theorem A] needs $2^{e_0K_0^2\lambda}(K_0(\lambda+B))^{C_5}$ and the induction uses e_0 only through e_1, so its other steps are unchanged. □

The value of the sweep was checked against brute force for $K_0 \leq 6$ and $\lambda \leq 3$, exactly where feasible (two settings with 2^{45} terms were not run), and the measured operation counts were within the stated bound in every run. Tree contractions cannot do better: the contraction complexity of a tensor network equals the treewidth of its line graph [53], so $\frac{1}{4}$ is optimal for them; we found no better method (matrix-multiplication groupings give at least 0.251, splitting and listing 5/16). Halving e_0 removes exactly two levels of the recursion, since $\rho = 1/\sqrt{2}$, and each level doubles the exponent of L: with the schedule of the Anthropic paper the threshold falls from $2.52 \cdot 10^8$ to $6.292 \cdot 10^7$, a factor 4.000, and with a free schedule ($x_0 = 8$, 18 levels) to $4.372 \cdot 10^7$. Both schedules were found in exact arithmetic, recomputed independently, and their arithmetic is checked in Lean (Section 8). Parts (a), (c) and (d) of the Engine Theorem are stated in [22] with $e_0 = \frac{1}{2}$ and are not used for the thresholds.

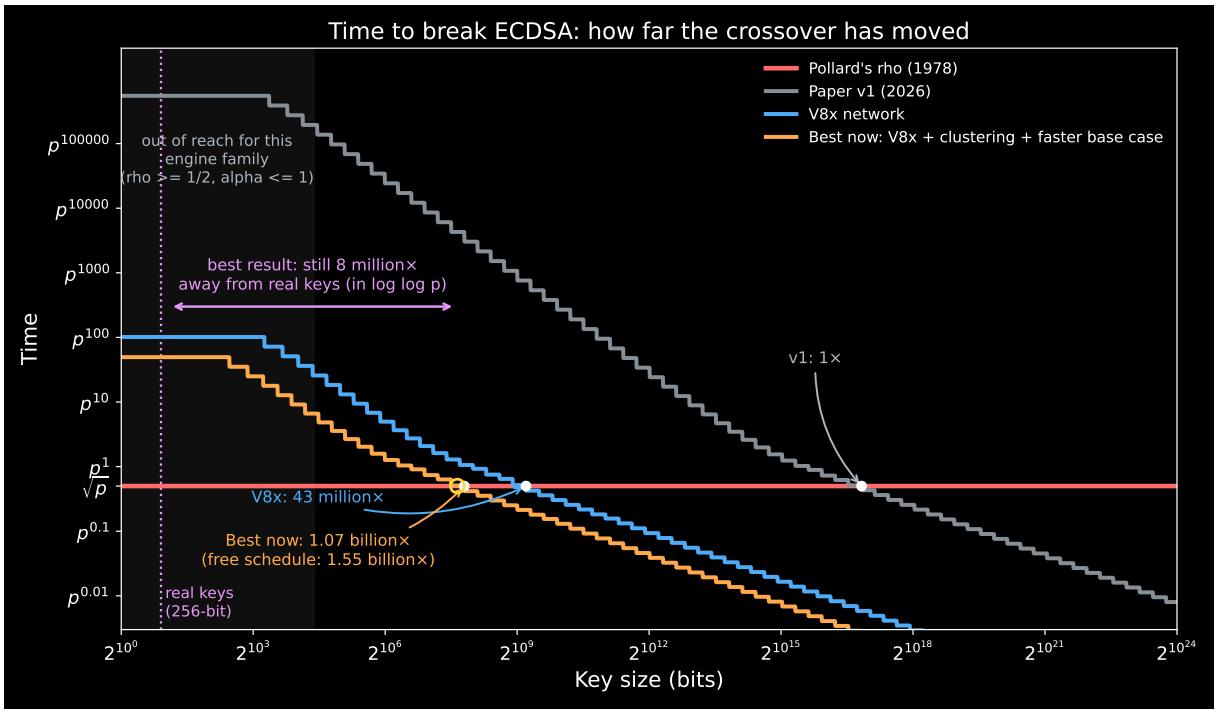


Figure 1: Proven cost of the algorithm against Pollard's rho as a function of the key size L (horizontal axis $\log_2 L$, double-logarithmic in p), for the edge network of version 1, the network V8x, and V8x with block clustering and the base case of Lemma 10.1 (paper schedule). The white dots are the crossovers of Table 5; the hollow circle is the free-schedule variant. The shaded region is below the threshold that any engine with $\rho \geq \frac{1}{2}$ can reach (argued); real 256-bit keys sit at $\log_2 L = 8$.

Earlier figures. Two refinements lower the V8x headline row further. Its clustering works for every g, which gives $K_\ell \le \theta L^{1/28}$, and the inequalities in the proof of the Engine Theorem that use $x_0 \ge 18$ also hold for $x_0 = 16$ (but fail for $x_0 = 15$), which is admissible since $c = 12 \le 2^{16} - 5$. Together they give $\log_2 L \ge 1.586 \cdot 10^9$. The second refinement re-checks part of the proof of [22, Theorem A] on paper and is not backed by its Lean development, so we do not use it for the headline row. The thresholds are lower bounds for these network designs: they take e_ℓ at its lower bound and ignore lower-order terms, so p must have more than 2^T bits, with T the figure shown. Tracing the Anthropic Lean proof gives $C_1 = 44$ and $C_0 = 300c + 3615$, and these change the thresholds by a negligible amount. With only the main-text pieces $\rho = 11/12$ and $c_1 = 1410$ the condition gives $\log_2 L \ge 6.3 \cdot 10^{35}$ when e_ℓ is taken at its lower bound $\frac{1}{2}\rho^\ell$, and $2.5 \cdot 10^{36}$ with the exact recursion (the exact e_ℓ is larger than the bound by a factor up to 1.29, which moves the threshold from level 108 to level 110); Section 7 quotes the second figure. For p of cryptographic size the engine does not even apply: it needs $g = 3K^8$ clusters with $K \ge 2^{18}$, far more than there are bits. The galactic size comes from the growth of K_ℓ in the Engine Theorem, not from our network: even $c_1 = 1$ would leave a threshold near $\log_2 L \approx 2.431 \cdot 10^5$. All these thresholds use the recursion of the Anthropic engine, that is (E) (Engine Theorem (a)). The oracle of Theorems 1 and 4–6 runs instead on the Lean engine of Rudich and Rousseau [24], whose cost constants we have not extracted; for that engine the thresholds are not computed.

No effect on deployed curves. Nothing here says anything about the security of P-256, secp256k1, Curve25519 or any other curve at its actual size. The statements are uniform over families as $p \to \infty$; they are not statements about one curve at one size.

The engine is unrefereed. Hypothesis (E) consists of statements from an unrefereed preprint [22], formalized in Lean by its authors; Section 2 quotes them in full. We read the statements of the Engine Lemma and the Engine Theorem from the paper text and from the Lean index, and we did not rebuild the Anthropic development. The Lean pipeline of Section 8 instead uses the Lean engine of Rudich and Rousseau [24], also an unrefereed preprint. If (E) fails, the deterministic y-digit extensions of Theorem 2 and Theorem 7 lose their proofs. The counting and the search of Theorem 2 over the x-string with sign bits do not: both are Lean theorems with no hypotheses, engine included, and independent audits found their statements faithful (Section 8). Theorems 1 and 4–6 do not either: their oracle (O) is that Lean theorem, with the points recovered from their labels by a Las Vegas square root (Section 5). Theorem 3 does not use the engine.

Constants are existential. The Anthropic paper gives no numerical values for C_0, x_0 and the exponent C_1 of the Engine Lemma. We traced $C_1 = 44$ and $C_0 = 300c + 3615$ through its Lean proof by hand, and $x_0 = 18$ is admissible for the pieces we use (Section 2). The constants C_k of Theorem 2, and hence the $o(1)$ terms in all exponents, are not explicit. Our own constants (c_1, c_2, $c_m(\kappa)$, the 13 of Lemma F) are explicit but loose against measurements, by factors from about 3 (the size constants of the network of Section 3.2; about 10^3 for the edge network) to about 10^6 (the coverage constant $c_m(\kappa)$ at $m = 2$).

Status of (N). Theorem 2 rests on the edge network, whose value and size bounds are Lean theorems (Proposition 3.7). The extension of the prefix to the y-digits (Lemma 3.8) and the cost τ and uniformity of the construction (Lemma 3.9) are paper proofs. One adversarial review

found gaps in both (a missing alignment convention for the y-digits, an unproved structural claim, and a cost argument that omitted the clustered network); the proofs were revised in response and have not been reviewed again. The x-and-sign case of Lemma 3.8 is Proposition 3.7, a Lean theorem. For the hand-optimized network of Section 3.1 and the networks X and V8x of Appendix A, that the value equals $|G(R,\pi)|$ is proved on paper and checked against brute force, not proved in Lean.

Degenerate decompositions. Theorem 2 counts generic decompositions only: no doubling steps, no steps $S+(-S)$, no partial sum O, and $R \neq O$. The extension to the full problem (Remark 4.2) is checked on small primes, but its size bound is argued, not proved. For the ECDLP algorithms this costs nothing, since the coverage bounds count generic tuples only.

Theorem 1 is the most recent result. Theorem 1 (every curve, every group order, every fixed m) was obtained last. It has had one independent adversarial review, which found one error in the probability bookkeeping (now corrected), and its algebra has been checked by two independent toy implementations. Its earlier and weaker versions, Theorems 5 and 6, have independent proofs. Theorem 4 does not depend on it.

Theorem 7 is conditional. Theorem 7, with growing m, is the only discrete-logarithm result that uses (E): its oracle runs on the Anthropic engine, because the Lean oracle of Theorem 2(ii) gives no bound uniform in m. It needs no hypothesis on the cost τ of the network's local functions: Lemma 7.1 gives $\log_2 \tau \leq \log_2 m + \frac{7}{4}\log_2 L + O(1)$ for the edge network, uniformly in m, through Lemma 3.9, which is a paper proof (per-node counts read by hand from the Lean definitions and independently recomputed). The correctness and size bounds of the network for every m (Proposition N) are Lean theorems for the prefixes Theorem 7 uses. The uniform engine bound (Proposition 7.2) rests on the absoluteness of the constants C_0 and C_1, which we read from the Lean statements `RecursionSolves` and `ExplicitCost.GroupingExplicit` without rebuilding that development. Theorem 7 has had one adversarial review of the engine bound and says nothing at any size anyone could compute: $m^* \geq 2$ already requires $\log_2 L \geq 6.25 \cdot 10^4$.

The plain box $u = 1$ is open. Theorem 4 randomizes over isomorphic models. Whether the factor base $\{x \leq p^{1/m}\}$ on the given model already has coverage bounded below is open. By Parseval, a sufficient condition is that every nontrivial character ψ of $E(\mathbb{F}_p)$ satisfies $|\sum_{P\in F}\psi(P)| \leq p^{1/(2m-2)}$. Known bounds for such sums over an x-interval (Kohel and Shparlinski [28] and later work) are nontrivial only for intervals longer than $p^{1/2+\varepsilon}$, and the sup-norm route fails even with Burgess-strength input [54] for $m = 3$. In exact computations on prime-order curves with p up to about 10^5 and $m = 2, 3$, the box $u = 1$ is statistically indistinguishable from a random dilate.

Why the engine is not applied to $dP = Q$ directly. The engine could count the d with $dP = Q$ and a given binary prefix, and prefix search would then find d with L counts. The obstacle is the size of the network. A check of $dP = Q$ by double-and-add chains $\Theta(L)$ group operations, each with $\Theta(L)$ bits of witnesses, so $U = \Theta(L^2)$. The engine's exponent is about $e_\ell U$ with $e_\ell \asymp (\log L)^{-\alpha'}$ (Section 4), which for $U = \Theta(L^2)$ is far above L: worse than exhaustive search. A decomposition has $L + O(m)$ free bits and needs only $O(m)$ group-law checks, so $U = O(mL)$. This is why the engine is used only to find relations.

Consistency with earlier obstructions. The algorithm reads coordinates, so the generic lower bounds of Nechaev and Shoup do not apply. In earlier unpublished work on this problem we proved several obstructions to fast decomposition, and we checked each against the present results. None is contradicted. Lang–Weil-type bounds [55] show that a usable factor base of size $p^{1/m}$ must be a zero-dimensional set with a succinct description; the box is one. A degree barrier holds only for field-arithmetic formulas without shared subterms, which the engine is not. A cell-probe barrier concerns random factor bases given as tables, which have no succinct description. A barrier conditional on NSETH is vacuous if the engine papers are correct, since their algorithms would refute NSETH [22, 23]. One conjecture from that work fails: that every succinct factor base of size about $\sqrt{p}$ requires time $p^{1/2-o(1)}$ per decomposition. Theorem 2(ii) with $m = 2$, a Lean theorem, refutes it. The same question for the factor base of k-th roots of unity ($x^k = 1$) is open. The obvious networks for $x^k = 1$ have $U = \Theta(L^2)$, and a network with $U = O(L)$ for exponentiation would give deterministic $p^{o(1)}$ discrete logarithms in $\mathbb{F}_p^*$.

11 Conclusion and open problems

We proved four results.

- *ECDLP below the square root.* For every fixed $m \geq 2$ and every elliptic curve over $\mathbb{F}_p$, of any group order and structure, a Las Vegas algorithm solves the ECDLP in expected time $p^{2/m+o(1)}$ (Theorem 1; Theorems 4–6 are special cases and weaker versions with independent proofs). For $m \geq 5$ the exponent is below $1/2$. The algorithms use no heuristic about decompositions or about the rank of the relation matrix, and their decomposition oracle is a Lean theorem.

- *Decomposition search.* For every fixed $m \geq 2$, a deterministic algorithm finds the lexicographically least x-string with sign bits of a generic decomposition $R = P_1 + \cdots + P_m$ into points with $x(P_i) < \lceil p^{1/m} \rceil$, or reports that there is none, in time $C_k 2^{L/k}$ for every $k \geq 1$ (Theorem 2(ii)). This is proved in Lean with no hypotheses, engine included, and so is the counting statement, Theorem 2(i), for the same prefixes.

- *Coverage.* On a random isomorphic model of any curve, a fraction at least $c_m(\kappa)$ of all points have a generic decomposition into points with x-coordinate in $[1, X]$, with probability at least $1 - 1/\kappa - 56/X$ (Theorem 3). This is unconditional and replaces the usual $1/m!$ heuristic.

- *Growing m.* Letting m grow as $(\log_2 L)^{\alpha'/2}$ gives expected time $p^{o(1)}$, conditional on (E) and (N) (Theorem 7, Section 7).

All bounds are asymptotic, the constants are galactic, and nothing here bears on any curve in use (Section 10).

The paper leaves the following problems open.

1. *The plain box.* Does the factor base $\{x \leq p^{1/m}\}$ on the given model ($u = 1$) already have coverage bounded below? A sufficient condition is a character-sum bound over x-intervals far shorter than known methods reach (Section 10).

2. *A practical decomposition step.* With the constants we can prove for the Anthropic engine, the guarantee beats Pollard's rho only when $\log_2 \log_2 p \geq 2.014 \cdot 10^9$ (network V8x, Appendix A), or $\geq 6.292 \cdot 10^7$ with the block clustering and the faster base case of Section 10 ($4.372 \cdot 10^7$

under the audited free schedule, Figure 1). The threshold comes mainly from the growth of the cluster parameters K_ℓ in the engine, not from our network. Can the decomposition step be made practical, or the crossover brought anywhere near real sizes?

3. *Other groups, per curve.* Our results are for elliptic curves over prime fields. The genus-2 extension (Appendix B) holds only on average over curves, needs (E) and (N_2), and reaches prime-order Jacobians only under the unproved density hypothesis (H_2). A per-curve statement in genus ≥ 2, and an analogue over extension fields $\mathbb{F}_{p^n}$, are open.

4. *The rest of Theorem 1 in Lean.* The coverage bound (which needs the Weil bound), point counting and factoring, the sparse linear algebra, the deterministic y-digit extension (Lemma 3.8) and Lemma 3.9 are paper proofs (Section 8). They have been checked by independent adversarial review and numerical checks, not by machine. The combinatorial core of the index calculus is in Lean.

5. *Other factor bases.* For the k-th roots of unity ($x^k = 1$) the obvious networks have $U = \Theta(L^2)$. A network with $U = O(L)$ for exponentiation would give deterministic $p^{o(1)}$ discrete logarithms in $\mathbb{F}_p^*$.

What a practical algorithm would need. No engine of this kind can reach deployed key sizes: the flattening rank of the swap tensor forces $\rho \geq \frac{1}{2}$ per level, so the threshold cannot fall below roughly $\log_2 \log_2 p \approx 2.5 \cdot 10^4$ (argued), against 8 for a 256-bit key. A direct decomposition would have to cost less than about $p^{0.34}$, $p^{0.37}$ or $p^{0.39}$ per attempt for index calculus to beat rho at 160, 256 or 521 bits ($m = 12$, all constants counted; computed). We tested the routes we could find, and each stays at $p^{1/2}$ or above or turns out to need discrete logarithms:

- *Filters for list merging.* Wagner-type and representation methods need a cheap map φ with $\varphi(P + Q)$ predictable from $\varphi(P), \varphi(Q)$. If φ takes s values and the prediction succeeds on a fraction α of pairs above the null level μ (the largest class mass), then some fibre of φ has a Fourier coefficient at least $(\alpha - \mu)/s$ at a nonzero frequency, and the sparse Fourier transform [56] then recovers discrete logarithms with $\tilde{O}(s^{14}(\alpha - \mu)^{-13})$ queries (proved, with the cited algorithm). This rules out filters with $s \leq n^{1/28}$; filters with larger image, or correct only on sums of factor-base points, are not covered.

- *Exact counting with additive characters.* Counting decompositions with the additive characters of $\mathbb{F}_p$ applied to S_3 avoids partial sums, but the outer sum is flat (root mean square about $1.4B$ for every t), so the cost is pB (computed and argued).

- *Lattices with automated shift selection.* Automated shift polynomials reproduce the hand bounds ($p^{1/6}$ per variable for S_3), and on a system with the curve equations the recovered polynomials are not algebraically independent (computed).

- *Structure in the solutions and symmetries.* About 1400 pre-registered statistics of the decomposition sets, replicated on held-out primes, found only the integer trace identity $x_1 + x_2 + x_3 = \lambda^2$ for small slopes and a 2-descent identity; neither helps. No natural map keeps small x-coordinates small beyond a factor 2 (computed).

- *Measured comparison.* A full index calculus with $m = 3$ loses to Pollard's rho with the negation map at every size from 10 to 40 bits, and the fitted trends do not cross (computed).

Routes that do not give a further improvement. We tried several routes to a faster decomposition step or a smaller network. None improved on the results above. Each statement below is an argued barrier or a computed negative result on the instances we tried. None is an impossibility proof, and each leaves open that a different construction works.

- *Meet in the middle and list merging.* Splitting $R = P_1 + \cdots + P_m$ into two halves costs $p^{\lceil m/2 \rceil / m} \geq p^{1/2}$ per decomposition, and $p^{(m+1)/(2m)}$ or more when relations are collected in batches, so never below $\sqrt{p}$ (argued; exact counts on toy primes agree). The partial sums of factor-base points are equidistributed in size (computed at $p \approx 2 \cdot 10^6$), so no bound on a partial sum filters the lists, and Wagner's k-list merging needs lists of size $p^{1/(1+\log_2 m)}$, larger than the factor base.

- *Character sums.* Counting decompositions through the characters of $E(\mathbb{F}_p)$ needs the full table of character sums over the factor base. The main term is $O(1)$ and the fluctuation, of size $|F|^{m/2}$, swamps it (computed).

- *Lattices and Coppersmith's method.* For $S_3(x_1, x_2, x_R) \equiv 0$ with x_1, x_2 in a box of side $p^{1/2}$, a lattice method with box shifts reaches total exponent 0.331 for $m = 2$ (the known 1/3 bound). The computed values for $m = 2, \ldots, 6$ fit $2/((m+1)2^{m-1})$ (an empirical fit), against the exponent 1 that a decomposition needs. On 200-bit primes the bivariate method reached side about $p^{0.13}$, not $p^{0.5}$. Guessing some points and applying the lattice to the rest costs at least $p^{5/7}$. The curve $S_3(x, y, x_R) = 0$ has genus 1 and is E itself, so there is no rational parametrization to scan, and translations and isogenies do not preserve smallness of x (argued).

- *Separator-based exact counting.* Contracting our networks along tree decompositions gives widths of about $18\,mL$, linear in L, so the separators do not shrink the cost below the $\Theta(mL)$ that the networks already carry, and a larger transform length does not help (computed; heuristic upper bounds). The exact ranks of the multiplication cut matrix are close to full on small cases, as the information-transfer criterion predicts for any separator method. This says nothing about algorithms that are not separator-based.

- *Cheaper networks.* Replacing the group-law identities by Semaev's x-only S_3 chain, or checking the congruences modulo p by small primes or by other ring structures, did not lower the size constant below a modelled $c_1(5) \approx 73$–84, against 91.8 for V8x (and 85.4 for a variant whose crossover rests on an argued recursion). One further engine level needs $c_1(5) < 61.4$. The cost of sending one variable in the evaluation-grid family is a floor of about $4L$ per end, the x-only chain needs odd group order and cannot test the y-digits of a prefix, and the small-prime variant costs $L^{5/4}$ bits, which is not linear. These costs are in a model calibrated on V8x and V8y (argued). A super-linear wire lower bound for multipliers modulo p, which would settle the question, is not known.

Appendix A: Smaller networks (X and V8x)

Theorem 2 uses the edge network of Section 3.3. This appendix describes two networks with constants about ten times smaller than those of Section 3.2, which give the best crossover estimates of Section 10. Neither needs any number-theoretic hypothesis. Nothing here is in Lean.

The network V7 and its defect. Version 1 of this paper described a network V7 with these changes to the network of Section 3.1. (1) Instead of an inverse transform and a carry chain per identity, the prover supplies the carry polynomial $\mathcal{Q}$ with $\mathcal{E}(\xi) = (\xi - 2^d)\mathcal{Q}(\xi)$ over $\mathbb{Z}$, and the middle nodes check $\hat{\mathcal{E}}_k = (\omega^k - 2^d)\hat{\mathcal{Q}}_k$ at the T transform points. (2) The condition $x(S_{k-1}) \neq x(P_k)$ is checked by a digit-comparison automaton instead of an inverse. (3) All variables are transformed once, and one shared group of middle nodes checks all identities. (4) The y-coordinates of partial sums are eliminated by substitution. (5) The last point is not a variable: it is determined by the two sum identities. Change (5) is wrong for our purpose. It excludes the tuples with $S_{m-2} = -R$, and since y_{m-1} is not a variable, the sign bit and the y-digits of the last point cannot be tested. So V7 does not compute $|G(R, \pi)|$ for every prefix π. Earlier checks used only x-prefixes without sign bits and missed this; on $p = 251$ and $p = 509$ with $m = 3$ an independent count finds the excluded tuples. Version 1 also took the transform modulus to be a power of the least prime $q \equiv 1 \pmod{T}$ and needed a hypothesis bounding q. Both points are repaired below.

Lemma A.1 (transform modulus)
Let M be a power of 2, $x \geq 2$ even, $N = x^M + 1$, $T = 2M$ and $\omega = x$. Then $\omega^T \equiv 1 \pmod{N}$, and $\omega^i - \omega^j$ is a unit modulo N for $0 \leq j < i < T$. Hence a polynomial of degree below T that vanishes at all ω^k is 0 modulo N.

Proof. $\omega^T = (x^M)^2 \equiv 1$. Let r be a prime dividing N; r is odd since N is. Then $x^M \equiv -1 \pmod{r}$, so the order of x modulo r divides $2M$ but not M, and since M is a power of 2 it equals $2M$. For $0 < a < T$, $x^a \not\equiv 1 \pmod{r}$ for every such r, so $x^a - 1$ is a unit modulo N, and $\omega^i - \omega^j = \omega^j(\omega^{i-j} - 1)$ is a unit. The Vandermonde matrix of the points ω^k is then invertible modulo N. □

The network X. X is V7 with the last point a variable, as in Section 3.1, the modulus of Lemma A.1, and a two-sided interval chain on every y_i that encodes its sign bit and the leading y-bits of the prefix. Its value is $|G(R, \pi)|$ for every prefix π of the string σ of Definition 3.1, proved on paper; the statement for V8x below is checked numerically only. Checks: network values equal brute-force counts on 292 instances, including prefixes into the sign bits and the y-digits (0 mismatches); Definition 2.2 holds on 200 instances, and 16,000 deliberate mutations were all rejected; Lemma A.1 was tested on 33,727 cases.

Proposition A.2 (sizes of X)
For all $L \geq 1024$ and $2 \leq m \leq \sqrt{L}$: $U \leq 125\,mL$, $U + H \leq 260\,mL$, the largest node weight is $O(L^{3/4})$, and the cumulative-weight clustering gives $gP \leq 400\,mL$ for $g \leq \frac{1}{2}L^{1/4}$.

The proof follows that of Propositions 3.3–3.6. Closed forms were checked exact against 378 constructed topologies and evaluated on about $9.5 \cdot 10^6$ pairs (L, m) up to $L = 2^{400}$; beyond that, monotonicity of the error terms is argued. The modulus of Lemma A.1 needs $M \geq D$, which lowers the cluster exponent from 1/2 to 1/4. In Theorem 2, Step 1 becomes: ℓ is the largest index with $K_\ell \leq (L/1296)^{1/32}$, so that $g = 3K^8 \leq \frac{1}{2}L^{1/4}$; Steps 2–4 are unchanged.

The network V8x. V8x is X with two further changes: the transform ring is $\mathbb{Z}/(2^{jD} + 1)$ with $\omega = 2^j$ of order $2D$, the case $x = 2^j$ of Lemma A.1 with the lemma's M equal to D (here D is a power of 2 by construction, which the lemma needs; the lemma's M is not the exponent jD of the modulus); and the range and not-equal automata run inside the digit-holding gates, as one-bit

state edges, instead of in separate chain gates. About 92% of U comes from 12 full transform sends per point (five variables, three quotients and four carry polynomials). For $L \geq 1024$, $U \leq c_1(m)\, mL$ with $c_1(5) \leq 91.8$ (and 89.0 asymptotically), $U + H \leq 161\, mL$, and the heaviest node has weight at most $613(m-1)L^{3/4}$. Checks: network values equal brute-force counts on 346 instances with full prefixes, including sign bits and y-digits (0 mismatches); Definition 2.2 holds on 2,337,768 setting pairs of the actual topology (0 failures); and an independent evaluator, written from the definition, matched brute force on 81 further instances. Within this style of network $c_1 \geq 96(m-1)/m$, so one further engine level, which needs $c_1 < 61.4$ at $m = 5$, is out of reach.

Clusters for every g. For X and V8x, as for the network of Section 3.1, the cumulative-weight clustering satisfies $P < W/g + w_{\max}$ for every $g \geq 1$, where $w_{\max}$ is the largest node weight. With $w_{\max} = O(L^{3/4})$ the level condition becomes $K_\ell \leq \theta L^{1/28}$ for an absolute θ, and 28 is the best exponent for this family of rings. This was checked on all constructed topologies for g up to beyond the number of gates.

With these substitutions, Theorems 1, 2 and 4–6 would hold with X or V8x in place of the edge network, as paper proofs, if the value and size statements above are accepted. For X the value is proved on paper; for V8x the value is checked numerically and has no proof in this paper, and the size bounds of both are checked up to $L = 2^{400}$ and extrapolated beyond (for the c_1 bound of V8x see the analytic bound in Section 10). Section 10 gives the resulting crossovers and the status of each.

Appendix B: Genus 2, on average over curves

The method extends to Jacobians of genus-2 curves $C : y^2 = f(x)$, f monic squarefree of degree 5, with $J = \mathrm{Jac}(C)(\mathbb{F}_p)$ of order about p^2. The factor base is $\{(x, y) \in C(\mathbb{F}_p) : x < \lceil p^{2/m} \rceil\}$, and the targets are divisor classes $D = (u, v)$ in Mumford form. A decomposition is generic if the x_i are distinct and $u(x_i) \neq 0$.

Proposition B.1 (relation lemma)
For $m \geq 3$ and distinct x_i with $u(x_i) \neq 0$, $\sum_i P_i - m\infty \sim D$ if and only if there is a unique monic $\varphi = a(x) + b(x)y \in \mathcal{L}((m+2)\infty)$ vanishing at the P_i with $a - bv \equiv 0 \pmod{u}$.

The proof is by Riemann–Roch on the quintic model. Since the witness φ is unique, the network counts each tuple once; its counts equal an independent Cantor-algorithm brute force on 34 instances and, with two further independent implementations, on 30 more. The measured sizes are $U/(mL) \approx 650$–740 for $m = 3, \ldots, 8$ at $L = 512$, about 2.4 times the elliptic network. We call the corresponding size hypothesis (N_2); it is extrapolated, not proved.

Proposition B.2 (average coverage)
For $3 \leq m \leq 6$, $K \geq 1$ and $p \geq p_0(m, K)$, for a fraction at least $1 - 1/K - 65/X - o(1)$ of monic squarefree quintics, the factor base has at least $X/2$ points, its m-fold energy is at most $KM_m p^2$, and a fraction at least $c_m(K) = 1/(16^m \cdot 256 \cdot KM_m)$ of the classes in J have a generic decomposition.

The proof uses a criterion for principal divisors, a tangent-map surjectivity argument (rank $2m$ for $3 \leq m \leq 6$), Heintz's Bézout inequality [57] and the point bound of Cafure and Matera [58] on each fiber, then Chebyshev and Cauchy–Schwarz as in Section 5. An independent adversarial

review re-derived it, and a rank test at $p = 1009$ confirmed rank $2m$ for $m \le 6$ and rank $2m-1$ at special points for $m = 7, 8$, which is why the statement stops at $m = 6$.

With the decomposition oracle extended to genus 2 (hypotheses (E) and (N_2)), index calculus solves the DLP in J in expected time $p^{4/m+o(1)}$ for $m \in \{5, 6\}$, for all curves outside a set of density $O(1/K + 65/X)$ in any isomorphism-stable family of density $p^{-o(1)}$. Pollard's rho method needs about $\sqrt{|J|} \approx p$. To reach curves with $|J|$ prime one needs a density hypothesis (H_2): monic squarefree quintics with prime-order Jacobian have density at least $p^{-\eta}$ with $\eta < 1 - 4/m$. (H_2) is unproved. There is no per-curve statement: one dilation parameter gives only one equation, and a two-parameter family of models fails for the CM curve $y^2 = x^5 + B$ when $p \equiv 1 \pmod 5$. All of this is an extension of the method, not part of the main claims.

Acknowledgments and declaration of AI use

This research was carried out by the author directing Claude agents. The coordinating model was Claude Opus 5.5, running in a custom research loop built on Claude Code: it split the work into parallel tasks, launched sub-agents (Claude Opus, Sonnet and Haiku models) for proofs, code and literature checks, and accepted a result only after it had passed the verifiers described below (the Lean 4 kernel, checker scripts that can fail, and independent adversarial review and red-team agents). The agents produced the proofs, the code, the Lean formalization and the first drafts of this text. The author chose the problem, set the research direction, split the work into tasks, decided which results to accept, and edited the text. The main results were obtained over about two days, October 30 and 31, 2026.

How claims were checked. No claim in this paper rests on an agent's assertion alone. Every claim was checked in at least one of three ways, with one exception: Lemmas 3.8 and 3.9, short paper proofs added in response to an independent review agent's report, had one adversarial review, and their revision in response to it has not been reviewed again. (1) Machine-checked proof in Lean 4 (Section 8), with the axioms of each theorem printed. (2) A script that recomputes the claim and can fail, re-run by a separate agent: brute-force decomposition counts, exact closed forms of network sizes, energy and coverage computations, toy index calculi. (3) Review by independent agents instructed to find errors. They re-derived the proofs and wrote their own code rather than reusing the authors'. The paper proofs of Theorems 1 and 4–6 were checked in the second and third ways, not in Lean. Each claim's label in the repository (`docs/`) states which checks it passed.

Errors found. Review caught real errors, each fixed before this version. Examples by kind:

- *A constant error.* An early version of the energy bound counted 2^m sign choices for trivial tuples instead of 4^m. The Lean formalization of the counting argument exposed it. A second, smaller one: an error term in Lemma F was $11\sqrt{p} + 2$ instead of $11\sqrt{p} + 5$.
- *A vacuous statement.* An early version of the average-over-curves result excluded a set of curves of density δ. Prime-order curves have density about $1/\log^2 p$, far below δ, so the excluded set could contain every prime-order curve. It was found by review and the statement was corrected.
- *An unsatisfiable hypothesis in the Lean pipeline.* Intermediate versions of the end-to-end Lean theorem carried hypotheses that could not hold for the real network. One required a

node count to be positive at every prefix level, which fails at internal levels; another required a frame cell to be zero in states where it need not be. Such theorems compile but say nothing. The agents found these by trying to discharge the hypotheses, and the pipeline was restructured. The final theorem, `subexponentialRate_all` (Section 8), has no hypotheses.

Review also found a wrong lemma statement for curves with $A = 0$ (Lemma N′ fails without the restriction $X < p/3$) and an unquantified Monte Carlo citation, replaced by the Las Vegas argument of Lemma 6.6.

Limits of this process. The reviewers are AI agents working in the same environment, not human referees, and agents can share blind spots. The engine papers we rely on are themselves recent, unrefereed, and (in Anthropic's case) produced with an internal Claude model; we therefore depend on work from the same company whose models carried out this research. We have tried to mark every step by how it was checked, and to say plainly what is open.

Responsibility. The author takes responsibility for the content of this paper, including any errors.

References

[1] V. S. Miller, "Use of elliptic curves in cryptography," in *Advances in Cryptology – CRYPTO '85*, ser. Lecture Notes in Computer Science, vol. 218. Springer, 1986, pp. 417–426.

[2] N. Koblitz, "Elliptic curve cryptosystems," *Mathematics of Computation*, vol. 48, no. 177, pp. 203–209, 1987.

[3] J. M. Pollard, "Monte Carlo methods for index computation (mod$p$)," *Mathematics of Computation*, vol. 32, no. 143, pp. 918–924, 1978.

[4] S. D. Galbraith and P. Gaudry, "Recent progress on the elliptic curve discrete logarithm problem," *Designs, Codes and Cryptography*, vol. 78, no. 1, pp. 51–72, 2016, also Cryptology ePrint Archive, Paper 2015/1022.

[5] K. Yokoyama, M. Yasuda, Y. Takahashi, and J. Kogure, "Complexity bounds on Semaev's naive index calculus method for ECDLP," *Journal of Mathematical Cryptology*, vol. 14, pp. 460–485, 2020.

[6] D. Shanks, "Class number, a theory of factorization, and genera," in *1969 Number Theory Institute*, ser. Proceedings of Symposia in Pure Mathematics, vol. 20. American Mathematical Society, 1971, pp. 415–440.

[7] V. I. Nechaev, "Complexity of a determinate algorithm for the discrete logarithm," *Mathematical Notes*, vol. 55, no. 2, pp. 165–172, 1994.

[8] V. Shoup, "Lower bounds for discrete logarithms and related problems," in *Advances in Cryptology – EUROCRYPT '97*, ser. Lecture Notes in Computer Science, vol. 1233. Springer, 1997, pp. 256–266.

[9] I. A. Semaev, "Evaluation of discrete logarithms in a group of p-torsion points of an elliptic curve in characteristic p," *Mathematics of Computation*, vol. 67, no. 221, pp. 353–356, 1998.

[10] T. Satoh and K. Araki, "Fermat quotients and the polynomial time discrete log algorithm for anomalous elliptic curves," *Commentarii Mathematici Universitatis Sancti Pauli*, vol. 47, no. 1, pp. 81–92, 1998.

[11] N. P. Smart, "The discrete logarithm problem on elliptic curves of trace one," *Journal of Cryptology*, vol. 12, no. 3, pp. 193–196, 1999.

[12] A. J. Menezes, T. Okamoto, and S. A. Vanstone, "Reducing elliptic curve logarithms to logarithms in a finite field," *IEEE Transactions on Information Theory*, vol. 39, no. 5, pp. 1639–1646, 1993.

[13] G. Frey and H.-G. Rück, "A remark concerning m-divisibility and the discrete logarithm in the divisor class group of curves," *Mathematics of Computation*, vol. 62, no. 206, pp. 865–874, 1994.

[14] D. J. Bernstein and T. Lange, "Computing small discrete logarithms faster," in *Progress in Cryptology – INDOCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7668. Springer, 2012, pp. 317–338.

[15] ——, "Non-uniform cracks in the concrete: the power of free precomputation," in *Advances in Cryptology – ASIACRYPT 2013*, ser. Lecture Notes in Computer Science, vol. 8270. Springer, 2013, pp. 321–340.

[16] L. Adleman, "A subexponential algorithm for the discrete logarithm problem with applications to cryptography," in *20th Annual Symposium on Foundations of Computer Science*. IEEE, 1979, pp. 55–60.

[17] I. Semaev, "Summation polynomials and the discrete logarithm problem on elliptic curves," Cryptology ePrint Archive, Paper 2004/031, 2004. [Online]. Available: https://eprint.iacr.org/2004/031

[18] P. Gaudry, "Index calculus for abelian varieties of small dimension and the elliptic curve discrete logarithm problem," *Journal of Symbolic Computation*, vol. 44, no. 12, pp. 1690–1702, 2009.

[19] C. Diem, "On the discrete logarithm problem in class groups of curves," *Mathematics of Computation*, vol. 80, no. 273, pp. 443–475, 2011.

[20] ——, "On the discrete logarithm problem in elliptic curves," *Compositio Mathematica*, vol. 147, pp. 75–104, 2011.

[21] ——, "On the discrete logarithm problem in elliptic curves II," *Algebra & Number Theory*, vol. 7, no. 6, pp. 1281–1323, 2013.

[22] Anthropic, "A $2^{o(n)}$ time algorithm for 3-SAT," Preprint, hexagonmath.org/pdf/2610.00577v1; Lean development in github.com/anthropics/formal-math, folder eth, 2026, written by Claude. Dated 26 October 2026; deposited at hexagonmath.org 27 October 2026.

[23] OpenAI, "The Exponential Time Hypothesis is false," Preprint, github.com/openai/math, folder preprints, 2026, dated 25 October 2026; released 26 October 2026.

[24] I. Rudich and L.-M. Rousseau, "Deterministic factoring in time $N^{o(1)}$," Preprint, Oct. 2026, polytechnique Montréal. Lean development registered in the Palomar registry as PALOMAR-2026-10-28-000015.

[25] A. Amadori, F. Pintore, and M. Sala, "On the discrete logarithm problem for prime-field elliptic curves," *Finite Fields and Their Applications*, vol. 51, pp. 168–182, 2018.

[26] A. Schönhage and V. Strassen, "Schnelle Multiplikation großer Zahlen," *Computing*, vol. 7, no. 3–4, pp. 281–292, 1971.

[27] S. DeSalvo and I. Pak, "Log-concavity of the partition function," *The Ramanujan Journal*, vol. 38, no. 1, pp. 61–73, 2015.

[28] D. R. Kohel and I. E. Shparlinski, "On exponential sums and group generators for elliptic curves over finite fields," in *Algorithmic Number Theory (ANTS-IV)*, ser. Lecture Notes in Computer Science, vol. 1838. Springer, 2000, pp. 395–404.

[29] A. Weil, "On some exponential sums," *Proceedings of the National Academy of Sciences of the USA*, vol. 34, pp. 204–207, 1948.

[30] R. Schoof, "Elliptic curves over finite fields and the computation of square roots mod p," *Mathematics of Computation*, vol. 44, no. 170, pp. 483–494, 1985.

[31] H. W. Lenstra, Jr. and C. Pomerance, "A rigorous time bound for factoring integers," *Journal of the American Mathematical Society*, vol. 5, no. 3, pp. 483–516, 1992.

[32] D. H. Wiedemann, "Solving sparse linear equations over finite fields," *IEEE Transactions on Information Theory*, vol. 32, no. 1, pp. 54–62, 1986.

[33] J. B. Rosser and L. Schoenfeld, "Approximate formulas for some functions of prime numbers," *Illinois Journal of Mathematics*, vol. 6, pp. 64–94, 1962.

[34] L. Guth and J. Maynard, "New large value estimates for Dirichlet polynomials," arXiv:2405.20552, 2024. [Online]. Available: https://arxiv.org/abs/2405.20552

[35] L. Chen, W. Eberly, E. Kaltofen, B. D. Saunders, W. J. Turner, and G. Villard, "Efficient matrix preconditioners for black box linear algebra," *Linear Algebra and its Applications*, vol. 343–344, pp. 119–146, 2002.

[36] S. C. Pohlig and M. E. Hellman, "An improved algorithm for computing logarithms over $GF(p)$ and its cryptographic significance," *IEEE Transactions on Information Theory*, vol. 24, no. 1, pp. 106–110, 1978.

[37] J. L. Massey, "Shift-register synthesis and BCH decoding," *IEEE Transactions on Information Theory*, vol. 15, no. 1, pp. 122–127, 1969.

[38] J. D. Dixon, "Exact solution of linear equations using p-adic expansions," *Numerische Mathematik*, vol. 40, no. 1, pp. 137–141, 1982.

[39] P. S. L. M. Barreto and M. Naehrig, "Pairing-friendly elliptic curves of prime order," in *Selected Areas in Cryptography – SAC 2005*, ser. Lecture Notes in Computer Science, vol. 3897. Springer, 2006, pp. 319–331.

[40] J. H. Silverman, "The xedni calculus and the elliptic curve discrete logarithm problem," *Designs, Codes and Cryptography*, vol. 20, no. 1, pp. 5–40, 2000.

[41] M. J. Jacobson, N. Koblitz, J. H. Silverman, A. Stein, and E. Teske, "Analysis of the xedni calculus attack," *Designs, Codes and Cryptography*, vol. 20, no. 1, pp. 41–64, 2000.

[42] J.-C. Faugère, L. Perret, C. Petit, and G. Renault, "Improving the complexity of index calculus algorithms in elliptic curves over binary fields," in *Advances in Cryptology – EUROCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7237. Springer, 2012, pp. 27–44.

[43] C. Petit and J.-J. Quisquater, "On polynomial systems arising from a Weil descent," in *Advances in Cryptology – ASIACRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7658. Springer, 2012, pp. 451–466.

[44] M.-D. A. Huang, M. Kosters, and S. L. Yeo, "Last fall degree, HFE, and Weil descent attacks on ECDLP," in *Advances in Cryptology – CRYPTO 2015*, ser. Lecture Notes in Computer Science, vol. 9215. Springer, 2015, pp. 581–600.

[45] J.-C. Faugère, P. Gaudry, L. Huot, and G. Renault, "Using symmetries in the index calculus for elliptic curves discrete logarithm," *Journal of Cryptology*, vol. 27, no. 4, pp. 595–635, 2014.

[46] A. Joux and V. Vitse, "Cover and decomposition index calculus on elliptic curves made practical," in *Advances in Cryptology – EUROCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7237. Springer, 2012, pp. 9–26.

[47] ——, "Elliptic curve discrete logarithm problem over small degree extension fields," *Journal of Cryptology*, vol. 26, no. 1, pp. 119–143, 2013.

[48] I. Semaev, "New algorithm for the discrete logarithm problem on elliptic curves," Cryptology ePrint Archive, Paper 2015/310, 2015. [Online]. Available: https://eprint.iacr.org/2015/310

[49] C. Petit, M. Kosters, and A. Messeng, "Algebraic approaches for the elliptic curve discrete logarithm problem over prime fields," in *Public-Key Cryptography – PKC 2016*, ser. Lecture Notes in Computer Science, vol. 9615. Springer, 2016, pp. 3–18.

[50] A. Mahalanobis and V. Mallick, "A Las Vegas algorithm to solve the elliptic curve discrete logarithm problem," arXiv:1703.07544, 2017, version 3, February 2018.

[51] R. Williams, "Improving exhaustive search implies superpolynomial lower bounds," in *Proceedings of the 42nd ACM Symposium on Theory of Computing (STOC)*, 2010, pp. 231–240.

[52] D. J. Harvey and D. R. Wood, "Treewidth of the line graph of a complete graph," *Journal of Graph Theory*, vol. 79, no. 1, pp. 48–54, 2015, arXiv:1210.8205.

[53] I. L. Markov and Y. Shi, "Simulating quantum computation by contracting tensor networks," *SIAM Journal on Computing*, vol. 38, no. 3, pp. 963–981, 2008.

[54] D. A. Burgess, "On character sums and primitive roots," *Proceedings of the London Mathematical Society*, vol. s3-12, no. 1, pp. 179–192, 1962.

[55] S. Lang and A. Weil, "Number of points of varieties in finite fields," *American Journal of Mathematics*, vol. 76, no. 4, pp. 819–827, 1954.

[56] A. Akavia, "Learning noisy characters, multiplication codes, and cryptographic hardcore predicates," Ph.D. dissertation, Massachusetts Institute of Technology, 2008.

[57] J. Heintz, "Definability and fast quantifier elimination in algebraically closed fields," *Theoretical Computer Science*, vol. 24, no. 3, pp. 239–277, 1983.

[58] A. Cafure and G. Matera, "Improved explicit estimates on the number of solutions of equations over a finite field," *Finite Fields and Their Applications*, vol. 12, no. 2, pp. 155–185, 2006, arXiv:math/0405302.